

Arnav-Containerization

Lab Experiments – Containerization & DevOps

This repository contains all the laboratory experiments performed as part of the **Containerization and DevOps** coursework.

Each experiment provides practical, hands-on exposure to essential DevOps concepts, tools, and industry-standard practices.

The experiments are organised sequentially, building a strong foundation in containerisation, automation, orchestration, and CI/CD workflows.

List of Experiments

The table below provides an overview of the lab experiments included in this repository:

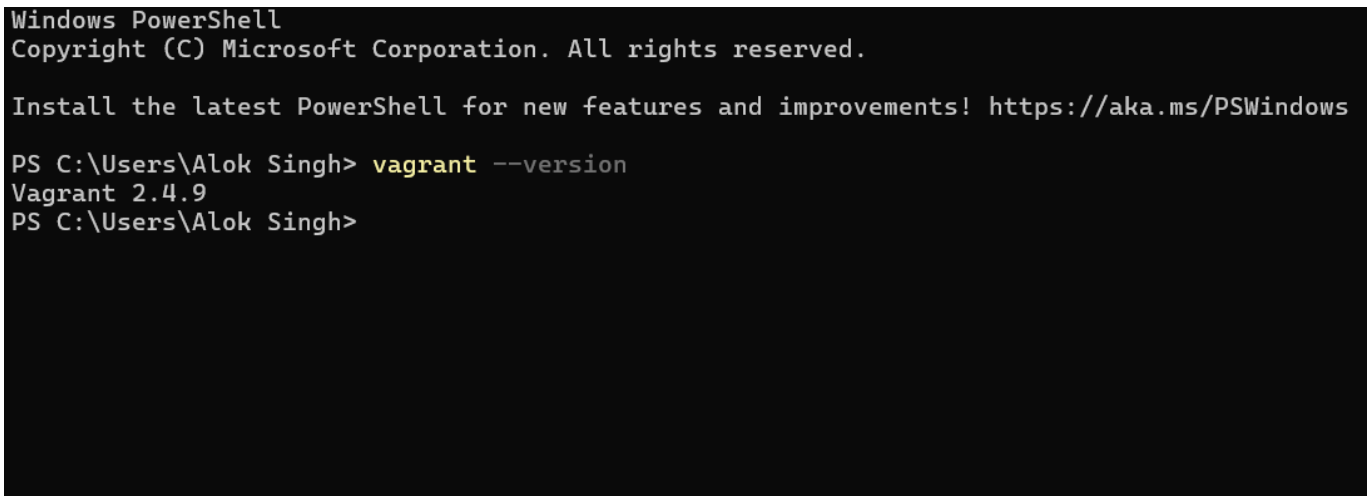
Experiment No.	Title	Result
Experiment-1	VM vs Container - Utilization	The experiment demonstrates that containers are significantly more lightweight and resource-efficient compared to virtual machines
Experiment-2	Docker Installation, Configuration, and Running Images	Docker images were successfully pulled, containers executed, and lifecycle commands performed.
Experiment-3	Docker Deploy NGINX Using Different Base Images and Comparing Image Layers	NGINX was deployed using different base images. The Alpine image was smaller and had fewer layers than Ubuntu, showing that lightweight base images improve efficiency and reduce storage usage.
Experiment-4	Containerizing Applications with Dockerfile	Containerizing prevents from dependency issue or version outages
Experiment-5	Docker - Volumes, Environment Variables, Monitoring & Networks	Essential Docker features for building, configuring, and managing production-ready containerized applications.
Experiment-6	Docker Run vs Docker-Compose	Compose is an easy way to create containers
Experiment-7	CI/CD via Jenkins	Automate Build via Jenkins
Experiment-9	Ansible	Automate Infrastructure Orchestration & Updates

Experiment No.	Title	Result
Experiment-10	Working with SonarQube	Generate Code Quality Report via SonarQube
Experiment-11	Orchestration with Docker Swarm	Compose defines the application. Swarm runs it reliably.
Experiment-12	Kubernetes	Kubernetes automatically decides where to run your app

Experiment 1:-VM vs Container Utilization

Vagrant Version

This screenshot shows the installed Vagrant version on my system.

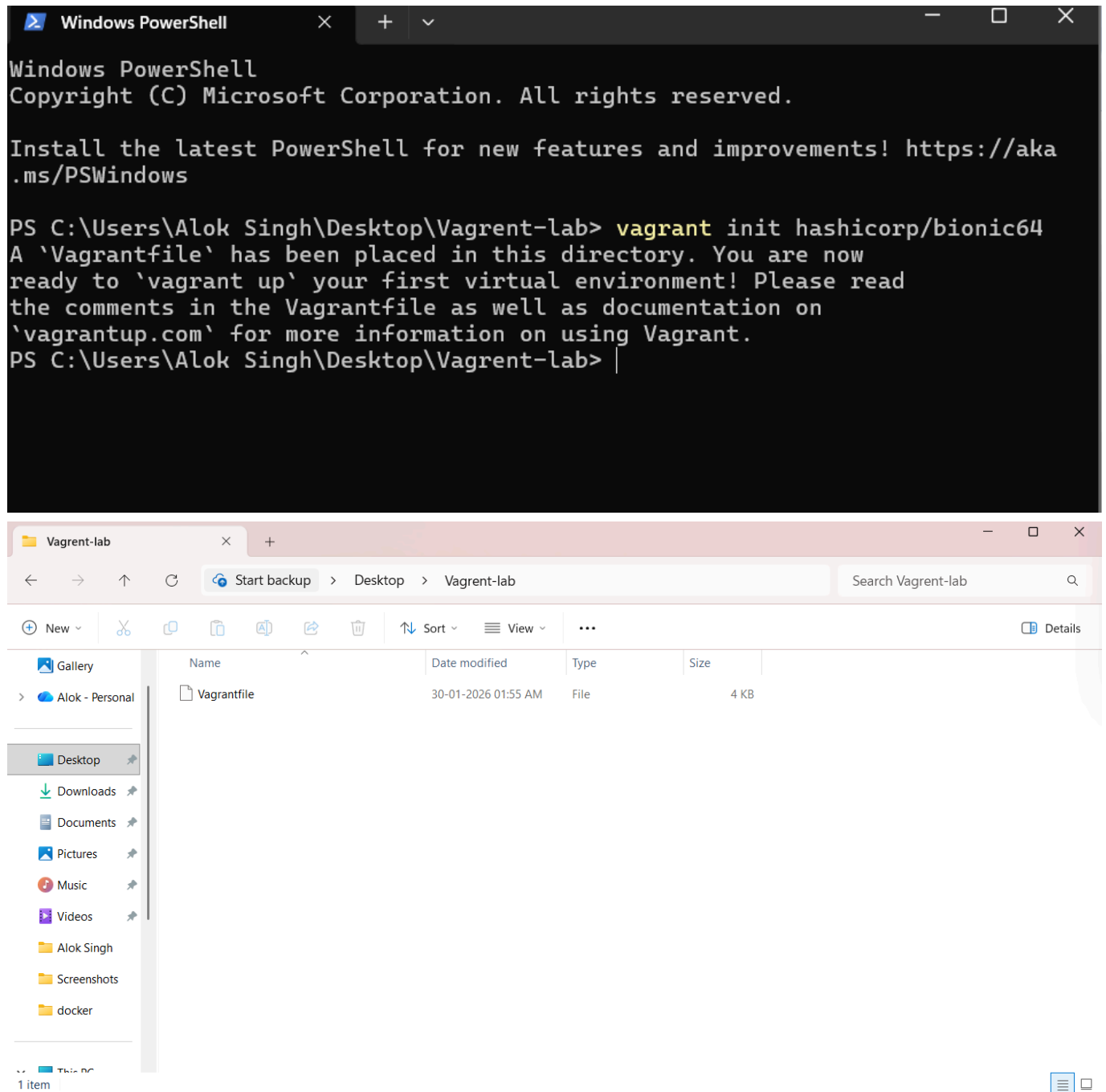


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

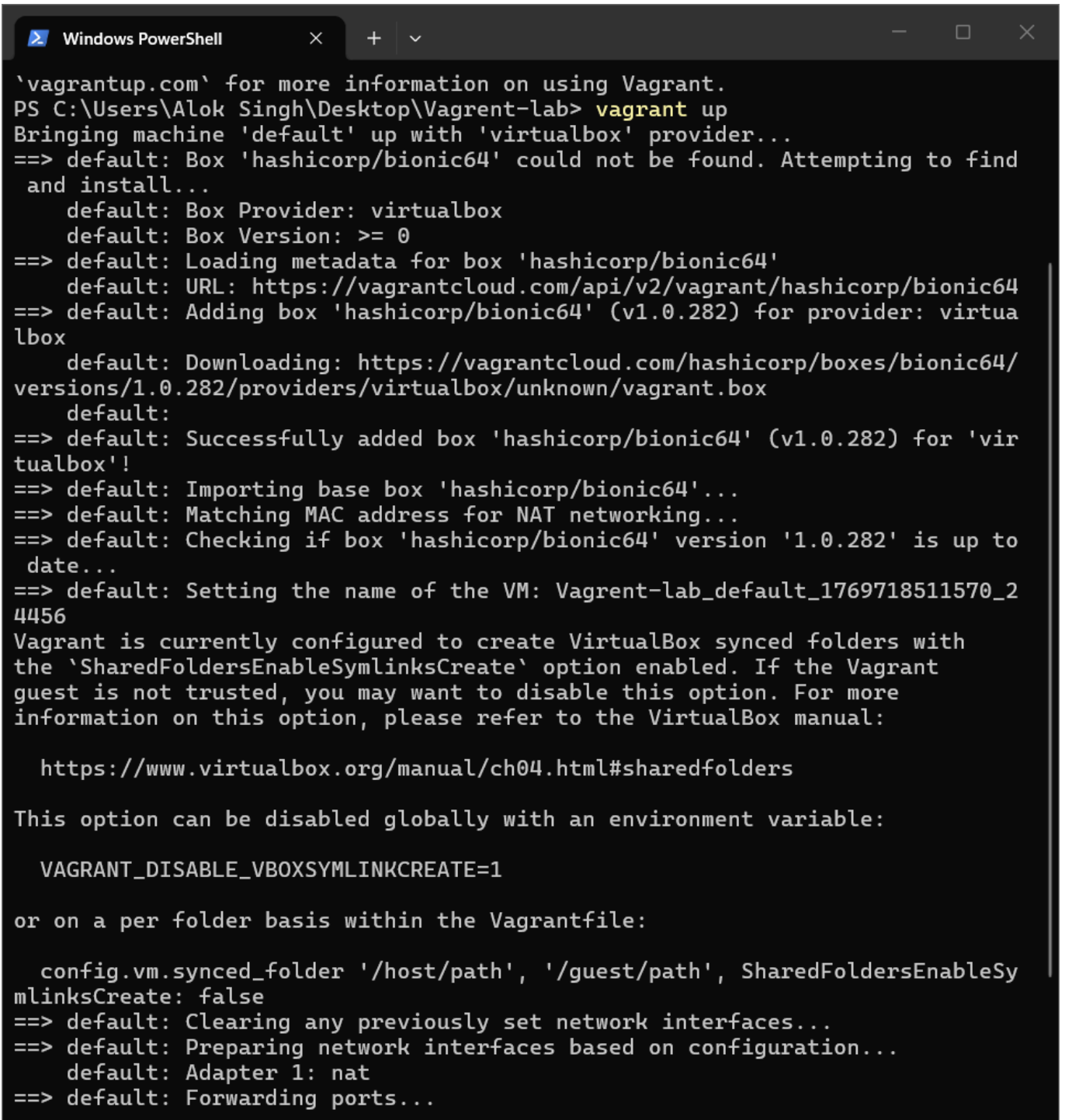
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Alok Singh> vagrant --version
Vagrant 2.4.9
PS C:\Users\Alok Singh>
```

Initializing the vagrant with the ubuntu box



This screenshot show about Vagrant up command



```
'vagrantup.com' for more information on using Vagrant.
PS C:\Users\Alok Singh\Desktop\Vagrent-lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find
and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtua
lbox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/
versions/1.0.282/providers/virtualbox/unknown/vagrant.box
    default:
==> default: Successfully added box 'hashicorp/bionic64' (v1.0.282) for 'vir
tualbox'!
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to
date...
==> default: Setting the name of the VM: Vagrent-lab_default_1769718511570_2
4456
Vagrant is currently configured to create VirtualBox synced folders with
the 'SharedFoldersEnableSymlinksCreate' option enabled. If the Vagrant
guest is not trusted, you may want to disable this option. For more
information on this option, please refer to the VirtualBox manual:

    https://www.virtualbox.org/manual/ch04.html#sharedfolders

This option can be disabled globally with an environment variable:

    VAGRANT_DISABLE_VBOXSYMLINKCREATE=1

or on a per folder basis within the Vagrantfile:

    config.vm.synced_folder '/host/path', '/guest/path', SharedFoldersEnableSy
mlinksCreate: false
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
==> default: Forwarding ports...
```

This screenshot show about Vagrant ssh command

```
PS C:\Users\Alok Singh\Desktop\Vagrent-lab> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information disabled due to load higher than 1.0

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8
s
  just raised the bar for easy, resilient and secure K8s cluster deployment
  .

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

0 packages can be updated.
0 updates are security updates.

vagrant@vagrant:~$ |
```

Installing the Nginx inside the VM

```
sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
```

Verify Nginx

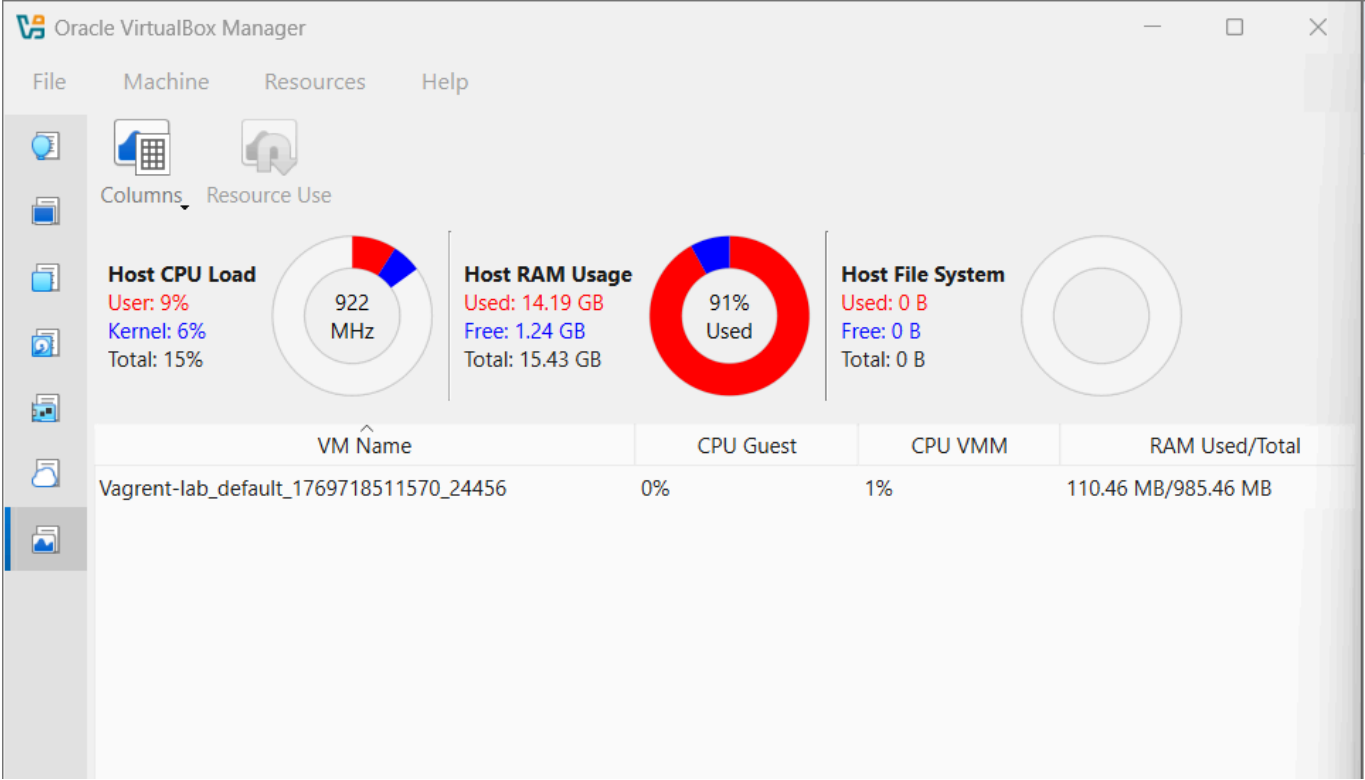
curl Host

```
Setting up fontconfig-config (2.12.6-0ubuntu2) ...
Setting up libnginx-mod-stream (1.14.0-0ubuntu1.11) ...
Setting up libx11-6:amd64 (2:1.6.4-3ubuntu0.4) ...
Setting up libtiff5:amd64 (4.0.9-5ubuntu0.10) ...
Setting up libxpm4:amd64 (1:3.5.12-1ubuntu0.18.04.2) ...
Setting up libfontconfig1:amd64 (2.12.6-0ubuntu2) ...
Setting up libgd3:amd64 (2.2.5-4ubuntu0.5) ...
Setting up libnginx-mod-http-image-filter (1.14.0-0ubuntu1.11) ...
Setting up nginx-core (1.14.0-0ubuntu1.11) ...
Setting up nginx (1.14.0-0ubuntu1.11) ...
Processing triggers for ureadahead (0.100.0-21) ...
Processing triggers for ufw (0.36-0ubuntu0.18.04.1) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@vagrant:~$ |
```

utilization of matrix in running state



stop VM

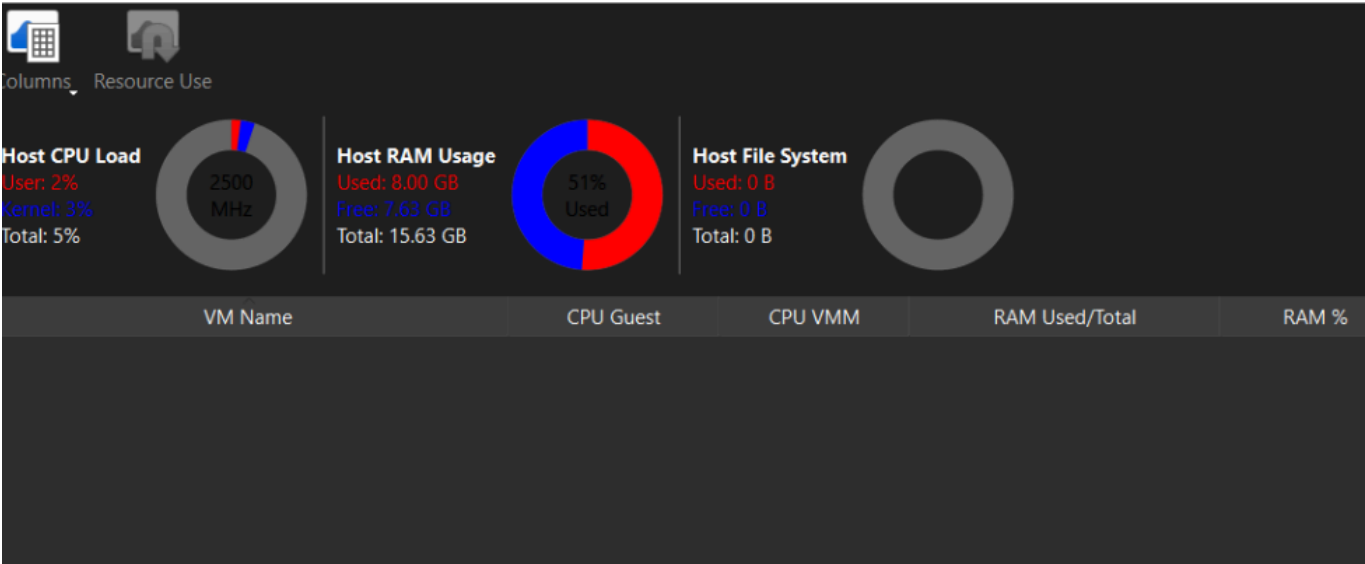
vagrant halt

```
Windows PowerShell
Setting up libnginx-mod-http-image-filter (1.14.0-0ubuntu1.11) ...
Setting up nginx-core (1.14.0-0ubuntu1.11) ...
Setting up nginx (1.14.0-0ubuntu1.11) ...
Processing triggers for ureadahead (0.100.0-21) ...
Processing triggers for ufw (0.36-0ubuntu0.18.04.1) ...
Processing triggers for libc-bin (2.27-3ubuntu1) ...
vagrant@vagrant:~$ sudo systemctl start nginx
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

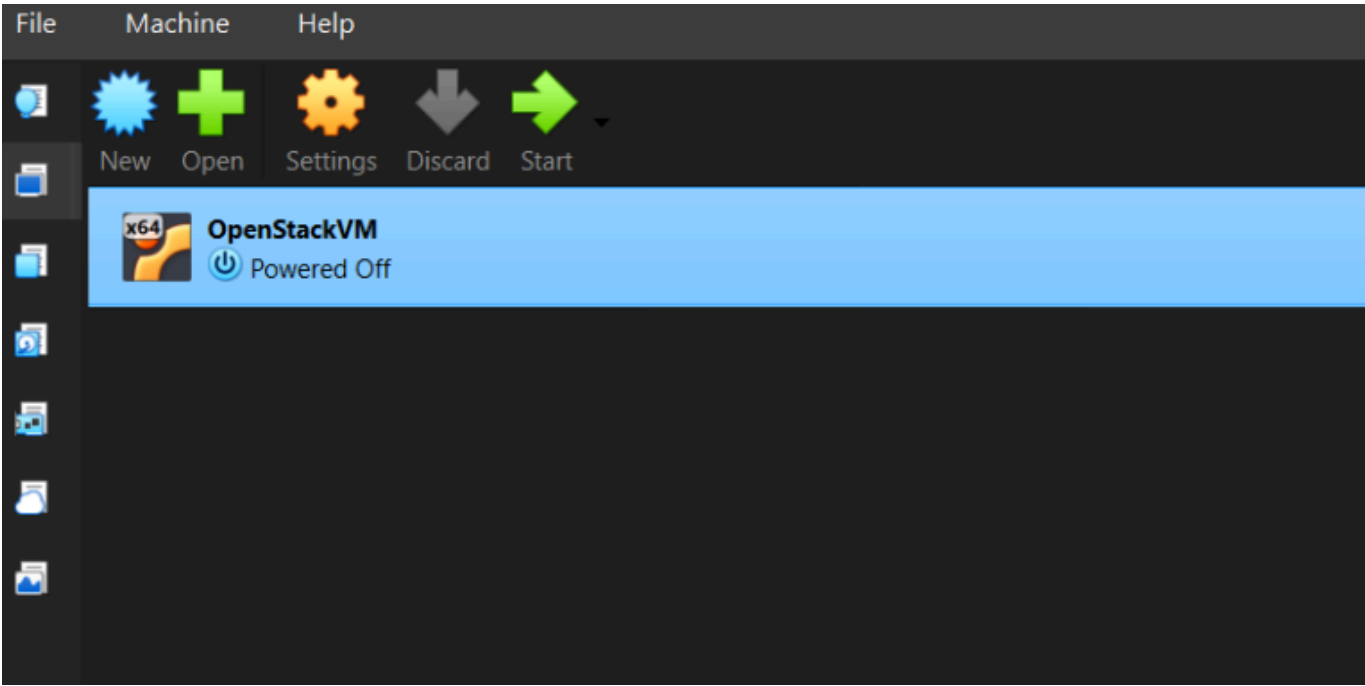
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@vagrant:~$ exit
logout
PS C:\Users\Alok Singh\Desktop\Vagrent-lab> vagrant halt
==> default: Attempting graceful shutdown of VM...
PS C:\Users\Alok Singh\Desktop\Vagrent-lab> vagrant destroy
    default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
PS C:\Users\Alok Singh\Desktop\Vagrent-lab> |
```

Utilization Matrix in stop state



Vagrant destroy



Experiment 2:- Docker Installation, Configuration, and Running Images

Step 1: Pull Image

```
docker --version
```

```
This message is shown once a day. To disable it please create the
/home/arnav/.hushlogin file.
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ |
```

Step 2: Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ sudo docker run -d -p 8080:80 nginx
[sudo] password for arnav:
e4d9f84a6cf6b62fc588e5457c57cceaaf839897ab3af668969f2ac69609d86d3
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ |
```

Step 3: Verify Running Containers

```
docker ps
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
e4d9f84a6cf6   nginx    "/docker-entrypoint..." 17 seconds ago Up 16 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/t
cp_compassionate_ganguly
```

Step 4: Stop Container

```
docker stop <container_id>
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker stop e4d
e4d
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAME
S
e4d9f84a6cf6   nginx    "/docker-entrypoint..." 55 seconds ago Exited (0) 9 seconds ago      comp
assionante_ganguly
24c341e47c63   myrepo/java-app:1.0    "bash"          20 hours ago   Exited (0) 20 hours ago      dete
rmined_hoover
ed9681e0f524   ubuntu:22.04          "bash"          20 hours ago   Exited (0) 20 hours ago      java
_lab
2fd9e059fa89   nginx    "/docker-entrypoint..." 2 days ago     Exited (0) 2 days ago      insp
iring_shamir
9b944c82a2a8   nginx    "/docker-entrypoint..." 2 days ago     Exited (0) 2 days ago      hope
ful_aryabhata
2b0b76aab246   ubuntu    "/bin/bash"        2 days ago     Exited (255) 2 days ago      Arna
v_new
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ |
```

Step 5: Remove Container

```
docker rm <container_id>
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker rm 24c
24c
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
e4d9f84a6cf6   nginx         "/docker-entrypoint..." 9 minutes ago  Exited (0)   8 minutes ago
ed9681e0f524   ubuntu:22.04  "bash"                  20 hours ago  Exited (0)   20 hours ago
2fd9e059fa89   nginx         "/docker-entrypoint..." 2 days ago    Exited (0)   2 days ago
9b944c82a2a8   nginx         "/docker-entrypoint..." 2 days ago    Exited (0)   2 days ago
2b0b76aab246   ubuntu        "/bin/bash"              2 days ago    Exited (255) 2 days ago
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
myrepo/java-app 1.0       e7fbb5c463dc  20 hours ago  774MB
myapp          latest    de289fd12b0a  2 days ago    78.1MB
ubuntu         latest    493218ed0f40  2 weeks ago   78.1MB
nginx          latest    4af177a024eb  2 weeks ago   161MB
ubuntu         22.04     65c77cbc27c2  3 weeks ago   77.9MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ |
```

Step 5: Remove Image

```
docker rmi nginx
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker rm c9a
c9a
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker ps-a
docker: unknown command: docker ps-a

Run 'docker --help' for more information
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
ed9681e0f524   ubuntu:22.04  "bash"                  20 hours ago  Exited (0)   20 hours ago  java_lab
2fd9e059fa89   nginx         "/docker-entrypoint..." 2 days ago    Exited (0)   2 days ago    inspiring_shamir
9b944c82a2a8   nginx         "/docker-entrypoint..." 2 days ago    Exited (0)   2 days ago    hopeful_aryabhata
2b0b76aab246   ubuntu        "/bin/bash"              2 days ago    Exited (255) 2 days ago    Arnav_new
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/docker$ |
```

Result Docker images were successfully pulled, containers executed, and lifecycle commands performed.

Experiment 3:- Docker Deploy NGINX Using Different Base Images and Comparing Image Layers

Part 1: Deploy NGINX Using Official Image (Recommended Approach)

Step 1: Pull the Image

```
docker pull nginx:latest
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ docker pull nginx:latest
latest: Pulling from library/nginx
bae5a1799a80: Pull complete
47cd406a84ef: Pull complete
f73400a233fd: Pull complete
46bf3a120c8e: Pull complete
0c8d55a45c0d: Pull complete
4f4efe02d542: Pull complete
7b6cb8ccac7b: Pull complete
2e02dba24409: Download complete
a5d78d617315: Download complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Step 2: Run the Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ docker run -d --name nginx-official -p 8080:80 nginx
b49482525c7b42df3effb6a91712f9674ac96c96f841abfb470578ba3879a170
```

Step 3: Verify

```
curl http://localhost:8080
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
```

You should see the **NGINX welcome page**.

Key Observations

```
docker images nginx
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ docker images nginx
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
nginx        latest    341bf0f3ce6c   2 days ago    240MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ |
```

- Image is **pre-optimized**
- Minimal configuration required
- Uses Debian-based OS internally

Part 2: Custom NGINX Using Ubuntu Base Image

Step 1: Create Dockerfile

```
FROM ubuntu:22.04

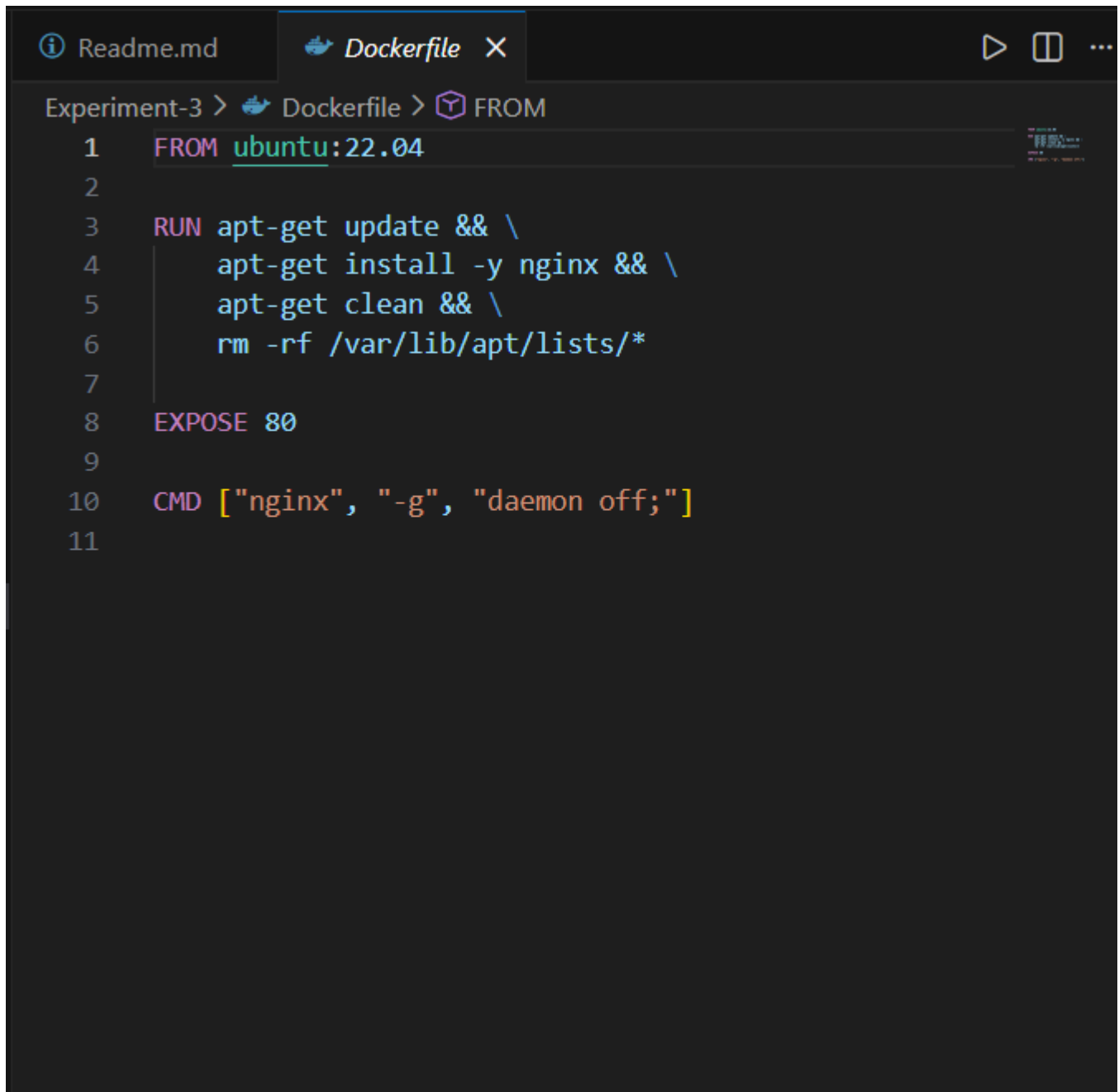
RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-ubuntu .
```



```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && \
4     apt-get install -y nginx && \
5     apt-get clean && \
6     rm -rf /var/lib/apt/lists/*
7
8 EXPOSE 80
9
10 CMD ["nginx", "-g", "daemon off;"]
11
```

```
arnav@LAPTOP-9T1CA4LD: /n × + ▾
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-3$ docker build -t nginx-ubuntu .
[+] Building 6.1s (6/6) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile             0.0s
=> => transferring dockerfile: 228B                             0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04 5.7s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2a 0.0s
=> => resolve docker.io/library/ubuntu:22.04@sha256:c7eb020043d8fc2a 0.0s
=> CACHED [2/2] RUN apt-get update && apt-get install -y nginx & 0.0s
=> exporting to image                                           0.1s
=> => exporting layers                                           0.0s
=> => exporting manifest sha256:b7ba6261f9309da4a91042e2733b7234584b 0.0s
=> => exporting config sha256:0981f9db7baf37b0228602ac8e385900c0abe3 0.0s
=> => exporting attestation manifest sha256:3a10469f5f270c2e74ac12f2 0.0s
=> => exporting manifest list sha256:b430157fc47ddfe22988905eb80a9a3 0.0s
=> => naming to docker.io/library/nginx-ubuntu:latest           0.0s
=> => unpacking to docker.io/library/nginx-ubuntu:latest         0.0s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-3$ |
```

Step 3: Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-3$ docker run -d --name nginx-ubuntu
-p 8081:80 nginx-ubuntu
209ca2a161944f9b7a3cc6192b763c40db6834d3fbf181ac3393a9b2f22b8430
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-3$ |
```

Observations

```
docker images nginx-ubuntu
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ docker images nginx-ubuntu
REPOSITORY      TAG          IMAGE ID      CREATED        SIZE
nginx-ubuntu    latest      b430157fc47d  10 minutes ago  199MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ |
```

- Much **larger image size**
- More layers
- Full OS utilities available

Part 3: Custom NGINX Using Alpine Base Image

Step 1: Create Dockerfile

```
FROM alpine:latest

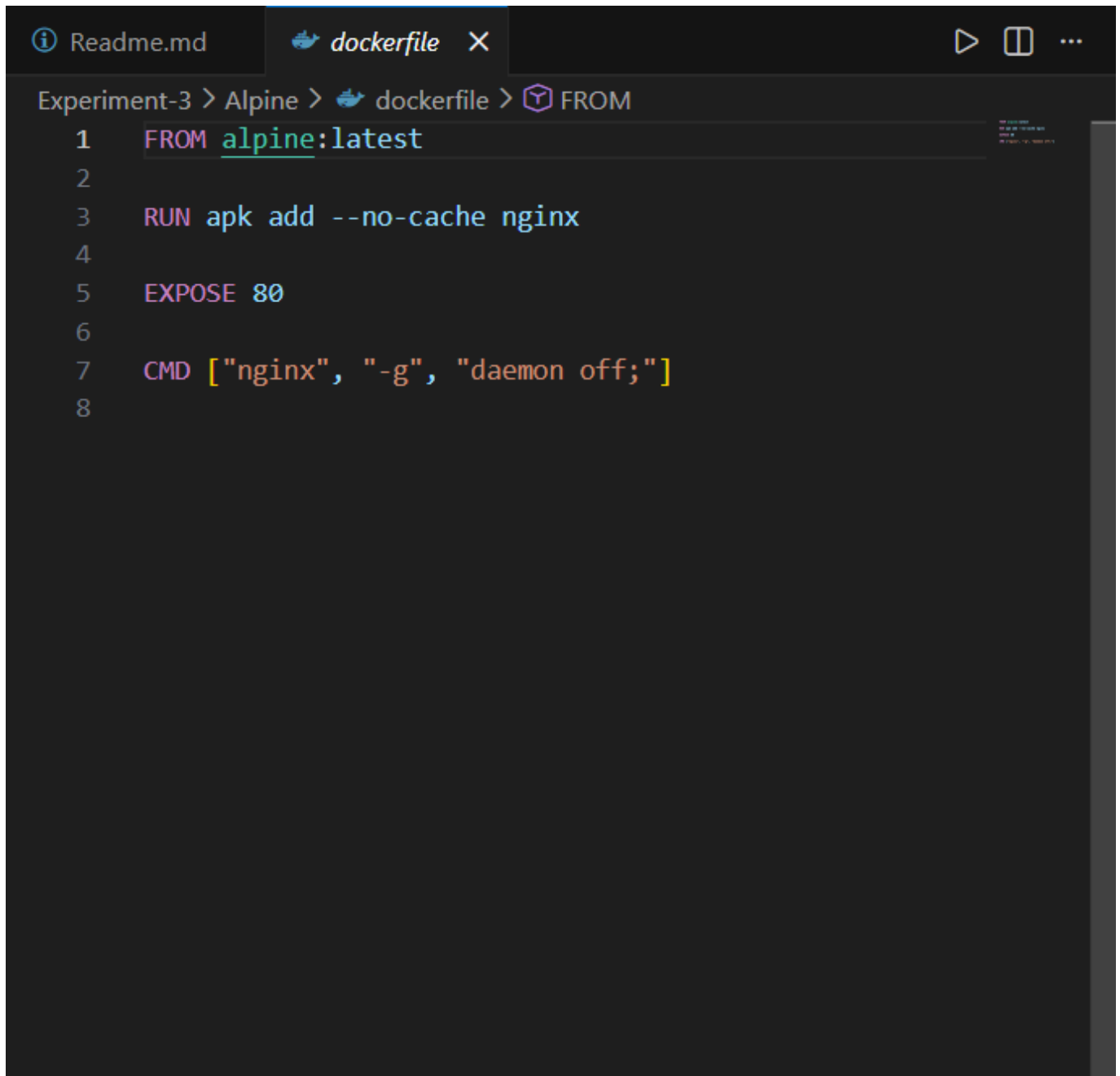
RUN apk add --no-cache nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 2: Build Image

```
docker build -t nginx-alpine .
```



The screenshot shows a code editor with a dark theme. At the top, there are tabs for 'Readme.md' and 'dockerfile'. The 'dockerfile' tab is active. The breadcrumb navigation shows 'Experiment-3 > Alpine > dockerfile > FROM'. The code in the editor is a Dockerfile for nginx-alpine, with line numbers 1 through 8 on the left. The code is: 1 FROM alpine:latest, 2, 3 RUN apk add --no-cache nginx, 4, 5 EXPOSE 80, 6, 7 CMD ["nginx", "-g", "daemon off;"], 8. There is a small Docker logo icon in the top right corner of the editor area.

```
1 FROM alpine:latest
2
3 RUN apk add --no-cache nginx
4
5 EXPOSE 80
6
7 CMD ["nginx", "-g", "daemon off;"]
8
```

Step 3: Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
8c7e014ff79311bce4ce5e30de1ed0700ca5d9996d128e8fd9de6d0475d44ce
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ |
```

Observations

```
docker images nginx-alpine
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ docker images nginx-alpine
REPOSITORY      TAG          IMAGE ID      CREATED        SIZE
nginx-alpine    latest      390eeeb66c73  4 minutes ago  16MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ |
```

- Extremely **small image**
- Fewer packages
- Faster pull and startup time

Part 4: Image Size and Layer Comparison

Compare Sizes

```
docker images | grep nginx
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ docker images | grep nginx
nginx-alpine    latest      390eeeb66c73  5 minutes ago  16MB
nginx-ubuntu    latest      b430157fc47d  17 minutes ago 199MB
nginx           latest      341bf0f3ce6c  2 days ago    240MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ |
```

Typical result (approx):

Image Type	Size
nginx:latest	161 MB
nginx-ubuntu	134 MB
nginx-alpine	10.4 MB

Inspect Layers

```
docker history nginx
docker history nginx-ubuntu
docker history nginx-alpine
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$ docker history nginx
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
341bf0f3ce6c   2 days ago      CMD ["nginx" "-g"  0B         buildkit.dockerfile.v0
<missing>      2 days ago      STOPSIGNAL SIGQUIT 0B         buildkit.dockerfile.v0
<missing>      2 days ago      EXPOSE map[80/tcp: 0B         buildkit.dockerfile.v0
<missing>      2 days ago      ENTRYPOINT ["/dock 0B         buildkit.dockerfile.v0
<missing>      2 days ago      COPY 30-tune-worker 16.4kB     buildkit.dockerfile.v0
<missing>      2 days ago      COPY 20-envsubst-on 12.3kB     buildkit.dockerfile.v0
<missing>      2 days ago      COPY 15-local-resol 12.3kB     buildkit.dockerfile.v0
<missing>      2 days ago      COPY 10-listen-on-i 12.3kB     buildkit.dockerfile.v0
<missing>      2 days ago      COPY docker-entryp 8.19kB     buildkit.dockerfile.v0
<missing>      2 days ago      RUN /bin/sh -c set 86.6MB     buildkit.dockerfile.v0
<missing>      2 days ago      ENV DYNPKG_RELEASE 0B         buildkit.dockerfile.v0
<missing>      2 days ago      ENV PKG_RELEASE=1~ 0B         buildkit.dockerfile.v0
<missing>      2 days ago      ENV ACME_VERSION=0 0B         buildkit.dockerfile.v0
<missing>      2 days ago      ENV NJS_RELEASE=1~ 0B         buildkit.dockerfile.v0
<missing>      2 days ago      ENV NJS_VERSION=0. 0B         buildkit.dockerfile.v0
<missing>      2 days ago      ENV NGINX_VERSION= 0B         buildkit.dockerfile.v0
<missing>      2 days ago      LABEL maintainer=NG 0B         buildkit.dockerfile.v0
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3/Alpine$

```

Observations:

- Ubuntu has many filesystem layers
- Alpine has minimal layers
- Official NGINX image is optimized but heavier than Alpine

Part 5: Functional Tasks Using NGINX

Task 1: Serve Custom HTML Page

```

mkdir html
echo "<h1>Arnav-500121412</h1>" > html/index.html

```

Run:

```

docker run -d \
  -p 8083:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx

```



```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ docker run -d \
  -p 8083:80 \
  -v "$(pwd)/html:/usr/share/nginx/html" \
  nginx
3d46200916fbfb356fcd5491e12cf6d89f3ee5dd97a6cd8279217f9f7735ee05
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-3$ |
```



Arnav - 500121412

Task 2: Reverse Proxy (Conceptual)

NGINX can:

- Forward traffic to backend services
- Load balance multiple containers
- Terminate SSL

Example use cases:

- Frontend for microservices
- API gateway
- Static file server

Part 6: Comparison Summary

Image Comparison Table

Feature	Official NGINX	Ubuntu + NGINX	Alpine + NGINX
Image Size	Medium	Large	Very Small
Ease of Use	Very Easy	Medium	Medium
Startup Time	Fast	Slow	Very Fast
Debugging Tools	Limited	Excellent	Minimal
Security Surface	Medium	Large	Small
Production Ready	Yes	Rarely	Yes

Part 7: When to Use What

Use Official NGINX Image When:

- Production deployment
- Standard web hosting
- Reverse proxy / load balancer

Use Ubuntu-Based Image When:

- Learning Linux + NGINX internals
- Heavy debugging needed
- Custom system-level dependencies

Use Alpine-Based Image When:

- Microservices
- CI/CD pipelines
- Cloud and Kubernetes workloads

 [PREVIOUS](#)

 [HOME](#)

Experiment 4: Docker Essentials

Dockerfile .dockerignore tagging publishing

Containerizing Applications with Dockerfile

Step-1: Create a Directory for Python application

```
mkdir my-flask-app
cd my-flask-app
```

```
PS C:\Users\Alok Singh\Desktop\Vagrent-lab\Containerization-and-devops\Cloud Lab\Experiment-4> wsl
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4$ mkdir my-flask-app
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-2:- Create an `app.py`:

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
GNU nano 7.2 app.py *
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from Docker!"

@app.route('/health')
def health():
    return "OK"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step-3:- Create requirements.txt:

```
Flask==2.3.3
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ cat requirements.txt
Flask==2.3.3
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-4: Create Dockerfile

```
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
CMD ["python", "app.py"]
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Usearna
v@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ nano dockerfile
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ cat dockerfile
# Use Python base image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy requirements file
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY app.py .

# Expose port
EXPOSE 5000

# Run the application
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |

```

Step-5:- Create a **.dockerignore**:

```

# Python files
__pycache__/
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
test_*.py

```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Usearna
v@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ nano .dockerignore
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ cat .dockerignore
# Python files
__pycache__/*
*.pyc
*.pyo
*.pyd

# Environment files
.env
.venv
env/
venv/

# IDE files
.vscode/
.idea/

# Git files
.git/
.gitignore

# OS files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Test files
tests/
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |

```

Why *.dockerignore* is Important

- Prevents unnecessary files from being copied
- Reduces image size
- Improves build speed
- Increases security

Step-6: Build image from Dockerfile

```
docker build -t my-flask-app .
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker build -t my-flask-
app .
[+] Building 13.1s (10/10) FINISHED                                docker:default
=> [internal] load build definition from dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 2.8s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 273B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acdale13cf1731b1b 0.1s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acdale13cf1731b1b 0.1s
=> [internal] load build context                                  0.2s
=> => transferring context: 321B                                     0.1s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> [3/5] COPY requirements.txt .                                   0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt       8.0s
=> [5/5] COPY app.py .                                           0.1s
=> exporting to image                                              1.3s
=> => exporting layers                                              0.7s
=> => exporting manifest sha256:585ff412dfdc8662ec24dcf35cbd6c6c98c3c602e40e165d8ad4eefdcce26b6 0.0s
=> => exporting config sha256:17efc573f65f52c9011662b24fd528799d94807ca3f52603722ca69b76e75692 0.0s
=> => exporting attestation manifest sha256:ec148c27e548073dca692fda1fff90411b2bbe7750260b4676e026b8d60d98be 0.1s
=> => exporting manifest list sha256:38d13c32090dd10cfb8720e609f2960357bfbe4559a19d442bc9ce7fe389e37e 0.0s
=> => naming to docker.io/library/my-flask-app:latest             0.0s
=> => unpacking to docker.io/library/my-flask-app:latest          0.4s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |

```

Step-7:- Verify Image

```
docker images
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Usearna
v@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-flask-app         latest             38d13c32090d       About a minute ago 200MB
backend-image        latest             ebe9d2e0dfb4       2 weeks ago        187MB
java-app             1.0               283504d1b8d8       3 weeks ago        1.17GB
java-app             2.0               0a964baa45a4       3 weeks ago        1.17GB
postgres-image       latest             31b1e91253e8       4 weeks ago        392MB
nginx                latest             0236ee02dcbc       4 weeks ago        240MB
gcr.io/k8s-minikube/kicbase <none>             eb4fec00e8ad       5 weeks ago        1.94GB
gcr.io/k8s-minikube/kicbase v0.0.50           b97074569ae9       5 weeks ago        1.94GB
ubuntu               latest             d1e2e92c075e       6 weeks ago        119MB
nginx-alpine         latest             390eeeb66c73       7 weeks ago        16MB
nginx-ubuntu         latest             b430157fc47d       7 weeks ago        199MB
alpine               latest             25109184c71b       2 months ago       13.1MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-8: Add Tag in Image

```
docker build -t my-flask-app:1.0 .
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Usearna
v@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker build -t my-flask-app:
1.0
[+] Building 2.0s (10/10) FINISHED                                docker:default
=> [internal] load build definition from dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.5s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 273B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 63B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                      0.0s
=> exporting to image                                              0.1s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:585ff412dfdc8662ec24dcf35cbd6c6c90c 0.0s
=> => exporting config sha256:17efc573f65f52c9011662b24fd528799d9480 0.0s
=> => exporting attestation manifest sha256:a7540e9c86ef4a49708a1732 0.0s
=> => exporting manifest list sha256:97d113c05f66725dcd1d7ed3a33dbee5 0.0s
=> => naming to docker.io/library/my-flask-app:1.0                 0.0s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-9:- Add multiple tags

```
docker build -t my-flask-app:latest -t my-flask-app:1.0 .
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Usearna
v@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker build -t my-flask-app:
latest -t my-flask-app:1.0 .
[+] Building 1.2s (10/10) FINISHED                                docker:default
=> [internal] load build definition from dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.6s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 273B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 63B                                       0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                      0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.0s
=> => exporting manifest sha256:585ff412dfdc8662ec24dcf35cbd6c6c90c 0.0s
=> => exporting config sha256:17efc573f65f52c9011662b24fd528799d9480 0.0s
=> => exporting attestation manifest sha256:2f657627bb71d79476a9777b 0.1s
=> => exporting manifest list sha256:483142b0bac0a964e2953794f6dcbef 0.0s
=> => naming to docker.io/library/my-flask-app:latest              0.0s
=> => unpacking to docker.io/library/my-flask-app:latest           0.0s
=> => naming to docker.io/library/my-flask-app:1.0                 0.0s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-10:- Add Tag with custom registry

```
docker build -t username/my-flask-app:1.0 .
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker build -t username/my-flask-app:1.0 .
[*] Building 1.3s (10/10) FINISHED                                docker:default
=> [internal] load build definition from dockerfile                0.1s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.7s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 273B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16b 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 63B                                     0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                            0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                      0.0s
=> exporting to image                                              0.2s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:585ff412dfdc8662ec24dcf35cbd6c6c90c 0.0s
=> => exporting config sha256:17efc573f65f52c9011662b24fd528709d9480 0.0s
=> => exporting attestation manifest sha256:3556239835e8b8b1c81dc71e 0.1s
=> => exporting manifest list sha256:54ada0268f478c2ab1095484592a04d 0.0s
=> => naming to docker.io/username/my-flask-app:1.0              0.0s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-11:- Tag existing image

```
docker tag my-flask-app:latest my-flask-app:v1.0
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker tag my-flask-app:latest my-flask-app:v1.0
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-12: View Image Details

```
docker images
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-flask-app        1.0                483142b0bac0       3 minutes ago      200MB
my-flask-app        latest             483142b0bac0       3 minutes ago      200MB
my-flask-app        v1.0              483142b0bac0       3 minutes ago      200MB
username/my-flask-app 1.0                54ada0268f47       3 minutes ago      200MB
backend-image       latest             ebe9d2e0dfb4       2 weeks ago        187MB
java-app            2.0                0a964baa45a4       3 weeks ago        1.17GB
java-app            1.0                283504d1b8d8       3 weeks ago        1.17GB
postgres-image      latest             31b1e91253e8       4 weeks ago        392MB
nginx               latest             0236ee02dc8c       4 weeks ago        240MB
gcr.io/k8s-minikube/kicbase <none>             eb4fec00e8ad       5 weeks ago        1.94GB
gcr.io/k8s-minikube/kicbase v0.0.50           b97074569ae9       5 weeks ago        1.94GB
ubuntu              latest             d1e2e92c075e       6 weeks ago        119MB
nginx-alpine        latest             390eeeb66c73       7 weeks ago        16MB
nginx-ubuntu        latest             b430157fc47d       7 weeks ago        199MB
alpine              latest             25109184c71b       2 months ago       13.1MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-13:- Show image history


```
docker history my-flask-app
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/
v@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker history my-flask-app
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
483142b0bac0   3 minutes ago   CMD ["python" "app.py"] 0B        buildkit.dockerfile.v0
<missing>      3 minutes ago   EXPOSE [5000/tcp]      0B        buildkit.dockerfile.v0
<missing>      3 minutes ago   COPY app.py . # buildkit 12.3kB    buildkit.dockerfile.v0
<missing>      3 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r... 13.4MB    buildkit.dockerfile.v0
<missing>      3 minutes ago   COPY requirements.txt . # buildkit 12.3kB    buildkit.dockerfile.v0
<missing>      5 weeks ago    WORKDIR /app           8.19kB    buildkit.dockerfile.v0
<missing>      4 months ago   CMD ["python3"]        0B        buildkit.dockerfile.v0
<missing>      4 months ago   RUN /bin/sh -c set -eux; for src in idle3 p... 16.4kB    buildkit.dockerfile.v0
<missing>      4 months ago   RUN /bin/sh -c set -eux; savedAptMark="$(a... 45.7MB    buildkit.dockerfile.v0
<missing>      4 months ago   ENV PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee8... 0B        buildkit.dockerfile.v0
<missing>      4 months ago   ENV PYTHON_VERSION=3.9.25 0B        buildkit.dockerfile.v0
<missing>      4 months ago   ENV GPG_KEY=E3FF2839C048B25C084DEBE9B26995E3... 0B        buildkit.dockerfile.v0
<missing>      4 months ago   RUN /bin/sh -c set -eux; apt-get update; a... 4.94MB    buildkit.dockerfile.v0
<missing>      4 months ago   ENV LANG=C.UTF-8       0B        buildkit.dockerfile.v0
<missing>      4 months ago   ENV PATH=/usr/local/bin:/usr/local/sbin:/usr... 0B        buildkit.dockerfile.v0
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-14:- Inspect image details

```
docker inspect my-flask-app
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/A
lok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker inspect my-flask-app
[
  {
    "Architecture": "amd64",
    "Author": "",
    "Comment": "buildkit.dockerfile.v0",
    "Config": {
      "ArgsEscaped": true,
      "Cmd": [
        "python",
        "app.py"
      ],
      "Entrypoint": null,
      "Env": [
        "PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "LANG=C.UTF-8",
        "GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568",
        "PYTHON_VERSION=3.9.25",
        "PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834"
      ],
      "ExposedPorts": {
        "5000/tcp": {}
      },
      "Labels": null,
      "OnBuild": null,
      "User": "",
      "Volumes": null,
      "WorkingDir": "/app"
    },
    "Created": "2026-03-29T18:19:04.921309411Z",
    "Descriptor": {
      "digest": "sha256:483142b0bac0a964e2953794f6dcbe021ea26f42371241e3631455eb369d138",
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "size": 856
    },
    "DockerVersion": "",
    "GraphDriver": {
      "Data": null,
      "Name": "overlayfs"
    }
  },
]
```

Step-15: Run Container

```
docker run -d -p 5000:5000 --name flask-container my-flask-app
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker run -d -p 5000:500
0 --name flask-container my-flask-app
f631d06ebb58fe33b643e0906176cfc2ab766bd20130e15003833b49c29596fc
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-16:- Test the application

```
curl http://localhost:5000
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ curl http://localhost:5000
Hello from Docker!arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-17:- View running containers

```
docker ps
```

```
lok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
f631d06ebb58   my-flask-app   "python app.py"         49 seconds ago Up 48 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp flask-container
4656bb75b9c6   backend-image  "docker-entrypoint.s..." 2 weeks ago   Up 8 minutes                               backend_api
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-18:- View container logs

```
docker logs flask-container
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/A
lok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker logs flask-container
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-19:- Stop container

```
docker stop flask-container
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker stop flask-container
flask-container
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-20:- Start stopped container

```
docker start flask-container
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker start flask-container
flask-container
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-21:- Remove container

```
docker rm flask-container
```

```
flask-container
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker rm flask-container
Error response from daemon: cannot remove container "flask-container": container is running: stop the container before removing or force remove
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-22:- Remove container forcefully

Since Container is in Running State, so we will remove it forcefully...

```
docker rm -f flask-container
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker rm -f flask-container
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Multi-stage Builds

Why Multi-stage Builds?

- Smaller final image size
- Better security (remove build tools)
- Separate build and runtime environments

Step-23: Create a Simple `Dockerfile.multistage`:

```
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000
```

```
# Run application
CMD ["python", "app.py"]
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ nano Dockerfile.multistage
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ cat Dockerfile.multistage
# STAGE 1: Builder stage
FROM python:3.9-slim AS builder

WORKDIR /app

# Copy requirements
COPY requirements.txt .

# Install dependencies in virtual environment
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# STAGE 2: Runtime stage
FROM python:3.9-slim

WORKDIR /app

# Copy virtual environment from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy application code
COPY app.py .

# Create non-root user
RUN useradd -m -u 1000 appuser
USER appuser

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-24:- Build regular image

```
docker build -t flask-regular .
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker build -t flask-regular .
[+] Building 3.5s (10/10) FINISHED                                docker:default
=> [internal] load build definition from dockerfile               0.1s
=> => transferring dockerfile: 361B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 3.0s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 273B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabb1acda1e13cf1731b1b 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aabb1acda1e13cf1731b1b 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 63B                                     0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                     0.0s
=> exporting to image                                              0.1s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:585ff412dfdc8662ec24dcf35cbd6c6c90c3c602e40e165d8ad4eefdcce26b6 0.0s
=> => exporting config sha256:17efc573f65f52c9011662b24fd528799d94807ca3f52603722ca69b76e75692 0.0s
=> => exporting attestation manifest sha256:e64afbd19b356f92672897b7c94815eafe30c96da8e4585838c9bdfba344b822 0.0s
=> => exporting manifest list sha256:c6e197bcf215a93964eb95c214bdecaea044c759fe1cc0f793a06a1213c276c0 0.0s
=> => naming to docker.io/library/flask-regular:latest            0.0s
=> => unpacking to docker.io/library/flask-regular:latest         0.0s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-25:- Build multi-stage image

```
docker build -f Dockerfile.multistage -t flask-multistage .
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 13.9s (13/13) FINISHED
=> [internal] load build definition from Dockerfile.multistage
=> transferring dockerfile: 675B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [internal] load .dockerignore
=> transferring context: 273B
=> [internal] load build context
=> transferring context: 63B
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acdale13cf1731b1b
=> resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acdale13cf1731b1b
=> CACHED [builder 2/5] WORKDIR /app
=> CACHED [builder 3/5] COPY requirements.txt .
=> [builder 4/5] RUN python -m venv /opt/venv
=> [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt
=> [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv
=> [stage-1 4/5] COPY app.py .
=> [stage-1 5/5] RUN useradd -m -u 1000 appuser
=> exporting to image
=> exporting layers
=> exporting manifest sha256:4e53c879b829dae24f2fe8c218337e534935a39f7a0f43dd3c579d7a5cb0f8be9
=> exporting config sha256:a771777a80ab01405eb381a560c01f988150331f5ba9c0173ad0ccd8c9d0b06d
=> exporting attestation manifest sha256:ada552586d3d9a491c4b455778789e5b87b372e202f155ffb524e00abcba7f58
=> exporting manifest list sha256:2d0327f0fd77b1aala788a0a453963ee7c4488ce3622f75896c8c1490a3f6aad
=> naming to docker.io/library/flask-multistage:latest
=> unpacking to docker.io/library/flask-multistage:latest
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$

```

Step-26:- Compare sizes

```
docker images | grep flask-
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker images | grep flask-
flask-multistage      latest      2d0327f0fd77      18 seconds ago      219MB
my-flask-app          1.0        483142b0bac0      12 minutes ago      200MB
my-flask-app          latest     483142b0bac0      12 minutes ago      200MB
my-flask-app          v1.0      483142b0bac0      12 minutes ago      200MB
flask-regular         latest     c6e197bcf215      12 minutes ago      200MB
username/my-flask-app 1.0        54ada0268f47      12 minutes ago      200MB
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$

```

Publishing to Docker Hub

Step-27:- Login to Docker Hub

```
docker login
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker login
Authenticating with existing credentials... [Username: arnavarnav]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded

```

Step-28:- Tag image for Docker Hub

```

docker tag my-flask-app:latest username/my-flask-app:1.0
docker tag my-flask-app:latest username/my-flask-app:latest

```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker tag my-flask-app:latest username/my-flask-app:1.0
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker tag my-flask-app:latest username/my-flask-app:latest
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$

```

Step-29:- Push to Docker Hub

```
docker push username/my-flask-app:1.0
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker push arnavarnav/my-flask-app:1.0
The push refers to repository [docker.io/arnavarnav/my-flask-app]
38513bd72563: Pushed
8e1483632ee8: Pushed
b3ec39b36ae8: Pushed
fc7443084902: Pushed
6afe72e43cce: Pushed
57b85ae2846e: Pushed
ea56f685404a: Pushed
078ca25eef6e: Pushed
beceff9c922f: Pushed
1.0: digest: sha256:483142b0bac0a964e2953794f6dcbe021ea26f42371241e3631455eb369d138 size: 856
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-30:- Push Latest Image

```
docker push username/my-flask-app:latest
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker push arnavarnav/my-flask-app:latest
The push refers to repository [docker.io/arnavarnav/my-flask-app]
38513bd72563: Layer already exists
fc7443084902: Layer already exists
57b85ae2846e: Layer already exists
078ca25eef6e: Already exists
8e1483632ee8: Layer already exists
6afe72e43cce: Layer already exists
ea56f685404a: Layer already exists
beceff9c922f: Layer already exists
b3ec39b36ae8: Layer already exists
latest: digest: sha256:483142b0bac0a964e2953794f6dcbe021ea26f42371241e3631455eb369d138 size: 856
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-31:- Pull from Docker Hub (on another machine)

```
docker pull username/my-flask-app:latest
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker pull arnavarnav/my-flask-app:latest
latest: Pulling from arnavarnav/my-flask-app
Digest: sha256:483142b0bac0a964e2953794f6dcbe021ea26f42371241e3631455eb369d138
Status: Image is up to date for arnavarnav/my-flask-app:latest
docker.io/arnavarnav/my-flask-app:latest
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Step-32:- Run the pulled image

```
docker run -d -p 5000:5000 username/my-flask-app:latest
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ docker run -d -p 5000:5000 arnavarnav/my-flask-app:latest
3b0aff0e442b65cf4487ae9a36a6c599aef6dc9cf2af7f694089c7b712155e78
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ |
```

Node.js Application Example (Quick Version)**

Step-33:- Create a directory Node.js Application

```
mkdir my-node-app
cd my-node-app
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ mkdir my-node-app
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app$ cd my-node-app
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ |
```

Step-34:- Create a `app.js`:

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ nano app.js
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ cat app.js
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Node.js Docker!');
});

app.get('/health', (req, res) => {
  res.json({ status: 'healthy' });
});

app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ |
```

Step-35:- Create a `package.json`:

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
```



```
"dependencies": {  
  "express": "^4.18.2"  
}
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ nano package.  
json  
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ cat package.j  
son  
{  
  "name": "node-docker-app",  
  "version": "1.0.0",  
  "main": "app.js",  
  "dependencies": {  
    "express": "^4.18.2"  
  }  
}  
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ |
```

Step-36:- Create a Node.js Dockerfile

```
FROM node:18-alpine  
  
WORKDIR /app  
  
COPY package*.json ./  
RUN npm install --only=production  
  
COPY app.js .  
  
EXPOSE 3000  
  
CMD ["node", "app.js"]
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ nano dockerfi  
le  
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ cat dockerfil  
e  
FROM node:18-alpine  
  
WORKDIR /app  
  
COPY package*.json ./  
RUN npm install --only=production  
  
COPY app.js .  
  
EXPOSE 3000  
  
CMD ["node", "app.js"]  
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ |
```

Step-37:- Build image

```
docker build -t my-node-app .
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ docker build
-t my-node-app .
[+] Building 11.4s (11/11) FINISHED                                docker:default
=> [internal] load build definition from dockerfile                0.1s
=> => transferring dockerfile: 182B                                0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine  3.4s
[auth] library/node:pull token for registry-1.docker.io          0.0s
[internal] load .dockerignore                                    0.1s
=> => transferring context: 2B                                      0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.1s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e  0.1s
=> [internal] load build context                                    0.1s
=> => transferring context: 514B                                     0.1s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> [3/5] COPY package*.json ./                                     0.1s
=> [4/5] RUN npm install --only=production                         6.1s
=> [5/5] COPY app.js .                                             0.1s
=> => exporting to image                                           0.9s
=> => exporting layers                                             0.5s
=> => exporting manifest sha256:4c81d726bda961e61d992f318df2ddb04b66048007340235388f1a40e76febff  0.0s
=> => exporting config sha256:6a2fc585f6b5a8ac08211995a84f015657e2981d4923e4eb627a811984146318  0.0s
=> => exporting attestation manifest sha256:ec5261cb806148deec408affac496c25e0a5468245c5ad6cc69790c425e6f7dd  0.0s
=> => exporting manifest list sha256:6777dba64a7a2841315a3e4c728b457f1a551550b0ae045d421683fd4e8bd954  0.0s
=> => naming to docker.io/library/my-node-app:latest              0.0s
=> => unpacking to docker.io/library/my-node-app:latest          0.3s
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ |
```

Step-38:- Run container

```
docker run -d -p 3000:3000 --name node-container my-node-app
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ docker run -d
-p 3000:3000 --name node-container my-node-app
34dbe183b2f935e159bcf1575c268333b557bb03f21831cc98989f576e62a0cf
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ |
```

Step-39:- Test on Terminal

```
curl http://localhost:3000
```

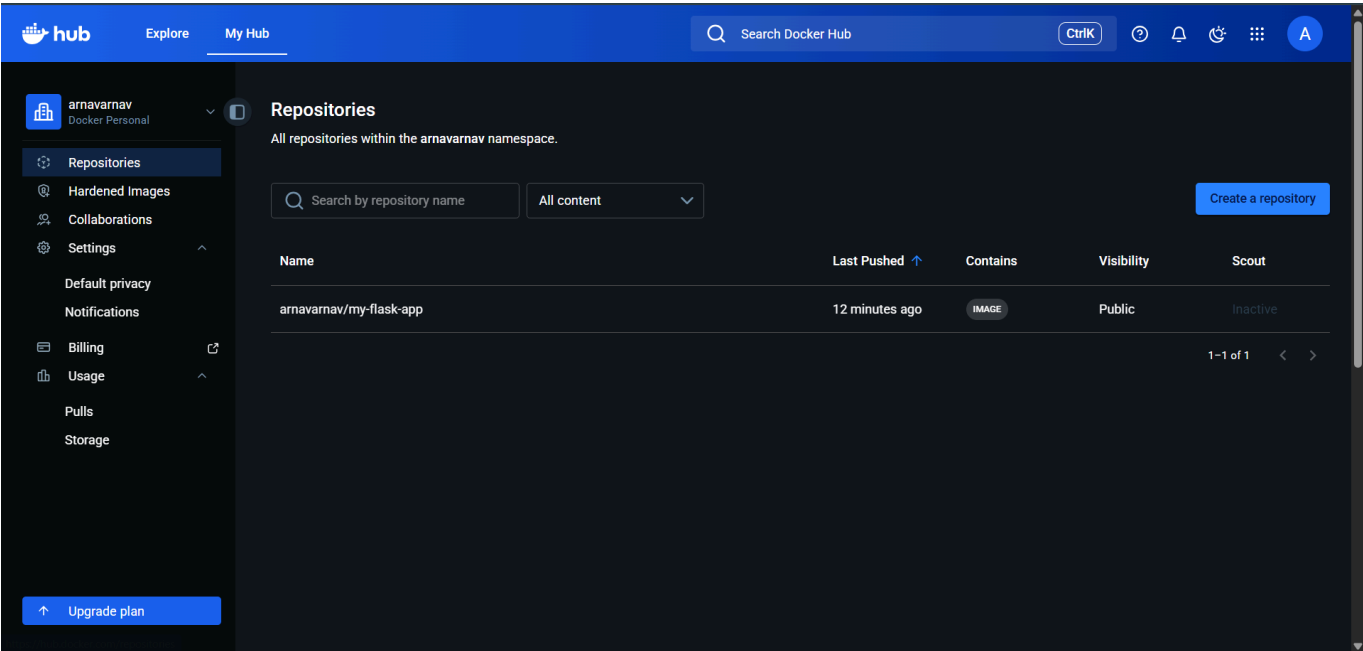
```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/my-node-app$ curl http://l
ocalhost:3000
Hello from Node.js Docker!arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-4/my-flask-app/
my-node-app$ |
```

Step-40:- Test on Browser



DockerHub

View Pushed Image on [DockerHub](#)



Conclusion

Essential Docker Commands Cheatsheet

Command	Purpose	Example
<code>docker build</code>	Build image	<code>docker build -t myapp .</code>
<code>docker run</code>	Run container	<code>docker run -p 3000:3000 myapp</code>
<code>docker ps</code>	List containers	<code>docker ps -a</code>
<code>docker images</code>	List images	<code>docker images</code>
<code>docker tag</code>	Tag image	<code>docker tag myapp:latest myapp:v1</code>
<code>docker login</code>	Login to Dockerhub using username and password or token	<code>`echo "token"</code>
<code>docker push</code>	Push to registry	<code>docker push username/myapp</code>
<code>docker pull</code>	Pull from registry	<code>docker pull username/myapp</code>
<code>docker rm</code>	Remove container	<code>docker rm container-name</code>
<code>docker rmi</code>	Remove image	<code>docker rmi image-name</code>
<code>docker logs</code>	View logs	<code>docker logs container-name</code>
<code>docker exec</code>	Execute command	<code>docker exec -it container-name bash</code>

Common Workflow Summary

Development Workflow:

```
# 1. Create Dockerfile and .dockerignore
# 2. Build image
docker build -t myapp .

# 3. Test locally
docker run -p 8080:8080 myapp

# 4. Tag for production
docker tag myapp:latest myapp:v1.0

# 5. Push to registry
docker push myapp:v1.0
```

Production Workflow:

```
# 1. Pull from registry
docker pull myapp:v1.0
```

```
# 2. Run in production
docker run -d -p 80:8080 --name prod-app myapp:v1.0

# 3. Monitor
docker logs -f prod-app
```

Key Takeaways

1. **Dockerfile** defines how to build your image
 2. **.dockerignore** excludes unnecessary files (important for security and performance)
 3. **Tagging** helps version and organize images
 4. **Multi-stage builds** create smaller, more secure production images
 5. **Docker Hub** allows sharing and distributing images
 6. **Always test** images locally before publishing
-

Cleanup

```
# Remove all stopped containers
docker container prune

# Remove unused images
docker image prune

# Remove everything unused
docker system prune -a
```

[< PREVIOUS](#)[HOME >](#)

Experiment 5: Docker – Volumes, Environment Variables, Monitoring & Networks

Docker – Volumes, Environment Variables, Monitoring & Networks

This experiment demonstrates essential Docker features for building, configuring, and managing production-ready containerized applications – persistent storage, environment configuration, container monitoring, and network communication.

Table of Contents

- [Part 1: Docker Volumes – Persistent Data Storage](#)
 - [Understanding Data Persistence](#)
 - [Volume Types](#)
 - [Practical Volume Examples](#)
 - [Volume Management Commands](#)
 - [Part 2: Environment Variables](#)
 - [Setting Environment Variables](#)
 - [Environment Variables in Applications](#)
 - [Part 3: Docker Monitoring](#)
 - [Basic Monitoring Commands](#)
 - [Container Inspection & Events](#)
 - [Part 4: Docker Networks](#)
 - [Network Types](#)
 - [Network Management](#)
 - [Part 5: Complete Real-World Example](#)
 - [Quick Reference Cheatsheet](#)
 - [Practice Exercises](#)
 - [Cleanup Commands](#)
-

Part 1: Docker Volumes – Persistent Data Storage

Understanding Data Persistence

By default, container data is lost when the container stops or is removed. The example below shows that manually created data survives a container restart but is lost when the container is removed.

```
# Start and write data to a container
docker start -ai test-container
mkdir /data
echo "hello World" > /data/Arnav-500121412.txt
exit

# Restart container – data is still there
```

```
docker start test-container
docker exec test-container cat /data/Arnav-500121412.txt
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker start -ai test-container
root@e1f70ada3bf1:/# mkdir /data
mkdir: cannot create directory '/data': File exists
root@e1f70ada3bf1:/# echo "hello World" > /data/Arnav-500121412.txt
root@e1f70ada3bf1:/# exit
exit
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker start test-container
test-container
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker exec test-container cat /data/Arnav-500121412.txt
hello World
```

Volume Types

1. Anonymous Volumes

Automatically created with a random name.

```
docker run -d -v /app/data --name web1 nginx
docker volume ls
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker run -d -v /app/data --name web1 nginx
9d55064357cb41bb2b42cfff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
9d55064357cb   nginx     "/docker-entrypoint...." 13 seconds ago Up 12 seconds 80/tcp       web1
e1f70ada3bf1   ubuntu   "/bin/bash"              3 weeks ago   Up 14 minutes                test-container
4656bb75b9c6   backend-image "docker-entrypoint.s..." 5 weeks ago   Up 20 minutes                backend_api
49e9553a9eec   postgres-image "docker-entrypoint.s..." 5 weeks ago   Up 20 minutes                postgres_db
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker volume ls
DRIVER      VOLUME NAME
local       6e57832caf61c24a0471e2a9c49240b53fe4f6eb9ce2fcc39968d65954941fd
local       998f620509bb2572b756b86f2c1d627a3bcc885fa446fc794b9089f87abd6cdf
local       devops-container-project_pgdata
local       e093cc0a56fbb7fabf09b4968cbcab28222e5f4644361d3fcfe4f483f18c72f
local       jenkins_data
local       minikube
local       mytestvol
local       sap-volume
```

2. Named Volumes

User-defined, easy to reuse.

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
mydata
49d78a21814e74eid3509e4bd4ea9b9c34945ef13e3554fdb793b0340baeb6ff
DRIVER      VOLUME NAME
local       6e57832caf61c24a0471e2a9c49240b53fe4f6eb9ce2fcc39968d65954941fd
local       998f620509bb2572b756b86f2c1d627a3bcc885fa446fc794b9089f87abd6cdf
local       devops-container-project_pgdata
local       e093cc0a56fbb7fabf09b4968cbcab28222e5f4644361d3fcfe4f483f18c72f
local       jenkins_data
local       minikube
local       mydata
local       mytestvol
local       sap-volume
```

3. Bind Mounts (Host Directory)

Mount a specific host directory into the container.

```
mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
8cc33e15a3f2c80d944eca3d7df5e0dee9ce63df98b73cb27b09bb45a5710612
From Host
```

Practical Volume Examples

Example 1: MySQL with persistent storage

```
docker run -d --name mysql-db -v mysql-data:/var/lib/mysql -e
MYSQL_ROOT_PASSWORD=secret mysql:8.0
docker stop mysql-db && docker rm mysql-db
# New container using the same volume - data is preserved
docker run -d --name new-mysql -v mysql-data:/var/lib/mysql -e
MYSQL_ROOT_PASSWORD=secret mysql:8.0
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker stop mysql-db
mysql-db
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker rm mysql-db
mysql-db
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
c817678ba9c7003ebd3eb62eda461d1175d972927ad28e806749cacbec2761dc
```

Example 2: Web app with custom configuration via bind mount

```
mkdir ~/nginx-config
echo 'server { listen 80; server_name localhost; location / { return 200 "Hello
from mounted config!"; }' > ~/nginx-config/nginx.conf
docker run -d --name nginx-custom -p 8080:80 -v ~/nginx-
config/nginx.conf:/etc/nginx/conf.d/default.conf nginx
curl http://localhost:8080
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ curl http://localhost:8080
Hello from mounted config!arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker run -d \
--name app1 \
-e DEBUG="true" \
-p 3000:3000 \
nginx
249034e39dd63caa23546fcf7ff2268bde14be36d1375ca9e50280364e8cb14d
```

Volume Management Commands

```
docker volume ls          # list all volumes
docker volume create app-volume # create a named volume
docker volume inspect app-volume # inspect volume details
docker volume prune       # remove unused volumes
```



```
docker volume rm <name> # remove a specific volume
docker cp local-file.txt container:/path # copy files into a volume
```

Part 2: Environment Variables

Setting Environment Variables

Method 1: -e flag

```
docker run -d --name app1 -e DEBUG="true" -p 3000:3000 nginx
```

```
Hello from mounted config!arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker run -d \
--name app1 \
-e DEBUG="true" \
-p 3000:3000 \
nginx
249034e39dd63caa23546fcf7ff2268bde14be36d1375ca9e50280364e8cb14d
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
249034e39dd6	nginx	"/docker-entrypoint.s..."	10 seconds ago	Up 9 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	app1
b8bde4bec0a	nginx	"/docker-entrypoint.s..."	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	nginx-custom
c817678ba9c7	mysql:8.0	"/docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	3306/tcp, 33060/tcp	new-mysql
8cc33e15a3f2	nginx	"/docker-entrypoint.s..."	4 minutes ago	Up 4 minutes	80/tcp	web3
49d78a21814e	nginx	"/docker-entrypoint.s..."	5 minutes ago	Up 5 minutes	80/tcp	web2
9d55064357cb	nginx	"/docker-entrypoint.s..."	6 minutes ago	Up 6 minutes	80/tcp	web1
e1f70ada3bf1	ubuntu	"/bin/bash"	3 weeks ago	Up 20 minutes		test-contain
er						
4656bb75b9c6	backend-image	"docker-entrypoint.s..."	5 weeks ago	Up 26 minutes		backend_api
49e9553a9eec	postgres-image	"docker-entrypoint.s..."	5 weeks ago	Up 26 minutes		postgres_db

Method 2: --env-file

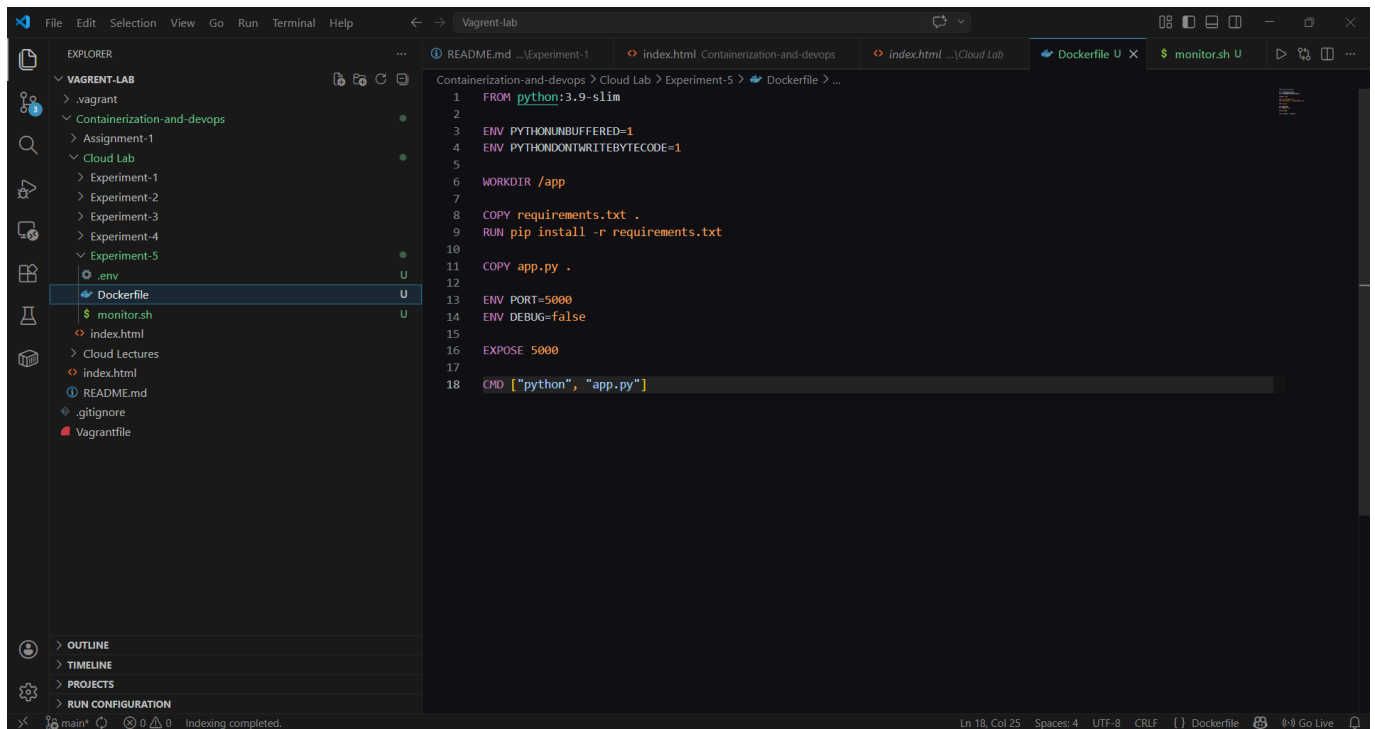
```
echo "DATABASE_HOST=localhost" > .env
echo "DATABASE_PORT=5432" >> .env
echo "API_KEY=secret123" >> .env
docker run -d --env-file .env --name app2 nginx
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
249034e39dd6	nginx	"/docker-entrypoint.s..."	10 seconds ago	Up 9 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	app1
b8bde4bec0a	nginx	"/docker-entrypoint.s..."	About a minute ago	Up About a minute	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	nginx-custom
c817678ba9c7	mysql:8.0	"/docker-entrypoint.s..."	3 minutes ago	Up 3 minutes	3306/tcp, 33060/tcp	new-mysql
8cc33e15a3f2	nginx	"/docker-entrypoint.s..."	4 minutes ago	Up 4 minutes	80/tcp	web3
49d78a21814e	nginx	"/docker-entrypoint.s..."	5 minutes ago	Up 5 minutes	80/tcp	web2
9d55064357cb	nginx	"/docker-entrypoint.s..."	6 minutes ago	Up 6 minutes	80/tcp	web1
e1f70ada3bf1	ubuntu	"/bin/bash"	3 weeks ago	Up 20 minutes		test-contain
er						
4656bb75b9c6	backend-image	"docker-entrypoint.s..."	5 weeks ago	Up 26 minutes		backend_api
49e9553a9eec	postgres-image	"docker-entrypoint.s..."	5 weeks ago	Up 26 minutes		postgres_db

Method 3: In Dockerfile

```
FROM python:3.9-slim
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
ENV PORT=5000
ENV DEBUG=false
EXPOSE 5000
CMD ["python", "app.py"]
```



Environment Variables in Applications (Python Flask example)

app.py

```
import os
from flask import Flask

app = Flask(__name__)

db_host = os.environ.get('DATABASE_HOST', 'localhost')
debug_mode = os.environ.get('DEBUG', 'false').lower() == 'true'

@app.route('/config')
def config():
    return {'db_host': db_host, 'debug': debug_mode}

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Verify environment variables

```
docker exec app1 env
docker exec app2 env
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker exec app1 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=249034e39dd6
DEBUG=true
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker exec app2 env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=66663596a6ce
DATABASE_HOST=localhost
DATABASE_PORT=5432
API_KEY=secret123
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

Part 3: Docker Monitoring

Basic Monitoring Commands

`docker stats` – real-time resource usage

```
docker stats
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker stats
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O
66663596a6ce   app2      0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
249034e39dd6   app1      0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
b8bde4bec00a   nginx-custom  0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
c817678ba9c7   new-mysql  0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
8cc33e15a3f2   web3      0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
49d78a21814e   web2      0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
9d55064357cb   web1      0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
e1f70ada3bf1   test-container  0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
4656bb75b9c6   backend_api  0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
49e9553a9eec   postgres_db  0.00%     1.5MiB / 1.5GiB      0.1%      0B / 0B
```

`docker top` – processes inside a container

```
docker top container-name
docker top container-name -ef
```

`docker logs` – application logs

```
docker logs -f container-name
docker logs --tail 100 container-name
docker logs --since 2024-01-15 container-name
```

Container Inspection & Events

```
docker inspect container-name
docker inspect --format='{{.State.Status}}' container-name
docker inspect --format='{{.NetworkSettings.IPAddress}}' container-name
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker inspect web1
[
  {
    "Id": "9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d",
    "Created": "2026-04-24T14:38:55.110929673Z",
    "Path": "/docker-entrypoint.sh",
    "Args": [
      "nginx",
      "-g",
      "daemon off;"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 963,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-04-24T14:38:55.216515405Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3392e",
    "ResolvConfPath": "/var/lib/docker/containers/9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d/hostname",
    "HostsPath": "/var/lib/docker/containers/9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d/hosts",
    "LogPath": "/var/lib/docker/containers/9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d/9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d-json.log",
    "Name": "/web1",
    "RestartCount": 0,
    "Driver": "overlayfs",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
```

Real-time Docker events

```
docker events
docker events --filter 'type=container' --filter 'event=start'
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker events
2026-04-24T14:54:22.336960800Z container kill 9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d (desktop.docker.io/ports.scheme=v2, desktop.d
ocker.io/wsl-distro=Ubuntu, image=nginx, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=web1, signal=3)
2026-04-24T14:54:22.601171997Z network disconnect 486ef30de94f64de78852558ec8bef2bb66c4882eae9835046a2ab0d7b648d5a (container=9d55064357cb41bb2b42cff02fcbc3
c63ce4bd2d27a6f3ae8d3f762552a0dc9d, name=bridge, type=bridge)
2026-04-24T14:54:22.603395000Z volume unmount e093cc0a56fbb7fabf09b4968cbbab28222e5f4644361d3fcfe4f483f18c72f (container=9d55064357cb41bb2b42cff02fcbc3c63c
e4bd2d27a6f3ae8d3f762552a0dc9d, driver=local)
2026-04-24T14:54:22.603208200Z container stop 9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d (desktop.docker.io/ports.scheme=v2, desktop.d
ocker.io/wsl-distro=Ubuntu, image=nginx, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=web1)
2026-04-24T14:54:22.610335200Z container die 9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d (desktop.docker.io/ports.scheme=v2, desktop.d
ocker.io/wsl-distro=Ubuntu, execDuration=927, exitCode=0, image=nginx, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=web1)
2026-04-24T14:54:22.620591460Z volume mount e093cc0a56fbb7fabf09b4968cbbab28222e5f4644361d3fcfe4f483f18c72f (container=9d55064357cb41bb2b42cff02fcbc3c63c
e4bd2d27a6f3ae8d3f762552a0dc9d, destination=/app/data, driver=local, propagation=, read/write=true)
2026-04-24T14:54:22.768795483Z network connect 486ef30de94f64de78852558ec8bef2bb66c4882eae9835046a2ab0d7b648d5a (container=9d55064357cb41bb2b42cff02fcbc3c63
ce4bd2d27a6f3ae8d3f762552a0dc9d, name=bridge, type=bridge)
2026-04-24T14:54:22.784316200Z container start 9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d (desktop.docker.io/ports.scheme=v2, desktop.
docker.io/wsl-distro=Ubuntu, image=nginx, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=web1)
2026-04-24T14:54:22.784476700Z container restart 9d55064357cb41bb2b42cff02fcbc3c63ce4bd2d27a6f3ae8d3f762552a0dc9d (desktop.docker.io/ports.scheme=v2, deskto
p.docker.io/wsl-distro=Ubuntu, image=nginx, maintainer=NGINX Docker Maintainers <docker-maint@nginx.com>, name=web1)
```

Monitoring script (monitor.sh)

```
#!/bin/bash
echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)"
echo "1. Running Containers:"
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
echo "2. Resource Usage:"
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
echo "3. Recent Events:"
docker events --since '5m' --until '0s' --format '{{.Time}} {{.Type}} {{.Action}}' | tail
echo "4. System Info:"
docker system df
```

```
1 #!/bin/bash
2
3 echo "=== Docker Monitoring Dashboard ==="
4 echo "Time: $(date)"
5
6
7 echo "1. Running Containers:"
8 docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
9 echo
10
11 echo "2. Resource Usage:"
12 docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}"
13 echo
14
15 echo "3. Recent Events:"
16 docker events --since '5m' --until '0s' --format '{{.Time}} {{.Type}} {{.Action}}' | tail
17 echo
18
19 echo "4. System Info:"
20 docker system df
```

Part 4: Docker Networks

List Available Networks

```
docker network ls
```

```
^Carnav@LAPTOP-9T1CAWLD:/mnt/c/Users/Alok_Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
486ef38de94f    bridge   bridge      local
c43d518935ef    host     host        local
3ac7f6f1f3bd    minikube bridge      local
a289298371ee    myipvlan ipvlan      local
f1e45aaf5345    none     null        local
```

Network Types

1. Bridge Network (default for custom networks)

```
docker network create my-network
docker run -d --name web1-net --network my-network nginx
docker run -d --name web2-net --network my-network nginx
docker exec web1-net curl http://web2-net
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker exec web1-net curl http://web2-net
net
 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed

 0     0    0     0    0     0      0      0      0      0  0!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

2. Host Network

```
docker run -d --name host-app --network host nginx
curl http://localhost
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-5$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

3. None Network (completely isolated)

```
docker run -d --name isolated-app --network none alpine sleep 3600
docker exec isolated-app ifconfig # only loopback
docker exec isolated-app ping google.com # fails
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker run -d --name isolated-app --network none alpine sleep 3600
81b3e5a2a5dd32313c9cbf3d1bc97a873f022f6af525d875ec210f4e6199cd3d
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker exec isolated-app ifconfig
lo          Link encap:Local Loopback
            inet addr:127.0.0.1  Mask:255.0.0.0
            inet6 addr: ::1/128 Scope:Host
            UP LOOPBACK RUNNING  MTU:65536  Metric:1
            RX packets:0 errors:0 dropped:0 overruns:0 frame:0
            TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:1000
            RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker exec isolated-app ping -c 2 google.com
ping: bad address 'google.com'
```

Network Management Commands

```
docker network create app-network
docker network connect app-network existing-container
docker network disconnect app-network container-name
docker network inspect bridge
docker network prune
```

Port Publishing vs Exposing

```
docker run -d -p 80:8080 nginx # host:container
docker run -d -p 8080 nginx # dynamic host port
docker run -d -p 80:80 -p 443:443 nginx # multiple ports
```

View All Running Containers

```
docker ps
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-5$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
81b3e5a2a5dd  alpine        "sleep 3600"            6 minutes ago Up 6 minutes                               isolated-app
8c6fc90b0cd3  nginx         "/docker-entrypoint...." 7 minutes ago Up 7 minutes                               host-app
b98da2460d45  nginx         "/docker-entrypoint...." 8 minutes ago Up 8 minutes                               web2-net
5fa0c7e32556  nginx         "/docker-entrypoint...." 8 minutes ago Up 8 minutes                               web1-net
66663596a6ce  nginx         "/docker-entrypoint...." 22 minutes ago Up 22 minutes                               app2
249034e39dd6  nginx         "/docker-entrypoint...." 23 minutes ago Up 23 minutes                               app1
b8bde4bec00a  nginx         "/docker-entrypoint...." 25 minutes ago Up 25 minutes                               nginx-custom
c817678ba9c7  mysql:8.0     "/docker-entrypoint.s..." 26 minutes ago Up 26 minutes                               new-mysql
8cc33e15a3f2  nginx         "/docker-entrypoint...." 28 minutes ago Up 28 minutes                               web3
49d78a21814e  nginx         "/docker-entrypoint...." 28 minutes ago Up 28 minutes                               web2
9d55064357cb  nginx         "/docker-entrypoint...." 29 minutes ago Up 14 minutes                               web1
e1f70ada3bf1  ubuntu        "/bin/bash"              3 weeks ago   Up 44 minutes                               test-container
4656bb75b9c6  backend-image  "docker-entrypoint.s..." 5 weeks ago   Up 49 minutes                               backend_api
```

Part 5: Complete Real-World Example

Architecture: Flask web app + PostgreSQL + Redis, all on a custom network.

```
# 1. Create network
docker network create myapp-network

# 2. Start database with volume
docker run -d --name postgres --network myapp-network \
  -e POSTGRES_PASSWORD=mysecretpassword -e POSTGRES_DB=mydatabase \
  -v postgres-data:/var/lib/postgresql/data postgres:15

# 3. Start Redis
docker run -d --name redis --network myapp-network -v redis-data:/data redis:7-alpine

# 4. Start Flask app (custom image)
docker run -d --name flask-app --network myapp-network -p 5000:5000 \
  -v $(pwd)/app:/app -v app-logs:/var/log/app \
  -e
DATABASE_URL="postgresql://postgres:mysecretpassword@postgres:5432/mydatabase" \
  -e REDIS_URL="redis://redis:6379" -e DEBUG="false" \
  --env-file .env.production flask-app:latest

# Monitoring
docker stats postgres redis flask-app
docker logs -f flask-app
docker exec flask-app ping -c 2 postgres
```

Quick Reference Cheatsheet

Category	Command
Volumes	<code>docker volume create <name></code>
	<code>docker run -v <volume>:/path</code>
	<code>docker run -v /host/path:/container/path</code>
Environment	<code>docker run -e VAR=value</code>
	<code>docker run --env-file .env</code>

Category	Command
	<code>ENV VAR=value</code> (Dockerfile)
Monitoring	<code>docker stats</code> <code>docker logs -f <container></code> <code>docker top <container></code> <code>docker inspect <container></code>
Networks	<code>docker network create <name></code> <code>docker run --network <name></code> <code>docker network connect <network> <container></code>

Practice Exercises

1. **Database Backup** – Create a PostgreSQL container with a volume, backup the data using `docker cp`, and restore to a new container.
2. **Multi-Service Setup** – Run a web app + database + cache on a custom network, configure them with environment variables, and monitor all services.
3. **Log Analysis** – Run a container that generates logs, use `docker logs` filters, and redirect logs to a host file using a bind mount.
4. **Network Isolation** – Create two separate networks, place containers in each, test connectivity, then connect one container to both networks.

Cleanup Commands

```
# Stop and remove all containers
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)

# Remove all unused volumes, networks, images
docker volume prune -f
docker network prune -f
docker image prune -f
```

Key Takeaways

- **Volumes** persist data beyond container lifecycles.
 - **Environment variables** make containers configurable and reusable.
 - **Monitoring commands** (`stats`, `logs`, `top`, `events`) help debug and optimise containers.
 - **Networks** allow secure, named communication between containers.
 - Always use **named volumes** for production data and **custom networks** for service isolation.
 - Use **.env files** for sensitive configuration.
-

This experiment covers the essential Docker features needed to build, configure, and manage production-ready containerised applications.

Author: Arnav

Date: April 24, 2026

****Important notes for GitHub:****

1. Create an ``Images/`` folder inside your ``Experiment-5`` directory and place all the PNG files (``1.png``, ``2.png``, ..., ``21.png``, ``6A.png``, etc.) there.
2. The ``monitor.sh`` script and the ``app.py`` + ``Dockerfile`` shown in the screenshots are already included as code blocks – you can also add them as separate files in your repository.
3. Adjust the image paths if you prefer to keep the screenshots in the same folder (just remove ``Images/`` from each ``src``). The above assumes the standard ``.`/Images/`` subdirectory.

Now you can copy the whole block into your and push it to GitHub.

Experiment 6: Docker Run vs Docker Compose

Docker Run vs Docker Compose

This experiment compares the imperative (`docker run`) and declarative (`docker-compose.yml`) approaches for container deployment. It covers single-container setups, multi-container applications (WordPress + MySQL), building custom images, resource limits, and an introduction to scaling with Docker Swarm.

Table of Contents

- [Part A – Theory](#)
 - [Part B – Practical Tasks](#)
 - [Task 1: Single Container Comparison \(Nginx\)](#)
 - [Task 2: Multi-Container Application \(WordPress + MySQL\)](#)
 - [Part C – Conversion & Build-Based Tasks](#)
 - [Task 3: Convert Docker Run to Compose \(WebApp\)](#)
 - [Task 4: Resource Limits Conversion](#)
 - [Task 5: Replace Standard Image with Dockerfile \(Node.js\)](#)
 - [Task 6: Multi-Stage Dockerfile with Compose \(Challenge\)](#)
 - [Part D – Multi-Container WordPress Stack \(Detailed\)](#)
 - [Scaling in Compose vs Swarm](#)
 - [Cleanup Commands](#)
 - [Key Takeaways](#)
-

Part A – Theory

1. Objective

To understand the relationship between `docker run` and Docker Compose, compare their syntax, and learn when to use each approach.

2.1 Docker Run (Imperative Approach)

`docker run` creates and starts a container with explicit flags: `-p`, `-v`, `-e`, `--network`, `--restart`, `--memory`, `-cpus`, `--name`.

Example:

```
docker run -d --name my-nginx -p 8080:80 -v ./html:/usr/share/nginx/html -e NGINX_HOST=localhost --restart unless-stopped nginx:alpine
```

2.2 Docker Compose (Declarative Approach)

Uses a YAML file (`docker-compose.yml`) to define services, networks, and volumes.

Start everything with: `docker compose up -d`

Equivalent Compose file:

```
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: my-nginx
    ports: ["8080:80"]
    volumes: ["/html:/usr/share/nginx/html"]
    environment:
      NGINX_HOST: localhost
    restart: unless-stopped
```

3. Mapping: Docker Run vs Docker Compose

docker run flag	docker-compose.yml equivalent
<code>-p 8080:80</code>	<code>ports: - "8080:80"</code>
<code>-v ./html:/usr/share/...</code>	<code>volumes: - ./html:/usr/share/...</code>
<code>-e VAR=value</code>	<code>environment: VAR: value</code>
<code>--network mynet</code>	<code>networks: - mynet</code>
<code>--restart always</code>	<code>restart: always</code>
<code>--name myapp</code>	<code>container_name: myapp</code>
<code>--memory 256m --cpus 0.5</code>	<code>deploy: resources: limits: cpus: '0.5' memory: 256m</code>

4. Advantages of Docker Compose

1. Simplifies multi-container applications
2. Reproducibility and version control
3. Unified lifecycle management (`up`, `down`, `logs`, `ps`)
4. Supports service scaling (`docker compose up --scale web=3`)

Part B – Practical Tasks

Task 1: Single Container Comparison (Nginx)

Step 1 – Run Nginx using `docker run`

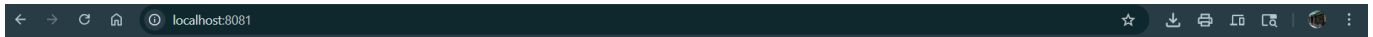
```
docker run -d --name lab-nginx -p 8081:80 -v $(pwd)/html:/usr/share/nginx/html
nginx:alpine
```

```
docker ps
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-Lab/Containerization-and-devops/Cloud Lab/Experiment-6/task1-nginx$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
817efacdbe76	nginx:alpine	"/docker-entrypoint..."	41 seconds ago	Up 41 seconds	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	lab-nginx
8c6fc90b0cd3	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours		host-app
b98da246d45	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web2-net
5fa80c7e32556	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web1-net
66663596a6ee	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	app2
249034e39dd6	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	app1
b8bde4bec08a	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	nginx-custom
c817678ba9c7	mysql:8.0	"/docker-entrypoint.s..."	2 hours ago	Up 2 hours	3306/tcp, 33060/tcp	new-mysql
8cc33e15a3f2	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web3
49d78a21814e	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web2
9d55064357cb	nginx	"/docker-entrypoint..."	2 hours ago	Up 2 hours	80/tcp	web1
e1f78ada3bf1	ubuntu	"/bin/bash"	3 weeks ago	Up 3 hours		test-container
4656bb75b9c6	backend-image	"/docker-entrypoint.s..."	5 weeks ago	Up 3 hours		backend_api
49e9553a9eec	postgres-image	"/docker-entrypoint.s..."	5 weeks ago	Up 3 hours		postgres_db

Access <http://localhost:8081> → you should see the Nginx welcome page.



Experiment 6 - Nginx Working

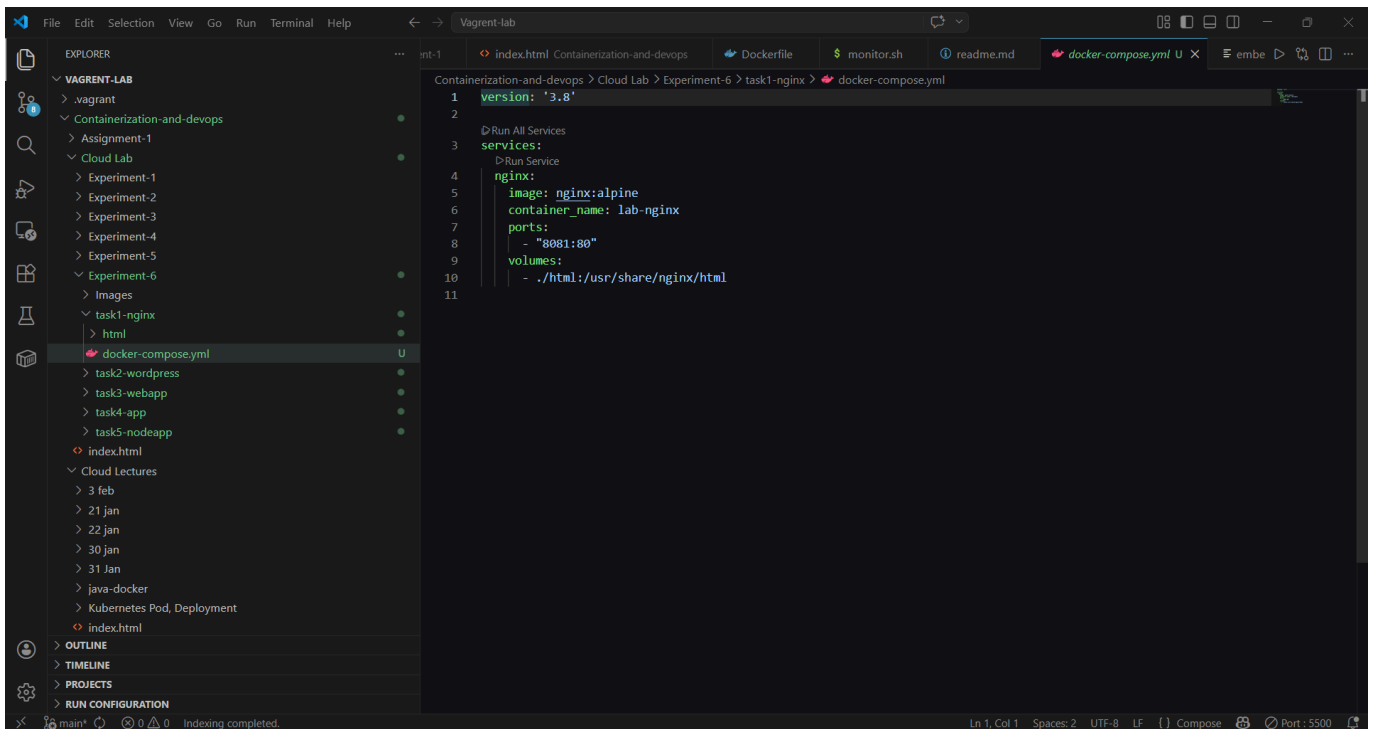
Stop and remove:

```
docker stop lab-nginx && docker rm lab-nginx
```

Step 2 – Same setup using Docker Compose

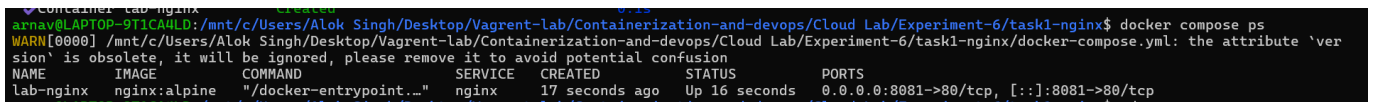
Create `docker-compose.yml`:

```
version: '3.8'
services:
  nginx:
    image: nginx:alpine
    container_name: lab-nginx
    ports: ["8081:80"]
    volumes: ["/html:/usr/share/nginx/html"]
```



Run:

```
docker compose up -d
docker compose ps
```



Stop:

```
docker compose down
```

Task 2: Multi-Container Application (WordPress + MySQL)

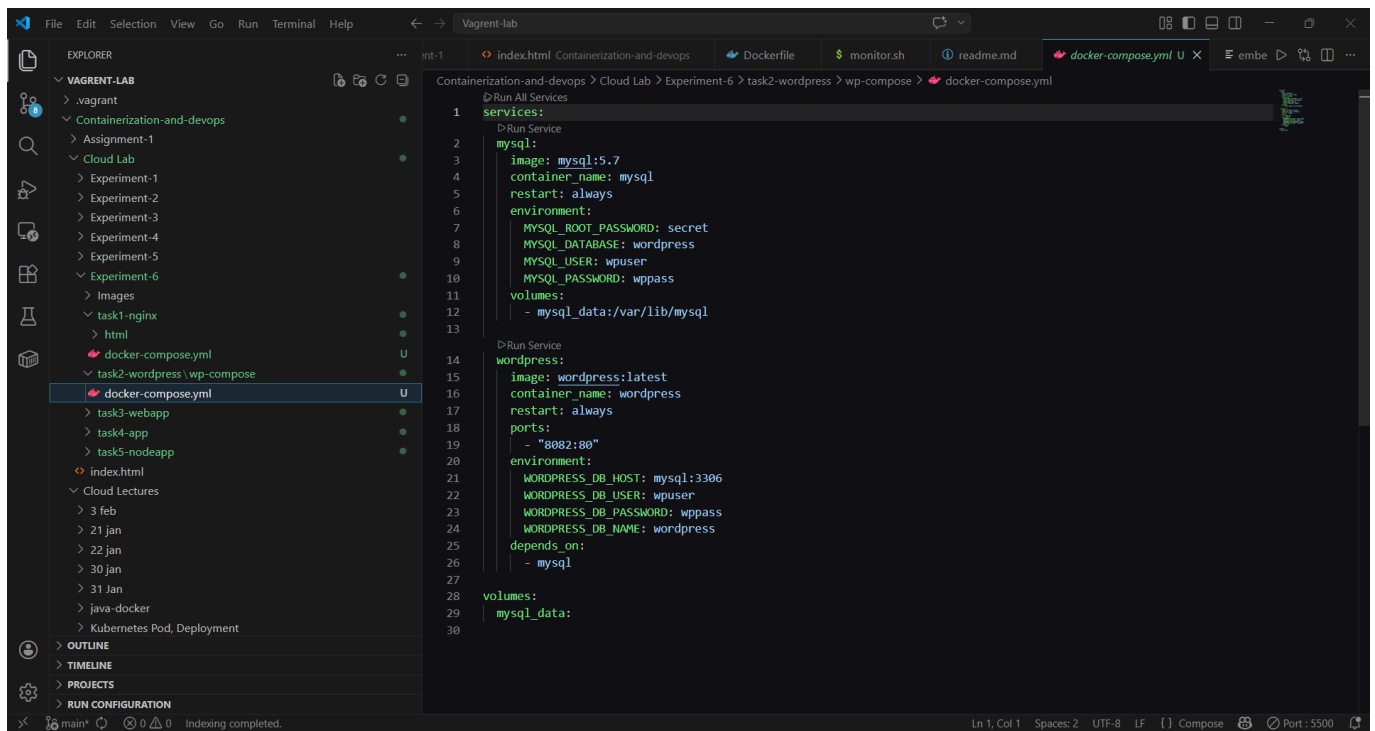
A. Using `docker run` (manual network) – not shown in screenshots but described in PDF.

B. Using Docker Compose

Create `docker-compose.yml` inside `task2-wordpress/wp-compose`:

```
version: '3.8'
services:
  mysql:
    image: mysql:5.7
    container_name: mysql
    restart: always
    environment:
```

```
MYSQL_ROOT_PASSWORD: secret
MYSQL_DATABASE: wordpress
MYSQL_USER: wpuser
MYSQL_PASSWORD: wppass
volumes:
  - mysql_data:/var/lib/mysql
wordpress:
  image: wordpress:latest
  container_name: wordpress
  ports: ["8082:80"]
  environment:
    WORDPRESS_DB_HOST: mysql:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wppass
    WORDPRESS_DB_NAME: wordpress
  depends_on: [mysql]
volumes:
  mysql_data:
```



Run:

```
docker compose up -d
docker ps
```

MySQL logs during startup:

Access <http://localhost:8082> → WordPress installation screen:

Stop and remove everything (including volumes):


```
docker compose down -v
```

Part C – Conversion & Build-Based Tasks

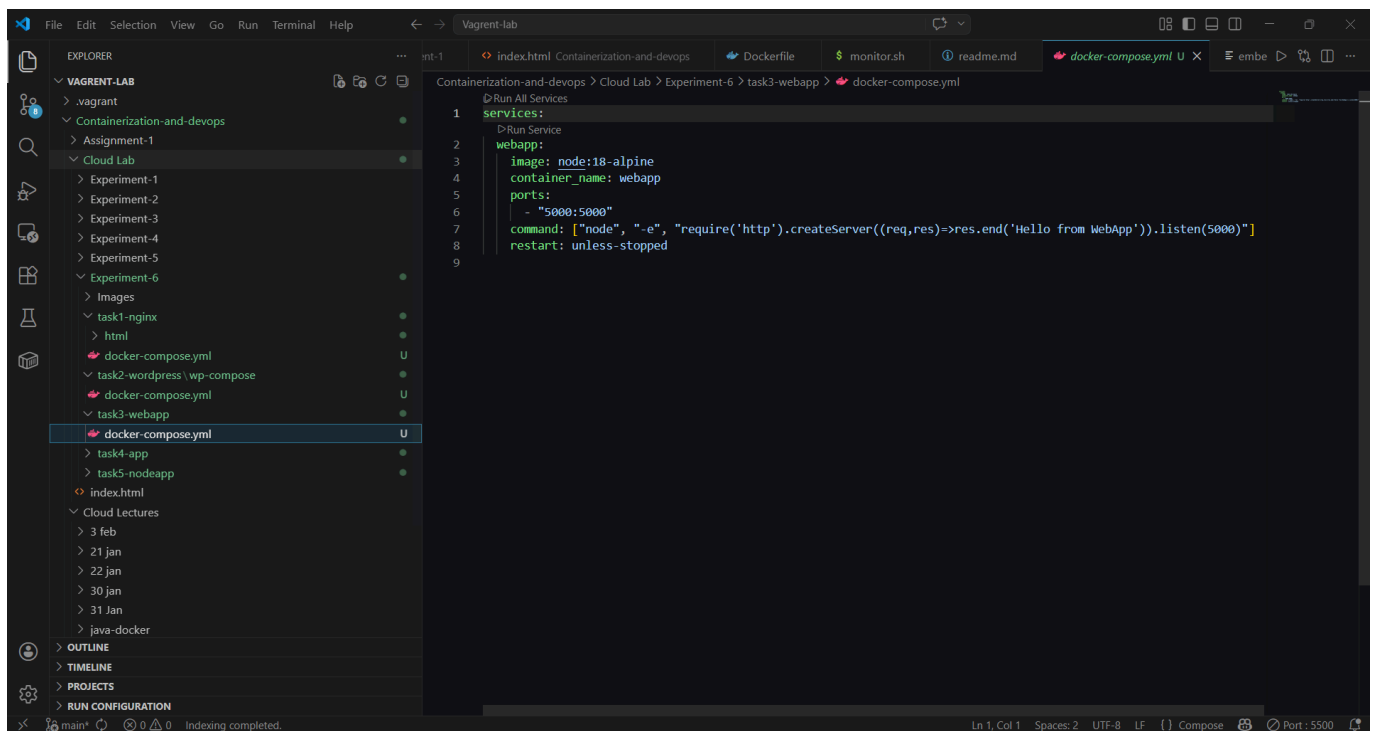
Task 3: Convert Docker Run to Compose (WebApp)

Given run command:

```
docker run -d --name webapp -p 5000:5000 -e APP_ENV=production -e DEBUG=false --
restart unless-stopped node:18-alpine
```

Equivalent `docker-compose.yml` (task3-webapp):

```
version: '3.8'
services:
  webapp:
    image: node:18-alpine
    container_name: webapp
    ports: ["5000:5000"]
    command: ["node", "-e", "require('http').createServer((req, res) =>
res.end('Hello from WebApp')).listen(5000)"]
    restart: unless-stopped
```



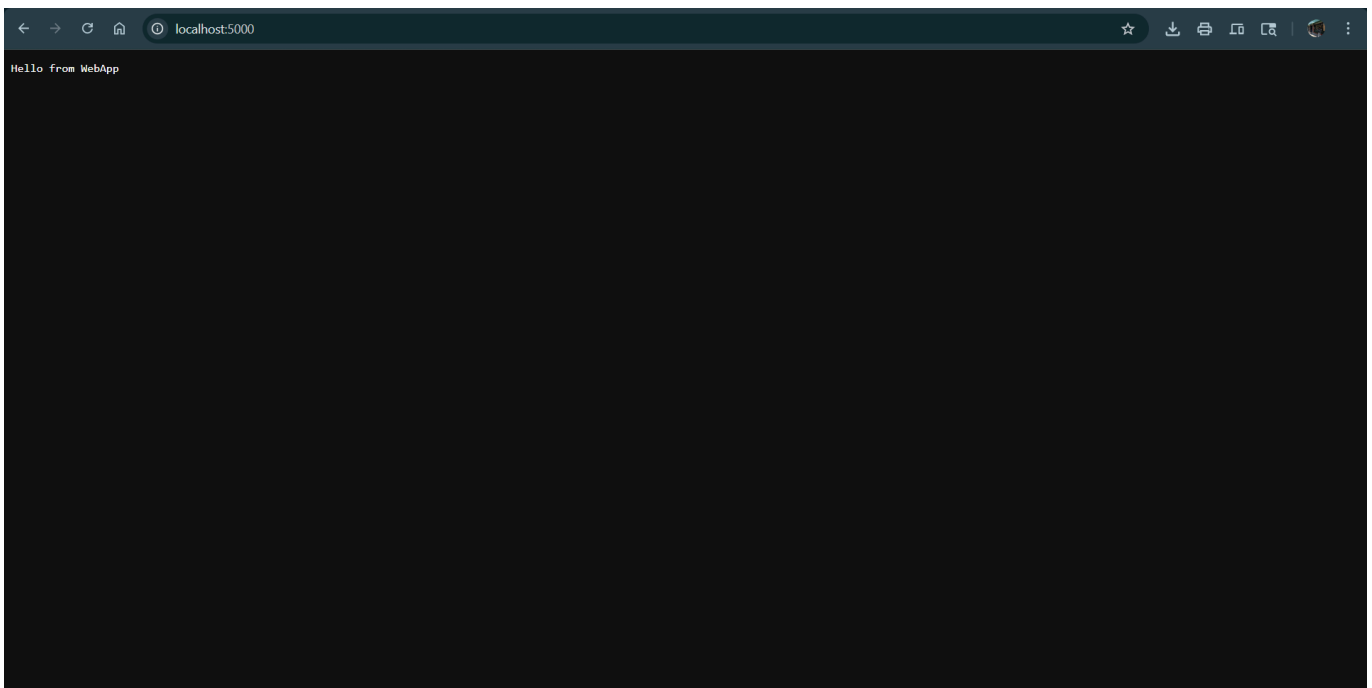
Run:

```
docker compose up -d
docker compose ps
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-6/task3-webapp$ docker compose ps
NAME        IMAGE          COMMAND                  SERVICE    CREATED         STATUS         PORTS
webapp      node:18-alpine "docker-entrypoint.s..." webapp     18 minutes ago  Up 18 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-6/task3-webapp$
```

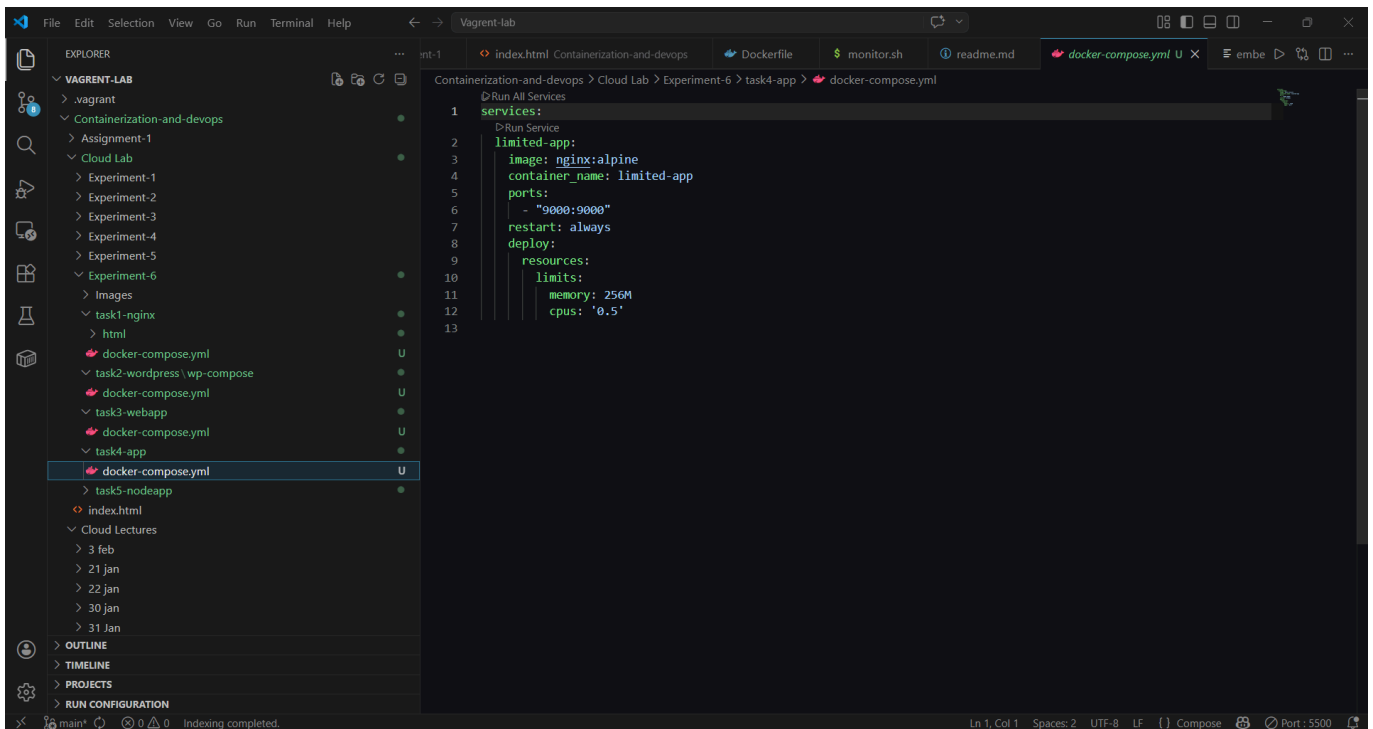
Test:

```
curl http://localhost:5000
```



Project structure overview:

The screenshot below shows the folder layout with multiple compose files for different tasks.



Task 4: Resource Limits Conversion

Given run command:

```
docker run -d --name limited-app -p 9000:9000 --memory=256m --cpus=0.5 --restart  
always nginx:alpine
```

Compose file (task4-app):

```
version: '3.8'  
services:  
  app:  
    image: nginx:alpine  
    container_name: limited-app  
    ports: ["9000:9000"]  
    deploy:  
      resources:  
        limits:  
          cpus: '0.5'  
          memory: 256m  
      restart: always
```

Run:

```
docker compose up -d  
docker ps
```

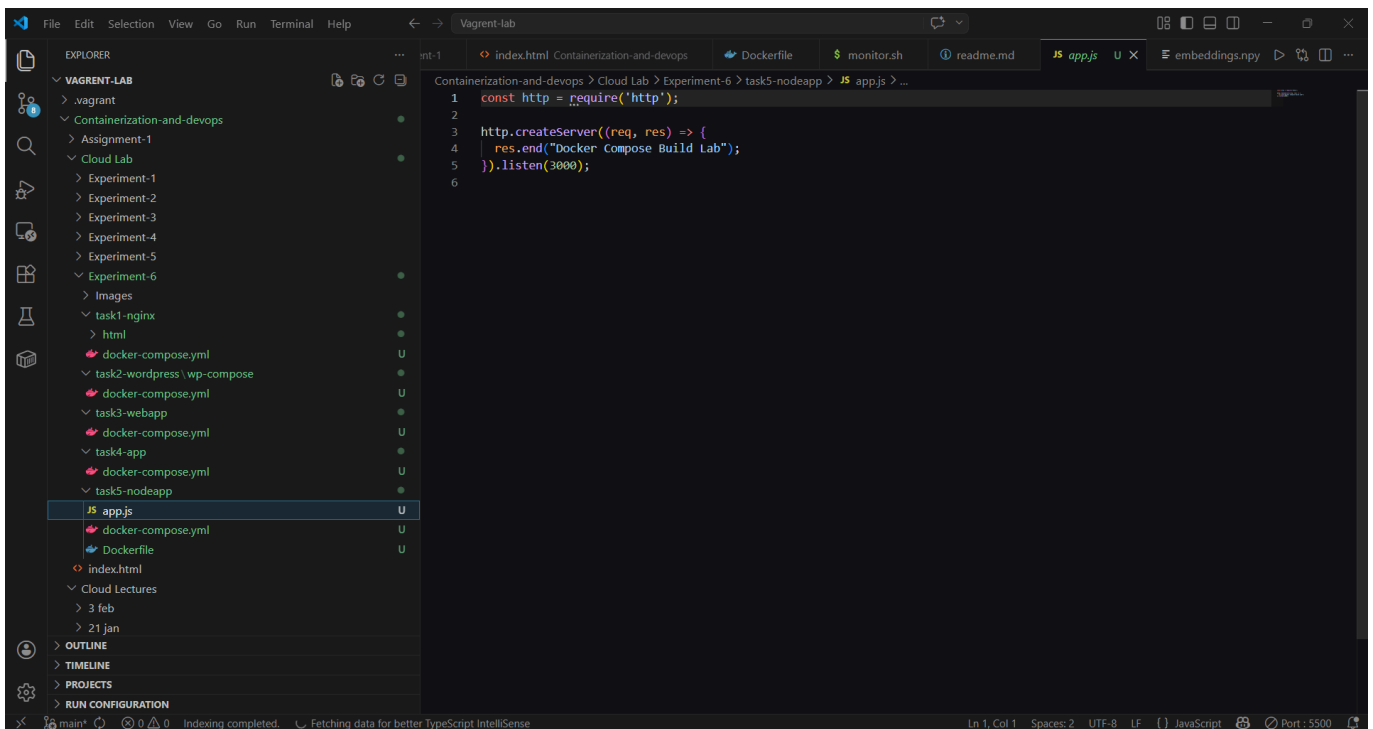
```
arnav@LAPTOP-9TICA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-6/task4-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
41dce334009f   nginx:alpine   "/docker-entrypoint.s..." 5 seconds ago  Up 5 seconds  0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp  limited-ap
0a5250c213a4   node:18-alpine "docker-entrypoint.s..." About a minute ago Up About a minute  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  webapp
fa0816c715a0   wordpress:latest "docker-entrypoint.s..." 9 minutes ago  Up 9 minutes  0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  wordpress
680e669d4622   mysql:5.7      "docker-entrypoint.s..." 9 minutes ago  Up 9 minutes  3306/tcp, 33060/tcp  mysql
```

Note: `deploy` resource limits are fully respected in Swarm mode. In standard Compose they may be ignored; use `--compatibility` or runtime flags if needed.

Task 5: Replace Standard Image with Dockerfile (Node.js)

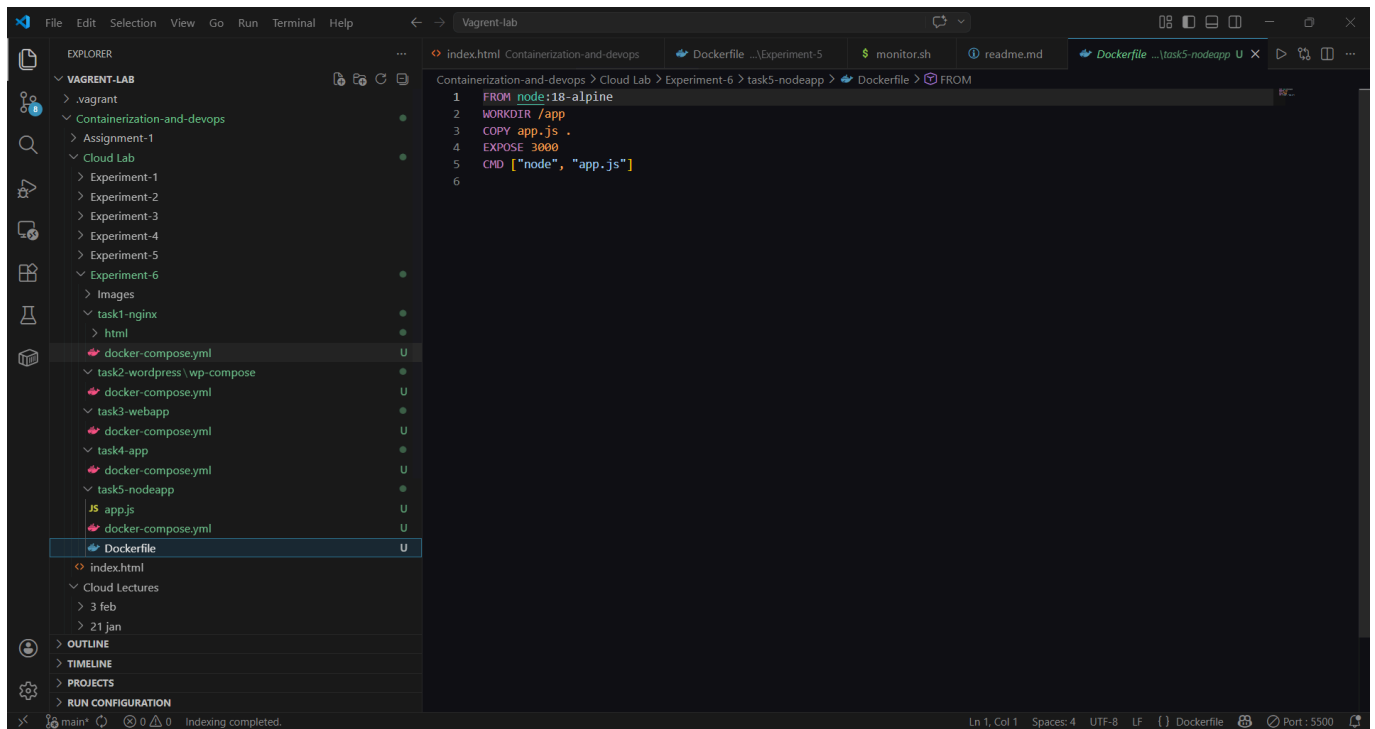
Step 1 – Create `app.js`:

```
const http = require('http');
http.createServer((req, res) => {
  res.end("Docker Compose Build Lab");
}).listen(3000);
```



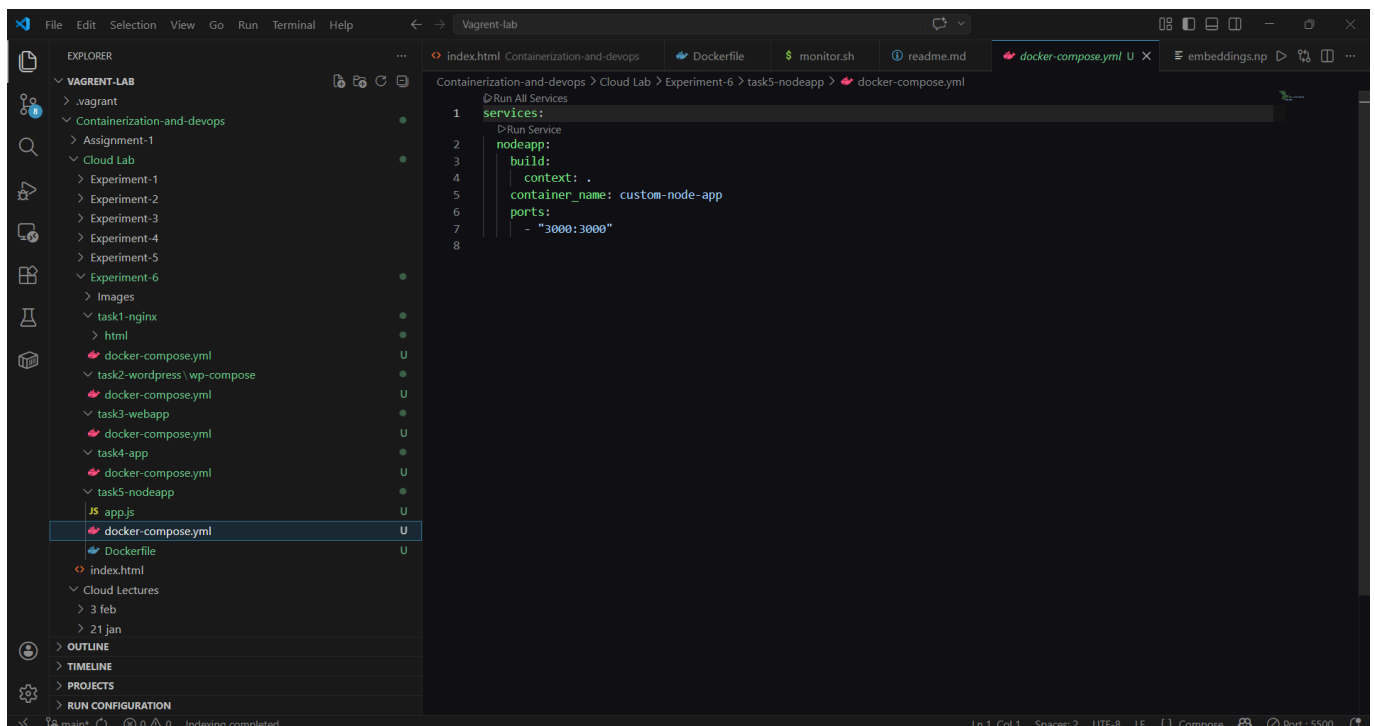
Step 2 – Create `Dockerfile`:

```
FROM node:18-alpine
WORKDIR /app
COPY app.js .
EXPOSE 3000
CMD ["node", "app.js"]
```



Step 3 – Create `docker-compose.yml`:

```
version: '3.8'
services:
  nodeapp:
    build: .
    container_name: custom-node-app
    ports: ["3000:3000"]
```



Build and run:

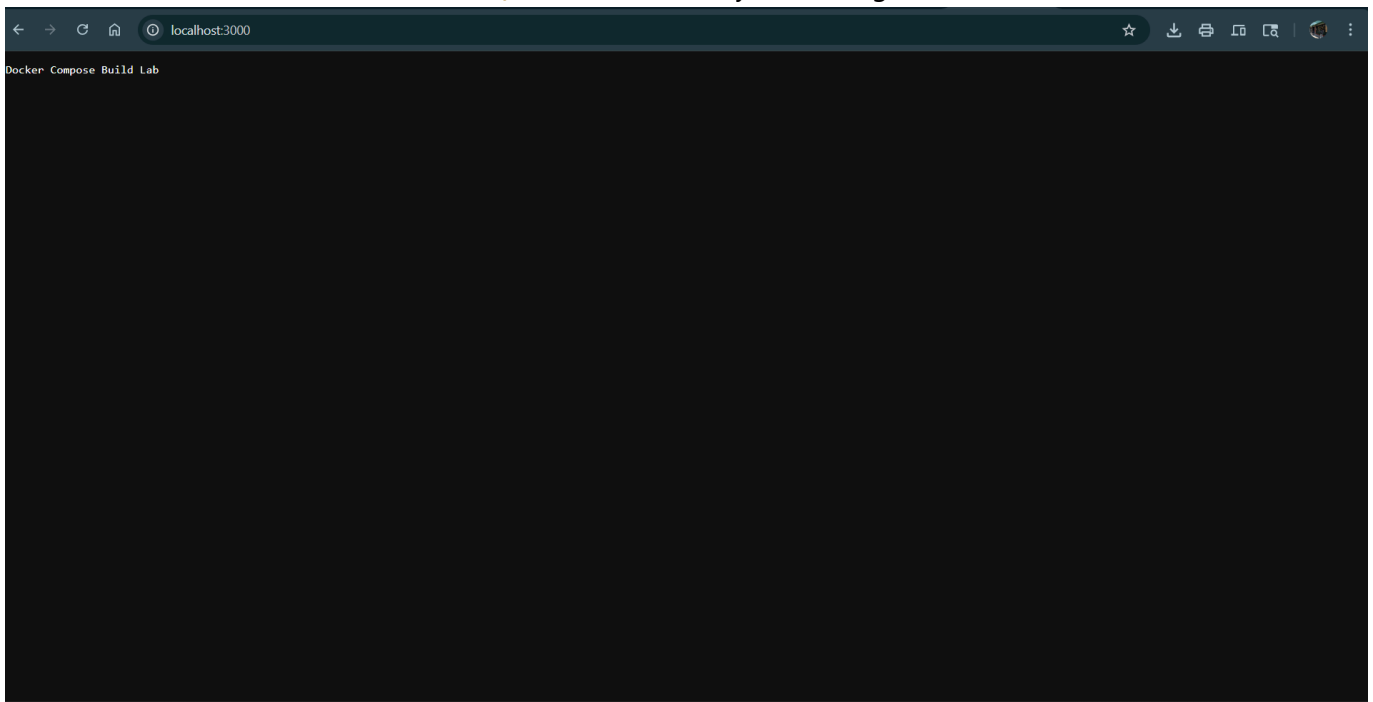
```
docker compose up --build -d
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-Lab/Containerization-and-devops/Cloud Lab/Experiment-6/task5-nodeapp$ docker compose up --build -d
[+] Building 1.2s (10/10) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 684B                                          0.0s
=> [internal] load build definition from Dockerfile                    0.0s
=> => transferring dockerfile: 119B                                     0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine      0.1s
=> [internal] load .dockerignore                                       0.0s
=> => transferring context: 2B                                          0.0s
=> [1/3] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c2  0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c2  0.0s
=> [internal] load build context                                       0.1s
=> => transferring context: 154B                                        0.0s
=> CACHED [2/3] WORKDIR /app                                           0.0s
=> [3/3] COPY app.js .                                                0.0s
=> => exporting to image                                               0.2s
=> => exporting layers                                                0.1s
=> => exporting manifest sha256:76f8025f77180da24d773f590fc639b69d39 0.0s
=> => exporting config sha256:b465006d4341c3dd0404bd8ca8533c65b17e02 0.0s
=> => exporting attestation manifest sha256:5d8dc5119935afbafab5104f  0.0s
=> => exporting manifest list sha256:401a451989f74d150950c99e0664b8f  0.0s
=> => naming to docker.io/library/task5-nodeapp-nodeapp:latest        0.0s
=> => unpacking to docker.io/library/task5-nodeapp-nodeapp:latest      0.0s
=> => resolving provenance for metadata file                           0.0s
[+] up 3/3
✔Image task5-nodeapp-nodeapp Built 1.3s
✔Network task5-nodeapp_default Created 0.0s
```

Verify at <http://localhost:3000>.

Change the message in `app.js`, then rebuild: `docker compose up --build -d`

Additional command: use `docker compose build` to only build images.



Difference between `image:` and `build:`

- `image:` uses a pre-existing image.
- `build:` tells Compose to build an image from a Dockerfile in the specified context.

Task 6: Multi-Stage Dockerfile with Compose (Challenge)

Create a multi-stage Dockerfile for a Python FastAPI app (or Node.js) that uses a builder stage and a lean final stage. Build with Compose (`build: .`), add environment variables and a development volume mount.

Compare image sizes:

docker images

Part D – Multi-Container WordPress Stack (Detailed)

Architecture

- WordPress container (PHP/Apache)
- MySQL container
- Custom network for internal DNS
- Volumes for persistent database and WordPress files

`docker-compose.yml` (full version with named volumes)

```
version: '3.9'
services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:
      - db_data:/var/lib/mysql
  wordpress:
    image: wordpress:latest
    container_name: wordpress_app
    depends_on: [db]
    ports: ["8080:80"]
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html
volumes:
  db_data:
  wp_data:
```

Start and verify:

```
docker compose up -d
docker ps
```

Access <http://localhost:8080> to complete WordPress installation.

Volumes persist after `docker compose down`. Remove them with `docker compose down -v`.

Scaling in Docker Compose vs Docker Swarm

Compose scaling (single host, no load balancing)

```
docker compose up --scale wordpress=3
```

Problem: Port conflict.

Solution: Use a reverse proxy (Nginx) or Swarm.

Docker Swarm (production)

```
docker swarm init
docker stack deploy -c docker-compose.yml wpstack
docker service scale wpstack_wordpress=3
```

Feature	Docker Compose	Docker Swarm
Scope	Single host	Multi-node cluster
Scaling	Manual (<code>--scale</code>)	Built-in (<code>service scale</code>)
Load balancing	No	Yes (internal ingress)
Self-healing	No	Yes
Rolling updates	No	Yes

Cleanup Commands

```
# Stop and remove containers, networks, and volumes (if -v added)
docker compose down -v

# Remove all unused resources
docker volume prune -f
docker network prune -f
docker image prune -f
```



```
# Leave Swarm mode if initialized
docker swarm leave --force
```

Key Takeaways

- **docker run** = imperative, good for quick tests.
- **Docker Compose** = declarative, ideal for multi-service apps.
- **Builds** integrated via **build:** in Compose.
- **Multi-stage Dockerfiles** produce smaller images.
- **Scaling** requires Swarm (or Kubernetes) for production.
- **Volumes** ensure data persistence.

This experiment covers the complete spectrum from single-container runs to production-orchestrated stacks using Docker Compose and Swarm.

Author: Arnav

Date: April 24, 2026

****Instructions for GitHub:****

1. Create an ``Images/`` folder inside ``Experiment-6/``.
2. Copy ****all**** screenshot files (``1.png` ... `18.png``) into ``Images/``.
3. Ensure that the directory structure matches the one shown in ``12.png`` (task1-nginx, task2-wordpress, task3-webapp, task4-app, task5-nodeapp folders).
4. Push the together with your code files.

Now every image is properly referenced and the README is fully self-contained.

Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

1. Aim

To design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

2. Objectives

- Understand CI/CD workflow using Jenkins (GUI-based tool)
- Create a structured GitHub repository with application + Jenkinsfile
- Build Docker images from source code
- Securely store Docker Hub credentials in Jenkins
- Automate build & push process using webhook triggers
- Use same host (Docker) as Jenkins agent

3. Theory

Jenkins – Web-based automation server for building, testing, and deploying software. Provides dashboard, plugin ecosystem, and Pipeline as Code using **Jenkinsfile**.

CI/CD – Continuous Integration (automated build/test after each commit) + Continuous Deployment (automatic delivery of built artifacts, e.g., Docker images).

Workflow: Developer → GitHub → Webhook → Jenkins → Build → Docker Hub

4. Prerequisites

- Docker & Docker Compose installed
- GitHub account
- Docker Hub account
- Basic Linux command knowledge

5. Part A: GitHub Repository Setup

5.1 Create Repository

Create a repository on GitHub (e.g., **my-app**).

5.2 Project Structure

```
my-app/  
├── app.py  
└── requirements.txt
```

```
|— Dockerfile
|— Jenkinsfile
```

5.3 Application Code (`app.py`)

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline!"

app.run(host="0.0.0.0", port=80)
```

5.4 Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 80
CMD ["python", "app.py"]
```

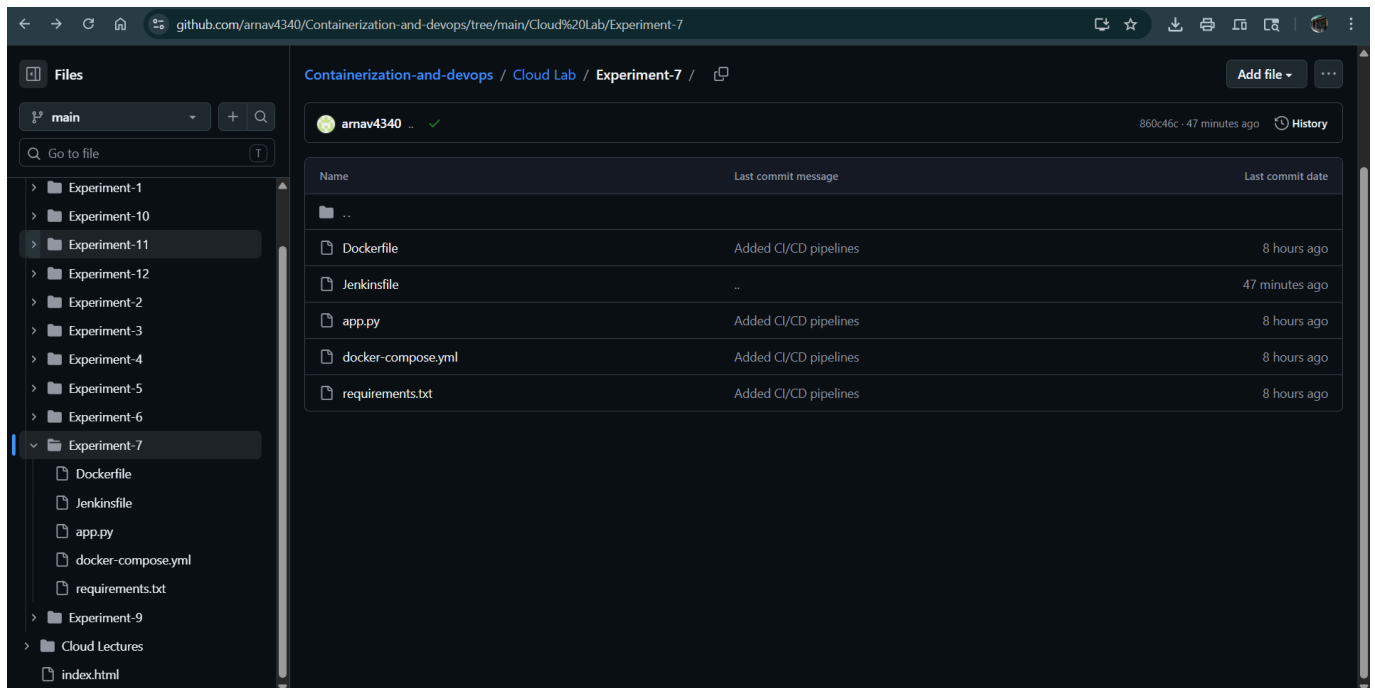
5.5 Jenkinsfile (Pipeline Definition)

```
pipeline {
    agent any
    environment {
        IMAGE_NAME = "arnavarnav/myapp"    # replace with your Docker Hub username
    }
    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/arnav4340/Containerization-and-devops.git'
            }
        }
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
            }
        }
        stage('Login to Docker Hub') {
            steps {
                withCredentials([string(credentialsId: 'dockerhub-token',
variable: 'DOCKER_TOKEN')]) {
```

```

        sh 'echo $DOCKER_TOKEN | docker login -u arnavarnav --
password-stdin'
    }
}
}
stage('Push to Docker Hub') {
    steps {
        sh 'docker push $IMAGE_NAME:latest'
    }
}
}
}
}

```



6. Part B: Jenkins Setup using Docker

6.1 Create `docker-compose.yml`

```

version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: always
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    user: root

```

```
volumes:  
  jenkins_home:
```

6.2 Start Jenkins

```
docker-compose up -d
```

Access: <http://localhost:8080>

6.3 Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

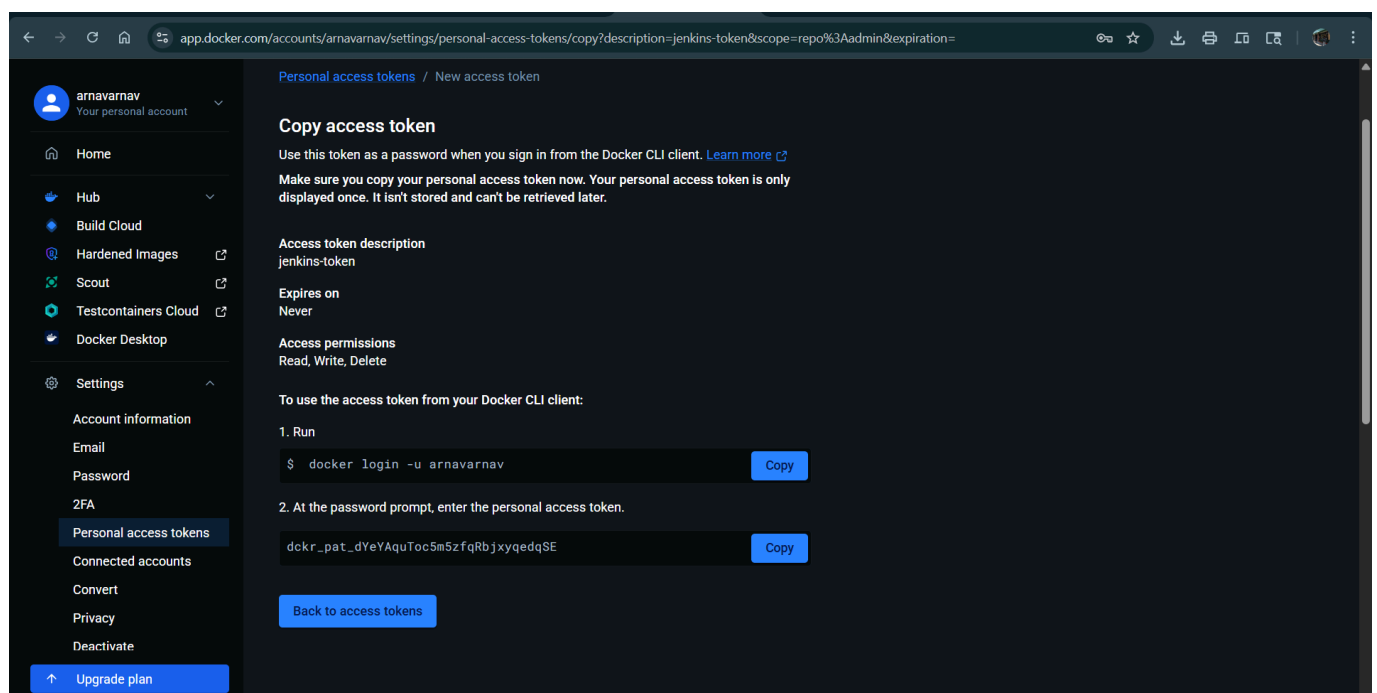
6.4 Initial Setup

- Install suggested plugins
- Create admin user

7. Part C: Jenkins Configuration

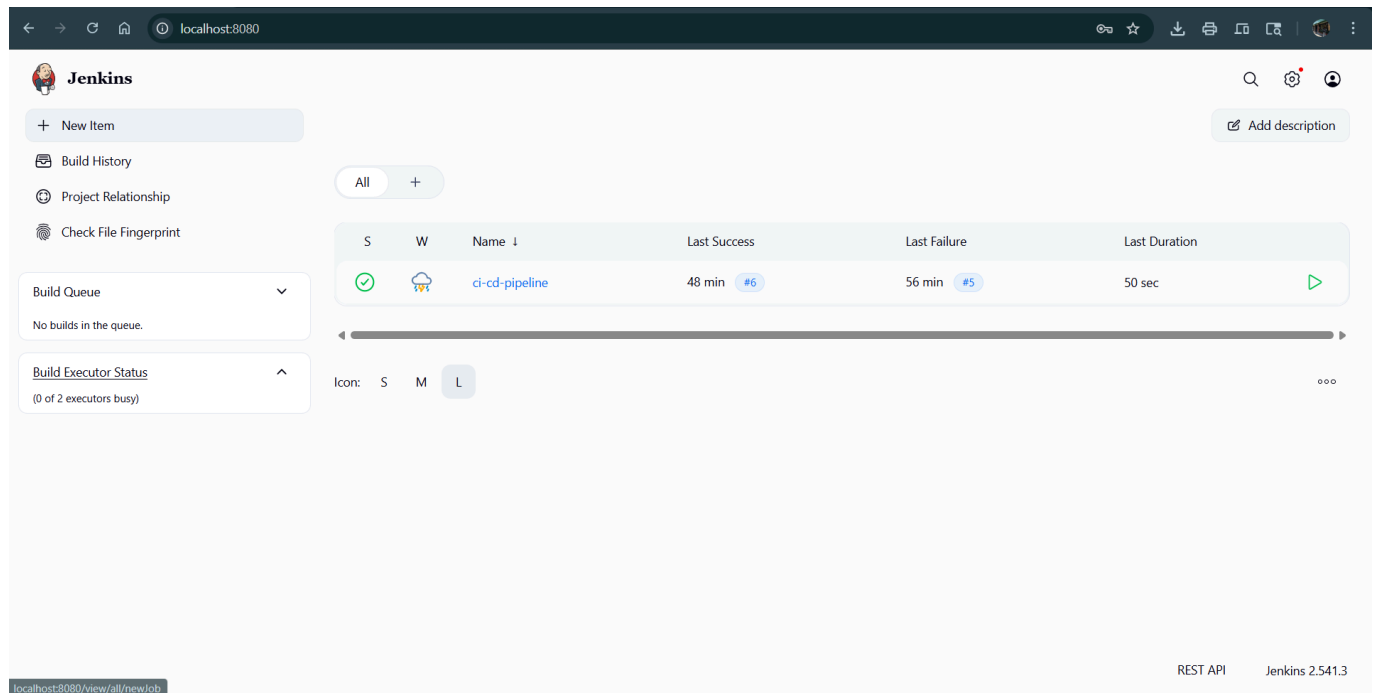
7.1 Add Docker Hub Credentials

- **Path:** Manage Jenkins → Credentials → Add Credentials
- **Type:** Secret Text
- **ID:** `dockerhub-token`
- **Value:** Docker Hub Access Token (generated from Docker Hub → Account Settings → Security → New Access Token)



7.2 Create Pipeline Job

1. New Item → Pipeline → Name: **ci-cd-pipeline**
2. Configure:
 - **Pipeline script from SCM**
 - **SCM:** Git
 - **Repository URL:** <https://github.com/arnav4340/Containerization-and-devops.git>
 - **Script Path:** Cloud Lab/Experiment-7/Jenkinsfile



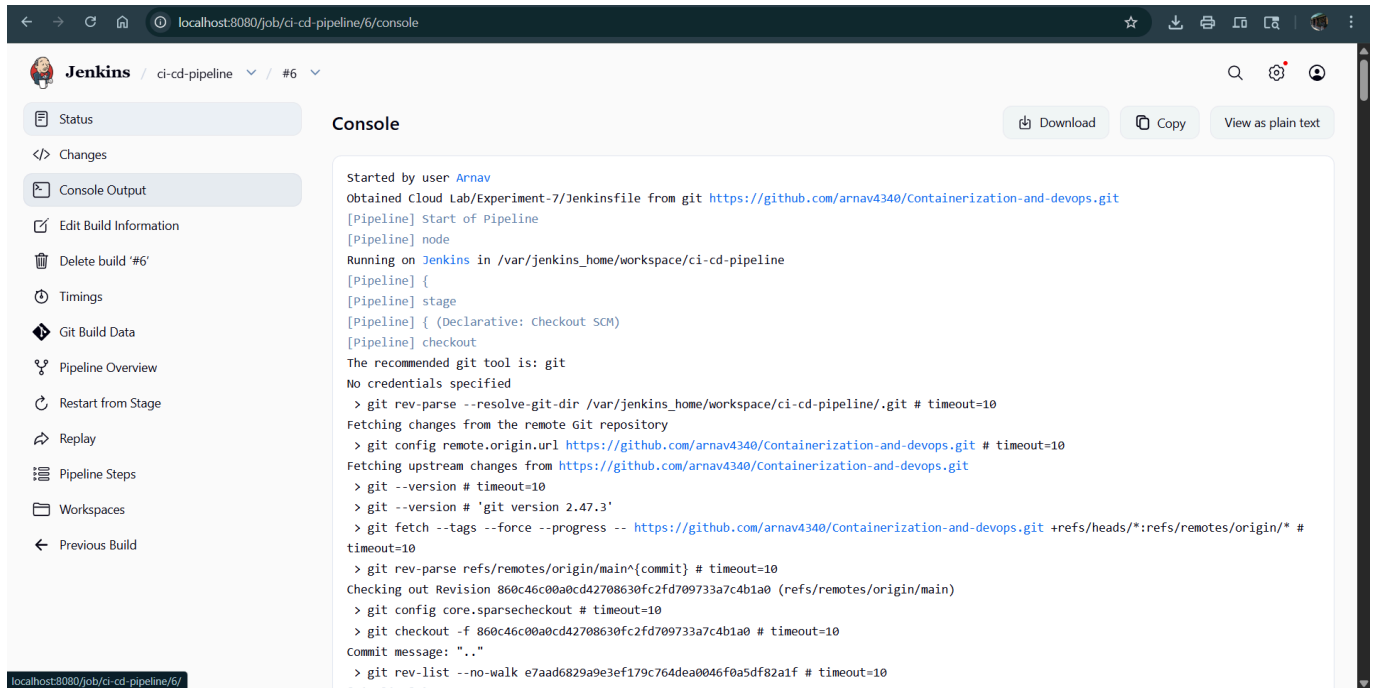
8. Part D: GitHub Webhook Integration

On GitHub repository:

- Settings → Webhooks → Add Webhook
- **Payload URL:** <http://<your-server-ip>:8080/github-webhook/>
- **Events:** Push events

9. Part E: Execution Flow (Stage-wise)

1. **Code Push** – Developer pushes code to GitHub
2. **Webhook Trigger** – GitHub sends event to Jenkins
3. **Jenkins Pipeline:**
 - *Clone* – pulls latest code
 - *Build* – **docker build** using Dockerfile
 - *Login* – logs into Docker Hub using stored token
 - *Push* – pushes image to Docker Hub
4. **Artifact Ready** – Docker image available globally



```
Started by user Arnab
Obtained Cloud Lab/Experiment-7/Jenkinsfile from git https://github.com/arnav4340/Containerization-and-devops.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/ci-cd-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
No credentials specified
> git rev-parse --resolve-git-dir /var/jenkins_home/workspace/ci-cd-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/arnav4340/Containerization-and-devops.git # timeout=10
Fetching upstream changes from https://github.com/arnav4340/Containerization-and-devops.git
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --tags --force --progress -- https://github.com/arnav4340/Containerization-and-devops.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 860c46c00a0cd42708630fc2fd709733a7c4b1a0 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 860c46c00a0cd42708630fc2fd709733a7c4b1a0 # timeout=10
Commit message: ".."
> git rev-list --no-walk e7aad6829a9e3ef179c764dea0046f0a5df82a1f # timeout=10
```

10. Role of Same Host Agent

Jenkins runs inside Docker, and the Docker socket (`/var/run/docker.sock`) is mounted. This allows Jenkins to directly control the host Docker daemon – building and pushing images without a separate agent.

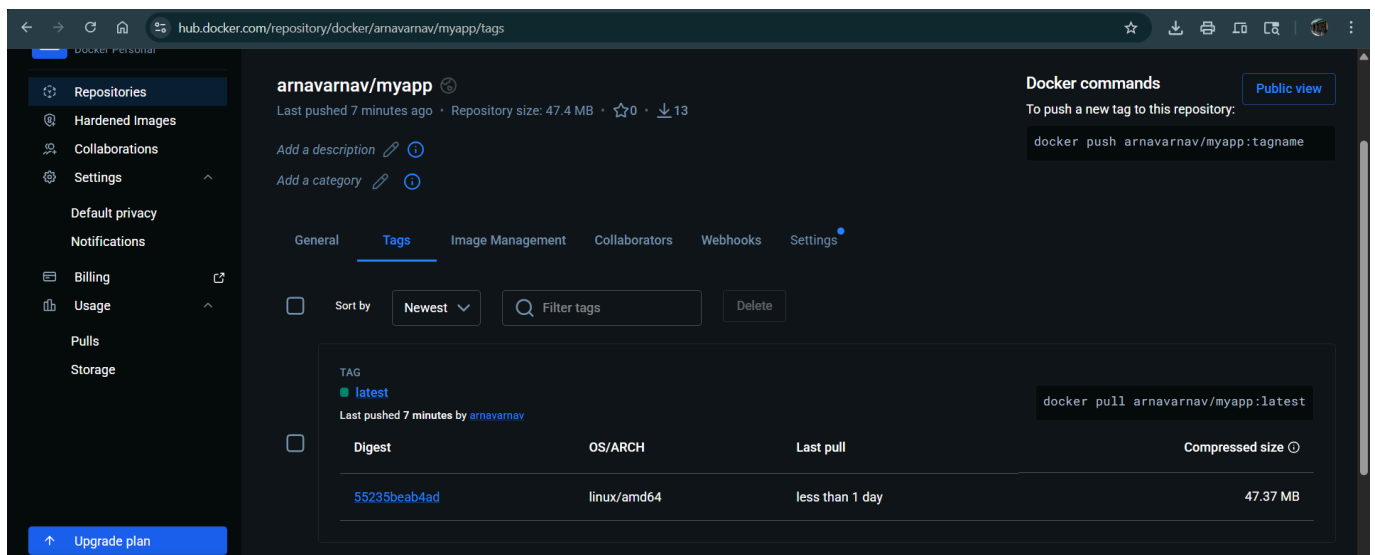
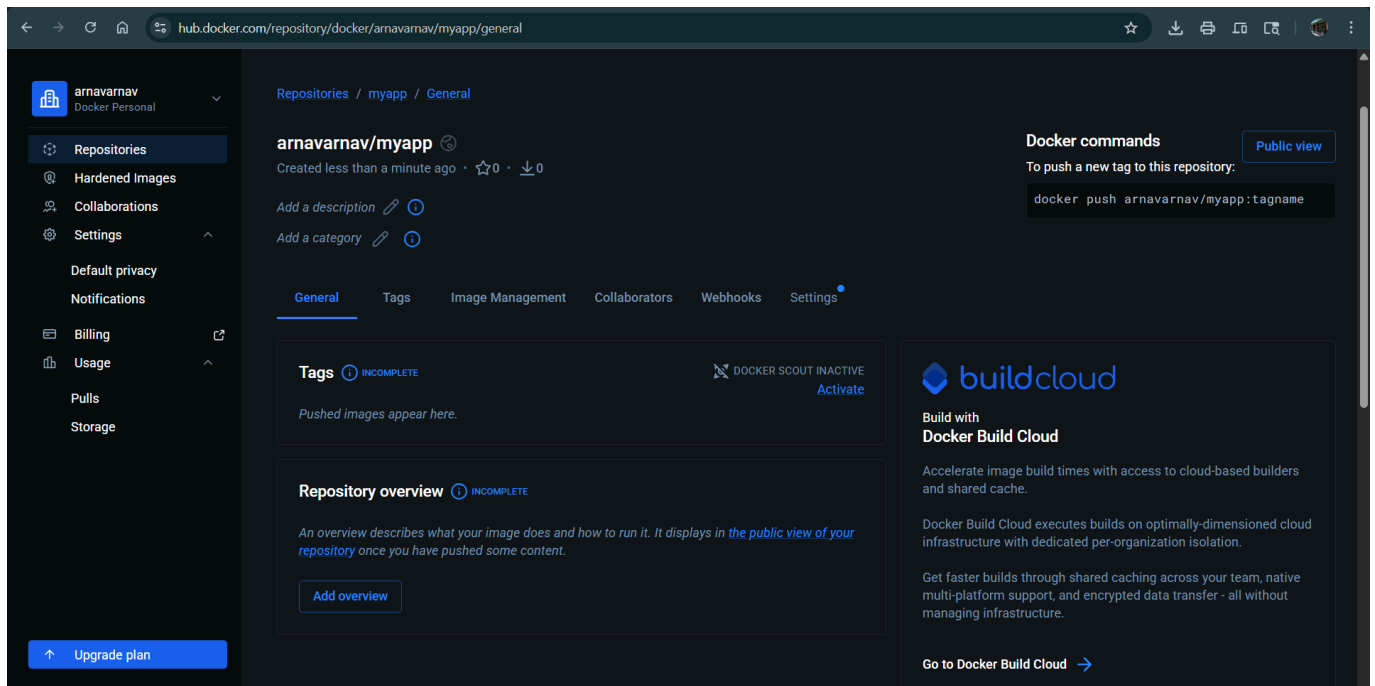
11. Observations

- Jenkins GUI simplifies CI/CD management
- GitHub acts as source + pipeline definition
- Docker ensures consistent builds
- Webhook enables automation

12. Result

Successfully implemented a complete CI/CD pipeline:

- Source code and pipeline maintained in GitHub
- Jenkins automatically detects changes
- Docker image is built on host agent
- Image is securely pushed to Docker Hub



Running the image locally:

```
docker run -p 5000:80 arnavarnav/myapp:latest
```



```
^Carnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-7$ docker run -p 5000:80 arnavarnav/myapp:latest
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:80
* Running on http://172.17.0.2:80
Press CTRL+C to quit
172.17.0.1 - - [01/May/2026 06:02:30] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [01/May/2026 06:02:30] "GET /favicon.ico HTTP/1.1" 404 -
```



List of built images:

docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
arnavarnav/myapp	latest	9381ddf5b450	About an hour ago	203MB
arnav4340/myapp	latest	0d4ea25c5c87	About an hour ago	203MB
ubuntu-server	latest	cab49d7f5f33	6 days ago	976MB
task5-nodeapp-nodeapp	latest	401a451989f7	6 days ago	181MB
wordpress	latest	054e61133439	9 days ago	1.08GB
wordpress	latest	73fe513dae90	9 days ago	1.08GB
mysql	8.0	d0304ed9fdb6	9 days ago	1.1GB
nginx	alpine	5616878291a2	2 weeks ago	93.5MB
my-node-app	latest	6777dba64a7a	4 weeks ago	196MB
flask-multistage	latest	2d0327f0dfd7	4 weeks ago	219MB
<none>	<none>	54ada0268f47	4 weeks ago	200MB
arnavarnav/my-flask-app	1.0	483142b0bac0	4 weeks ago	200MB
arnavarnav/my-flask-app	latest	483142b0bac0	4 weeks ago	200MB
my-flask-app	1.0	483142b0bac0	4 weeks ago	200MB
my-flask-app	latest	483142b0bac0	4 weeks ago	200MB
my-flask-app	v1.0	483142b0bac0	4 weeks ago	200MB
username/my-flask-app	1.0	483142b0bac0	4 weeks ago	200MB
username/my-flask-app	latest	483142b0bac0	4 weeks ago	200MB
flask-regular	latest	c6e197bcbf215	4 weeks ago	200MB
jenkins/jenkins	lts	c4098086090c	6 weeks ago	806MB
backend-image	latest	ebe9d2e0dfb4	6 weeks ago	187MB
java-app	1.0	283504d1b8d8	8 weeks ago	1.17GB
java-app	2.0	0a964baa45a4	8 weeks ago	1.17GB
postgres-image	latest	31b1e91253e8	2 months ago	392MB
nginx	latest	0236ee02dcbc	2 months ago	240MB
gcr.io/k8s-minikube/kicbase	v0.0.50	b97074569ae9	2 months ago	1.94GB
gcr.io/k8s-minikube/kicbase	<none>	eb4fec00e8ad	2 months ago	1.94GB
ubuntu	latest	d1e2e92c075e	2 months ago	119MB
nginx-alpine	latest	390eeeb66c73	2 months ago	16MB
nginx-ubuntu	latest	b430157fc47d	2 months ago	199MB
alpine	latest	25109184c71b	3 months ago	13.1MB
sonarqube	lts	f709975ab31d	8 months ago	1.02GB
hello-world	latest	ef54e839ef54	8 months ago	25.9kB
node	18-alpine	8d6421d663b4	13 months ago	181MB
mysql	5.7	4bc6bc963e6d	2 years ago	700MB

13. Viva Questions

1. What is the role of Jenkinsfile?
2. How does Jenkins integrate with GitHub?
3. Why is Docker used in CI/CD?
4. What is a webhook?
5. Why store Docker Hub token in Jenkins credentials?
6. What is the benefit of using same host as agent?

14. Key Notes

- Jenkins is GUI-based but pipeline is code-driven
- Always use credentials store (never hardcode secrets)
- Webhook makes CI/CD fully automatic
- This setup is ideal for learning and small deployments

Understanding Jenkins Pipeline Syntax

Basic structure:

```
pipeline {  
  agent any  
  stages {  
    stage('Name') {  
      steps {  
        // commands  
      }  
    }  
  }  
}
```

Common steps:

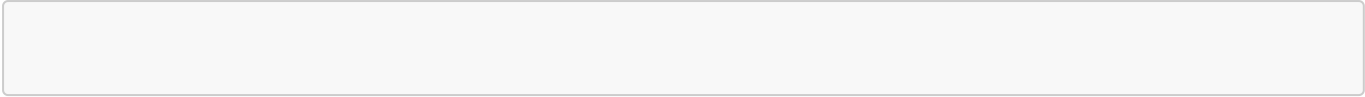
- `git 'url'` – clones repository
- `sh 'command'` – runs shell command
- `echo "message"` – prints to console

withCredentials – secure secret handling:

```
withCredentials([string(credentialsId: 'dockerhub-token', variable:  
'DOCKER_TOKEN')]) {  
  sh 'echo $DOCKER_TOKEN | docker login -u username --password-stdin'  
}
```

This injects the secret temporarily – never writes passwords in code.

End of Experiment



Experiment 9: Docker – Ansible – Agentless Configuration Management

Ansible – Agentless Configuration Management

This experiment demonstrates how Ansible solves infrastructure management problems through agentless automation, idempotent playbooks, and declarative YAML syntax. We will set up a control node (WSL/Ubuntu), create four Docker containers as managed servers, and use Ansible to configure them consistently.

Table of Contents

- [What is Ansible?](#)
 - [Key Concepts](#)
 - [How Ansible Works](#)
 - [Benefits of Ansible](#)
 - [Part A – Ansible Installation & Demo](#)
 - [Installation via apt](#)
 - [Post-Installation Test](#)
 - [Part B – Setting Up Docker Containers as Managed Nodes](#)
 - [1. Generate SSH Key Pair](#)
 - [2. Create Dockerfile with OpenSSH Server](#)
 - [3. Build the Docker Image](#)
 - [4. Run Four SSH Server Containers](#)
 - [Part C – Ansible Inventory & Connectivity Test](#)
 - [Create Inventory File](#)
 - [Test Connectivity with `ansible ping`](#)
 - [Part D – Writing and Running Playbooks](#)
 - [Example Playbook: `playbook1.yml`](#)
 - [Running the Playbook](#)
 - [Ad-Hoc Command to Verify](#)
 - [Cleanup](#)
 - [Key Takeaways](#)
-

What is Ansible?

Ansible is an open-source automation tool for configuration management, application deployment, and orchestration. It follows an **agentless architecture** (uses SSH for Linux, WinRM for Windows) and uses **YAML-based playbooks** to define automation tasks.

Key Concepts

Concept	Description
Control Node	Machine with Ansible installed (e.g., your WSL/Ubuntu)

Concept	Description
Managed Nodes	Target servers (no Ansible agent required)
Inventory	File listing managed nodes (e.g., <code>inventory.ini</code>)
Playbooks	YAML files containing sequences of automation tasks
Modules	Units of code Ansible executes (e.g., <code>ping</code> , <code>apt</code> , <code>copy</code>)
Idempotency	Running a playbook multiple times yields the same result

How Ansible Works

- **Agentless** – No extra software on managed nodes.
- **Push-based** – Control node initiates changes over SSH.
- **Declarative** – You describe the desired state; Ansible figures out the steps.

Benefits of Using Ansible

- Free and open-source, battle-tested.
- Easy to start – no special coding skills.
- Simple deployment workflow, no agents.
- Idempotent, predictable, and secure.

Part A – Ansible Installation & Demo

1. Installation via apt (Debian/Ubuntu)

```
sudo apt update -y
sudo apt install ansible -y
ansible --version
```

```
arnav@LAPTOP-9TICA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/arnav/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/arnav/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

3. Post-Installation Test

Test with a local ping:

```
ansible localhost -m ping
```

Expected output:

```
localhost | SUCCESS => { "changed": false, "ping": "pong" }
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Part B – Setting Up Docker Containers as Managed Nodes

We will create a custom Ubuntu image with OpenSSH server, then run four containers as our managed servers.

1. Create SSH Key Pair in WSL

Generate a 4096-bit RSA key pair (accept default paths, empty passphrase):

```
ssh-keygen -t rsa -b 4096
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/arnav/.ssh/id_rsa):
Created directory '/home/arnav/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/arnav/.ssh/id_rsa
Your public key has been saved in /home/arnav/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:Xrp5N9g4SudoACLgEQV/MFKDxIrIUgMQCMLwKu1Pbyg arnav@LAPTOP-9T1CA4LD
The key's randomart image is:
+---[RSA 4096]-----+
|/B**
|+*+ +
|B..o .
|*+. o
|+... . S .
|.. .. o
|... ..+ .+
|Eo.....*+
|.....o.o .
+---[SHA256]-----+
```

Copy the public and private keys to the experiment directory:

```
cp ~/.ssh/id_rsa .
cp ~/.ssh/id_rsa.pub .
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ cp ~/.ssh/id_rsa .
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ cp ~/.ssh/id_rsa.pub .
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ ls
id_rsa id_rsa.pub
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ nano Dockerfile
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ cat dockerdile
cat: dockerdile: No such file or directory
```

2. Create Dockerfile for Ubuntu SSH Server

Create a **Dockerfile** with the following content:

```
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server
```

```

RUN mkdir -p /var/run/sshd

RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' \
/etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' \
/etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' \
/etc/ssh/sshd_config

RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh

COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional \
pam_loginuid.so@g' /etc/pam.d/sshd

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]

```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok_Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ cat Dockerfile
FROM ubuntu

RUN apt update -y
RUN apt install -y python3 python3-pip openssh-server

RUN mkdir -p /var/run/sshd

RUN mkdir -p /run/sshd && \
    echo 'root:password' | chpasswd && \
    sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config && \
    sed -i 's/#PubkeyAuthentication yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh

COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

RUN chmod 600 /root/.ssh/id_rsa && \
    chmod 644 /root/.ssh/authorized_keys

RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /etc/pam.d/sshd

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]

```

3. Build the Docker Image

```
docker build -t ubuntu-server .
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ docker build -t ubuntu-server .
[*] Building 91.4s (15/15) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 837B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.2s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                      0.0s
=> [internal] load build context                                  0.1s
=> => transferring context: 4.21kB                                  0.0s
=> [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:d1e2e92c075e5 0.1s
=> => resolve docker.io/library/ubuntu:latest@sha256:d1e2e92c075e5ca 0.0s
=> [ 2/10] RUN apt update -y                                       19.9s
=> [ 3/10] RUN apt install -y python3 python3-pip openssh-server 56.8s
=> [ 4/10] RUN mkdir -p /var/run/ssh                                0.3s
=> [ 5/10] RUN mkdir -p /run/ssh && echo 'root:password' | chpasswd 0.3s
=> [ 6/10] RUN mkdir -p /root/.ssh && chmod 700 /root/.ssh         0.2s
=> [ 7/10] COPY id_rsa /root/.ssh/id_rsa                          0.1s
=> [ 8/10] COPY id_rsa.pub /root/.ssh/authorized_keys             0.1s
=> [ 9/10] RUN chmod 600 /root/.ssh/id_rsa && chmod 644 /root/.ssh/ 0.2s
=> [10/10] RUN sed -i 's@session\\s*required\\s*pam_loginuid.so@sess 0.3s
=> => exporting to image                                           21.7s
=> => exporting layers                                             16.3s
=> => exporting manifest sha256:ed68f3b2aca7a47ba46485986bacc947e9c0 0.0s
=> => exporting config sha256:bb29830cb4a218ae69c5e73e390597f35ffb 0.0s
=> => exporting attestation manifest sha256:5447f29e708c9666c2ee545e 0.0s
=> => exporting manifest list sha256:cab49d7f5f334e9c34c4e622bcd17ae 0.0s
=> => naming to docker.io/library/ubuntu-server:latest            0.0s
=> => unpacking to docker.io/library/ubuntu-server:latest         5.3s

```

4. Run Four SSH Server Containers

Start four containers with mapped SSH ports 2201–2204:

```

for i in {1..4}; do
    echo "Creating server${i} on port 220${i}"
    docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
done

```

Verify containers are running:

```
docker ps
```

```

arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
b6ebf674ce16   ubuntu-server  "/usr/sbin/sshd -D"      7 seconds ago Up 7 seconds  0.0.0.0:2204->22/tcp, [::]:2204->22/tcp  server4
787a19b44842   ubuntu-server  "/usr/sbin/sshd -D"      8 seconds ago Up 7 seconds  0.0.0.0:2203->22/tcp, [::]:2203->22/tcp  server3
855059ab6a96   ubuntu-server  "/usr/sbin/sshd -D"      8 seconds ago Up 7 seconds  0.0.0.0:2202->22/tcp, [::]:2202->22/tcp  server2
3b5b893d37b9   ubuntu-server  "/usr/sbin/sshd -D"      8 seconds ago Up 8 seconds  0.0.0.0:2201->22/tcp, [::]:2201->22/tcp  server1
f33b23f27455   task5-nodeapp-nodeapp  "docker-entrypoint.s..." 2 hours ago   Up 2 hours    0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  custom-node-app
41dce334009f   nginx:alpine    "docker-entrypoint.s..." 2 hours ago   Up 2 hours    0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp  limited-app
0a5250c213a4   node:18-alpine  "docker-entrypoint.s..." 2 hours ago   Up 2 hours    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  webapp
fa0816c715a0   wordpress:latest  "docker-entrypoint.s..." 2 hours ago   Up 2 hours    0.0.0.0:8082->80/tcp, [::]:8082->80/tcp  wordpress

```

Part C – Ansible Inventory & Connectivity Test

Step 2: Create Ansible Inventory

Create `inventory.ini` with the four servers (using localhost and mapped ports):

```

[servers]
server1  ansible_host=localhost  ansible_port=2201
server2  ansible_host=localhost  ansible_port=2202
server3  ansible_host=localhost  ansible_port=2203
server4  ansible_host=localhost  ansible_port=2204

[servers:vars]
ansible_user=root

```



```
ansible_ssh_private_key_file=./id_rsa
ansible_python_interpreter=/usr/bin/python3
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-9$ cat inventory.ini
[servers]
server1 ansible_host=localhost ansible_port=2201
server2 ansible_host=localhost ansible_port=2202
server3 ansible_host=localhost ansible_port=2203
server4 ansible_host=localhost ansible_port=2204

[servers:vars]
ansible_user=root
ansible_ssh_private_key_file=./id_rsa
ansible_python_interpreter=/usr/bin/python3
```

Step 4: Test Connectivity

Run an Ansible ping to all servers:

```
ansible all -i inventory.ini -m ping
```

You may be prompted to accept host keys; answer **yes**. The output should show **"ping": "pong"** for each server.

```
arnav@LAPTOP-9T1CA4LD:~/Experiment-9$ ansible all -i inventory.ini -m ping
The authenticity of host '[localhost]:2201 ([127.0.0.1]:2201)' can't be established.
ED25519 key fingerprint is SHA256:dxbhPLBjcOmJBhtd3Ylt6CGf0QFDrfa8us2MAZ76vZo.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
The authenticity of host '[localhost]:2203 ([127.0.0.1]:2203)' can't be established.
ED25519 key fingerprint is SHA256:dxbhPLBjcOmJBhtd3Ylt6CGf0QFDrfa8us2MAZ76vZo.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
The authenticity of host '[localhost]:2204 ([127.0.0.1]:2204)' can't be established.
ED25519 key fingerprint is SHA256:dxbhPLBjcOmJBhtd3Ylt6CGf0QFDrfa8us2MAZ76vZo.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? server2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
yes
server1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
yes
server3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
yees
Please type 'yes', 'no' or the fingerprint: yes
server4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
```

Part D – Writing and Running Playbooks

Example Playbook: **playbook1.yml**

Create a playbook that:

- Updates apt cache
- Installs packages (**vim**, **curl**, **wget**)
- Creates a test file on each server

```

---
- name: Configure multiple servers
  hosts: servers
  become: yes

  tasks:
    - name: Update apt package index
      apt:
        update_cache: yes

    - name: Install packages
      apt:
        name: ["vim", "curl", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on {{ inventory_hostname }}"

```

```

arnav@LAPTOP-9T1CA4LD:~/Experiment-9$ cat playbook1.yml
---
- name: Configure multiple servers
  hosts: servers
  become: yes

  tasks:
    - name: Update apt packages
      apt:
        update_cache: yes

    - name: Install packages
      apt:
        name: ["vim", "htop", "wget"]
        state: present

    - name: Create test file
      copy:
        dest: /root/ansible_test.txt
        content: "Configured by Ansible on {{ inventory_hostname }}"

```

Running the Playbook

```
ansible-playbook -i inventory.ini playbook1.yml
```

Ad-Hoc Command to Verify

Check the content of the created file on all servers:

```
ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
```

```

arnav@LAPTOP-9T1CA4LD:~/Experiment-9$ ansible all -i inventory.ini -m command -a "cat /root/ansible_test.txt"
server4 | CHANGED | rc=0 >>
Configured by Ansible on server4
server2 | CHANGED | rc=0 >>
Configured by Ansible on server2
server1 | CHANGED | rc=0 >>
Configured by Ansible on server1
server3 | CHANGED | rc=0 >>
Configured by Ansible on server3
arnav@LAPTOP-9T1CA4LD:~/Experiment-9$ |

```

Each server shows its own name, proving that Ansible configured them individually.

Cleanup

Stop and remove the four server containers:

```
for i in {1..4}; do
  docker stop server${i}
done
```

Remove the Docker image (optional):

```
docker rmi ubuntu-server
```

Key Takeaways

- **Ansible is agentless** – only requires SSH access to managed nodes.
 - **Inventory** defines which hosts to manage.
 - **Playbooks** (YAML) are declarative and idempotent.
 - **Modules** like `ping`, `apt`, `copy` provide reusable functionality.
 - **Ad-hoc commands** are great for quick checks (`ansible -m command -a`).
 - Using Docker containers as managed nodes is an excellent way to learn Ansible without physical or cloud servers.
-

This experiment demonstrates how Ansible solves configuration drift and enables scalable, consistent server management.

Author: Arnav

****Instructions for GitHub:****

1. Create an ``Images/`` folder inside ``Experiment-9/``.
2. Copy all screenshot files (``1.png`` ... ``10.png``, ``12.png``, and optionally the folder structure image if you want to include it) into ``Images/``.
3. Place the ``Dockerfile``, ``id_rsa``, ``id_rsa.pub``, ``inventory.ini``, and ``playbook1.yml`` in the same directory (the keys can be omitted from the repository for security; the README explains how to generate them).
4. Push to GitHub.

Now your Ansible experiment documentation is ready, properly formatted, and includes all referenced images.

Experiment 10: Static Code Analysis with SonarQube

Static Code Analysis with SonarQube

This experiment demonstrates how to set up SonarQube (a continuous code quality inspection tool) using Docker, integrate it with a Maven-based Java project, and analyse the code for bugs, vulnerabilities, and code smells.

Table of Contents

- [Overview](#)
 - [Prerequisites](#)
 - [Step 1 – Start SonarQube Server with Docker](#)
 - [Step 2 – Prepare the Java Maven Project](#)
 - [Step 3 – Configure SonarQube Project and Generate Token](#)
 - [Step 4 – Run SonarQube Analysis with Maven](#)
 - [Step 5 – View Analysis Results](#)
 - [Step 6 – Understand the Issues Found](#)
 - [Cleanup](#)
 - [Key Takeaways](#)
-

Overview

SonarQube is an open-source platform for continuous inspection of code quality. It detects bugs, vulnerabilities, code smells, and security hotspots in over 25 programming languages. In this experiment we:

- Run SonarQube Community Edition inside a Docker container.
- Analyse a simple Java Maven project ([sample-java-app](#)).
- Use the SonarQube Maven plugin to push analysis results.
- Explore the SonarQube dashboard.

Prerequisites

- Docker installed and running.
 - Java (JDK 8 or later) and Maven installed (or use the Maven wrapper).
 - Basic knowledge of Maven projects.
-

Step 1 – Start SonarQube Server with Docker

SonarQube requires a database; for evaluation we use the embedded H2 database (not for production).

```
docker run -d --name sonar -p 9001:9000 sonarqube:lts
```

Verify the container is running:

```
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d502c780d117	sonarqube:lts	"/opt/sonarqube/dock..."	28 minutes ago	Up 28 minutes	0.0.0.0:9001->9000/tcp, [::]:9001->9000/tcp	sonar
b6ebf674ce16	ubuntu-server	"/usr/sbin/sshd -D"	About an hour ago	Up About an hour	0.0.0.0:2204->22/tcp, [::]:2204->22/tcp	server4
787a19b44842	ubuntu-server	"/usr/sbin/sshd -D"	About an hour ago	Up About an hour	0.0.0.0:2203->22/tcp, [::]:2203->22/tcp	server3
855059ab6a96	ubuntu-server	"/usr/sbin/sshd -D"	About an hour ago	Up About an hour	0.0.0.0:2202->22/tcp, [::]:2202->22/tcp	server2
3b5b893d37b9	ubuntu-server	"/usr/sbin/sshd -D"	About an hour ago	Up About an hour	0.0.0.0:2201->22/tcp, [::]:2201->22/tcp	server1
f33b23f27455	task5-nodeapp-nodeapp	"docker-entrypoint.s..."	3 hours ago	Up 3 hours	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	custom-
node-app						
41dce334009f	nginx:alpine	"/docker-entrypoint.s..."	3 hours ago	Up 3 hours	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp	limited
-app						
0a5250c213a4	node:18-alpine	"docker-entrypoint.s..."	3 hours ago	Up 3 hours	0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp	webapp
fa0816c715a0	wordpress:latest	"docker-entrypoint.s..."	4 hours ago	Up 4 hours	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp	wordpre
ss						
680e669d4622	mysql:5.7	"docker-entrypoint.s..."	4 hours ago	Up 4 hours	3306/tcp, 33060/tcp	mysql

Access SonarQube web interface at:

http://localhost:9001

Default login:

- Username: **admin**
- Password: **admin**

You will be prompted to change the password immediately (optional for this experiment).

The screenshot shows the SonarQube web interface. At the top, there is a navigation bar with links for Projects, Issues, Rules, Quality Profiles, Quality Gates, and Administration. A search bar is also present. Below the navigation bar, the user profile for 'admin' is displayed. The profile includes sections for Login (username: admin), Groups (sonar-administrators, sonar-users), SCM Accounts (admin), and Preferences. The 'Enable Keyboard Shortcuts' preference is shown as a toggle switch that is currently turned on.

Step 2 – Prepare the Java Maven Project

The project structure is shown below:

```
sample-java-app/  
├── pom.xml  
├── src/  
│   ├── main/java/com/example/App.java  
│   └── test/java/com/example/AppTest.java  
└── target/ (generated after build)
```

```
arnav@LAPTOP-9TICA4LD: /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-10/sample-java-app/sample-java-app$ ls -R
.:
pom.xml

./src:
main test

./src/main:
java

./src/main/java:
com

./src/main/java/com:
example

./src/main/java/com/example:
App.java

./src/test:
java

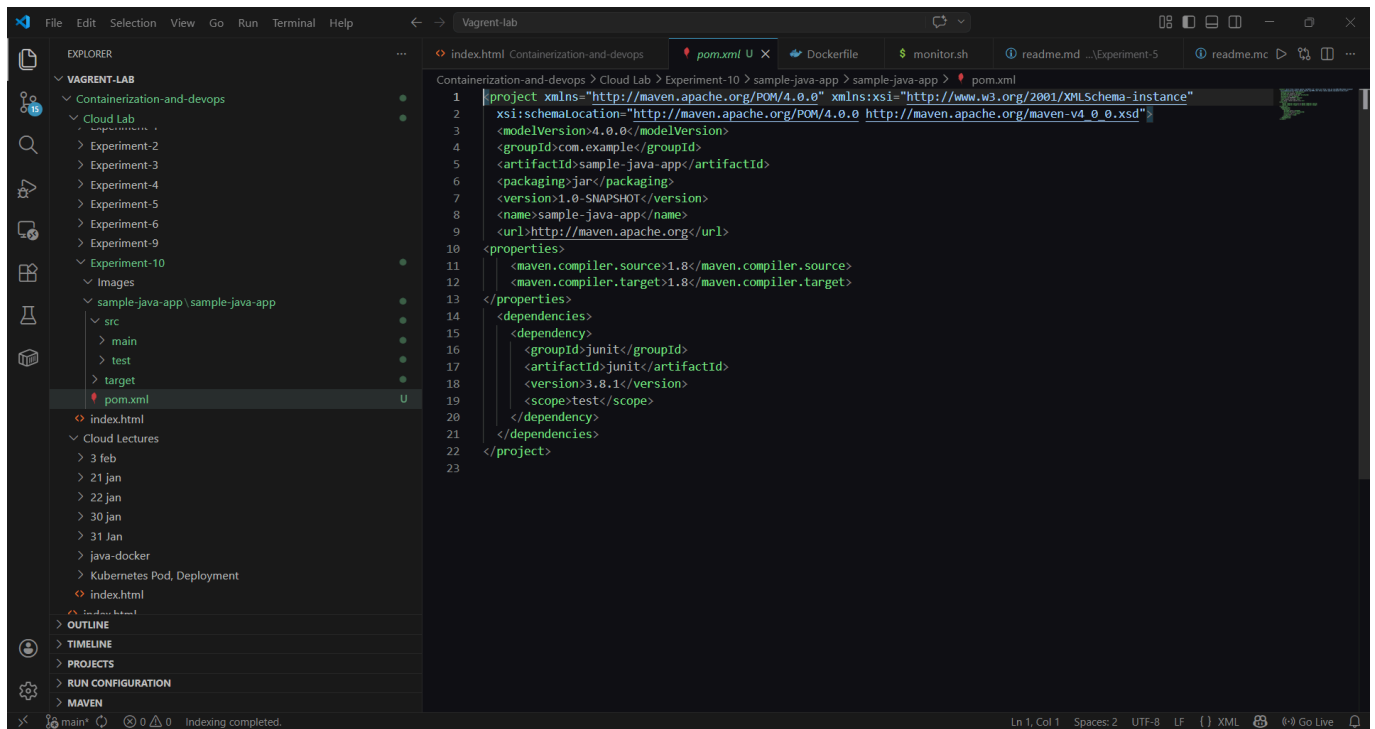
./src/test/java:
com

./src/test/java/com:
example

./src/test/java/com/example:
AppTest.java
```

pom.xml (Maven configuration) – includes the SonarQube plugin dependency:

```
<project ...>
  <groupId>com.example</groupId>
  <artifactId>sample-java-app</artifactId>
  <version>1.0-SNAPSHOT</version>
  ...
  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

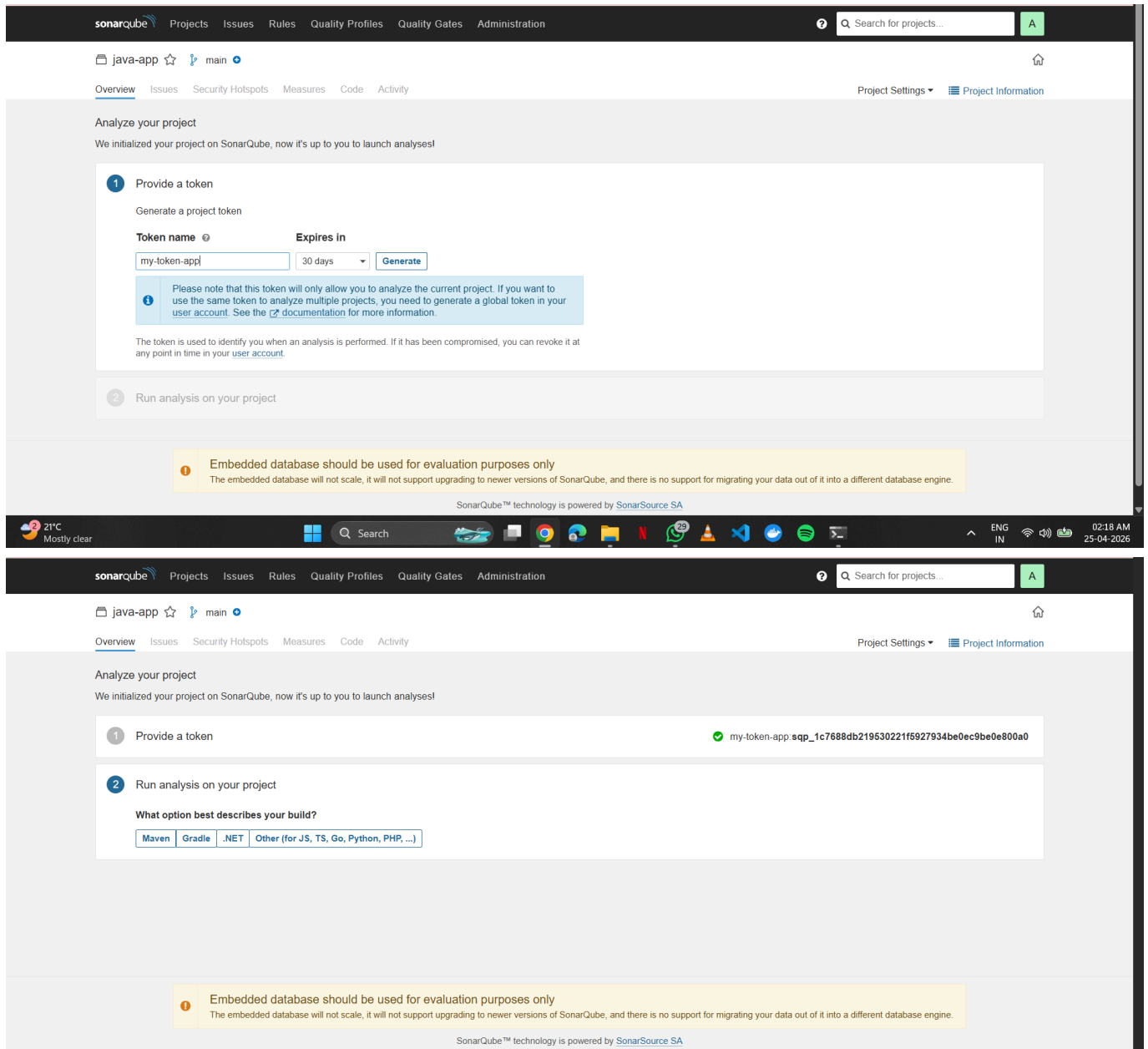


Step 3 – Configure SonarQube Project and Generate Token

1. After logging in, go to **Projects** → **Create Project**.
2. Choose **Manually** → Give project key **java-app** and display name **sample-java-app**.
3. In the analysis method, choose **Locally** → **Maven**.
4. Generate a project token (e.g., name **my-token-app**, expiration 30 days).

Copy the token – it will look like:

sqp_1c7688db219530221f5927934be0ec9be0e800a0



Step 4 – Run SonarQube Analysis with Maven

From the root directory of your Maven project (`sample-java-app/`), execute:

```
mvn clean verify sonar:sonar \
-Dsonar.projectKey=java-app \
-Dsonar.host.url=http://localhost:9001 \
-Dsonar.login=YOUR_GENERATED_TOKEN
```

Replace `YOUR_GENERATED_TOKEN` with the token you copied.

Maven will:

- Compile the code
- Run tests (`verify`)
- Execute the SonarQube scanner and send the report to the server


```

arnav@LAPTOP-9TICA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-10/sample-java-app/sample-java-app$ mvn clean verify sonar:sonar \
-Dsonar.projectKey=java-app \
-Dsonar.host.url=http://localhost:9001 \
-Dsonar.login=sqp_1c7688db219530221f5927934be0ec9be0e800a0
[INFO] Scanning for projects...
[WARNING] The artifact org.codehaus.mojo:sonar-maven-plugin:jar:4.0.0.4121 has been relocated to org.sonarsource.scanner.maven:sonar-maven-plugin:jar:4.0.0.4121: SonarQube plugin was moved to SonarSource organisation
[INFO] -----< com.example:sample-java-app >-----
[INFO] Building sample-java-app 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ sample-java-app ---
[INFO] Deleting /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-10/sample-java-app/sample-java-app/target
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ sample-java-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-10/sample-java-app/sample-java-app/src/main/resources
[INFO] --- maven-compiler-plugin:3.1:compile (default-compile) @ sample-java-app ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-10/sample-java-app/sample-java-app/target/classes
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ sample-java-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-10/sample-java-app/sample-java-app/src/test/resources
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ sample-java-app ---

```

If successful, you will see:

```

[INFO] ANALYSIS SUCCESSFUL, you can find the results at:
http://localhost:9001/dashboard?id=java-app
[INFO] BUILD SUCCESS

```

```

[INFO] 20:55:29.160 Sensor VB.NET Properties [vbnet] (done) | time=0ms
[INFO] 20:55:29.160 Sensor IaC Docker Sensor [iac]
[INFO] 20:55:29.162 0 source files to be analyzed
[INFO] 20:55:29.198 0/0 source files have been analyzed
[INFO] 20:55:29.198 Sensor IaC Docker Sensor [iac] (done) | time=38ms
[INFO] 20:55:29.201 ----- Run sensors on project
[INFO] 20:55:29.237 Sensor Analysis Warnings import [csharp]
[INFO] 20:55:29.238 Sensor Analysis Warnings import [csharp] (done) | time=1ms
[INFO] 20:55:29.238 Sensor Zero Coverage Sensor
[INFO] 20:55:29.249 Sensor Zero Coverage Sensor (done) | time=10ms
[INFO] 20:55:29.249 Sensor Java CPD Block Indexer
[INFO] 20:55:29.261 Sensor Java CPD Block Indexer (done) | time=12ms
[INFO] 20:55:29.263 SCM Publisher SCM provider for this project is: git
[INFO] 20:55:29.264 SCM Publisher 3 source files to be analyzed
[WARNING] 20:55:29.315 Thread[#72,JGit-FileStoreAttributeReader-1.5,main]: got smaller file timestamp on /mnt/c (C:\), /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/.git: 2026-04-24T20:55:29Z < 2026-04-24T20:55:29.3142503Z. Aborting measurement at resolution PT0.68574975s.
[INFO] 20:55:29.484 SCM Publisher 0/3 source files have been analyzed (done) | time=220ms
[WARNING] 20:55:29.485 Missing blame information for the following files:
[WARNING] 20:55:29.485 * src/main/java/com/example/App.java
[WARNING] 20:55:29.485 * src/test/java/com/example/AppTest.java
[WARNING] 20:55:29.485 * pom.xml
[WARNING] 20:55:29.485 This may lead to missing/broken features in SonarQube
[INFO] 20:55:29.488 CPD Executor 1 file had no CPD blocks
[INFO] 20:55:29.488 CPD Executor Calculating CPD for 0 files
[INFO] 20:55:29.488 CPD Executor CPD calculation finished (done) | time=0ms
[INFO] 20:55:29.651 Analysis report generated in 110ms, dir size=128.0 kB
[INFO] 20:55:29.943 Analysis report compressed in 290ms, zip size=19.6 kB
[INFO] 20:55:30.010 Analysis report uploaded in 66ms
[INFO] 20:55:30.013 ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9001/dashboard?id=java-app
[INFO] 20:55:30.013 Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] 20:55:30.013 More about the report processing at http://localhost:9001/api/ce/task?id=AZ3BRrXPxcMFhDXThkq
[INFO] 20:55:30.073 Analysis total time: 5.926 s
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.736 s
[INFO] Finished at: 2026-04-24T20:55:30Z

```

Step 5 – View Analysis Results

Go back to **http://localhost:9001**. The project dashboard shows:

- **Quality Gate** status (Passed / Failed)
- **Bugs, Vulnerabilities, Code Smells, Coverage, Duplications**

The screenshot shows the SonarQube interface with the 'Projects' tab selected. The left sidebar contains filters for Quality Gate (Passed: 1, Failed: 0), Reliability (A rating: 1, B: 0, C: 0, D: 0, E: 0), Security (A rating: 1, B: 0, C: 0, D: 0, E: 0), and Security Review (Security Hotspots: 0). The main area displays the project 'sample-java-app' with a 'Passed' status. The analysis summary shows 0 Bugs, 0 Vulnerabilities, 1 Code Smell (A rating), 0.0% Coverage, 0.0% Duplications, and 30 Lines of code. A warning message at the bottom states: 'Embedded database should be used for evaluation purposes only. The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.'

In this sample project, the analysis found:

- 0 Bugs
- 0 Vulnerabilities
- 1 Code Smell (major severity)

Step 6 – Understand the Issues Found

Click on **Issues** to see detailed information.

The single code smell is located in **App.java** (line 11):

Replace this use of `System.out` or `System.err` by a logger.

SonarQube recommends using a logging framework (e.g., SLF4J) instead of `System.out.println()` for production code.

The screenshot shows the SonarQube 'Issues' tab. The left sidebar contains filters for Type (Bug: 0, Vulnerability: 0, Code Smell: 1), Severity (Blocker: 0, Critical: 0, Major: 1, Minor: 0, Info: 0), Scope, Resolution, Status, Security Category, Creation Date, Language, Rule, Tag, and Project. The main area displays a single issue: 'Replace this use of System.out or System.err by a logger.' located in 'sample-java-app / src/main/java/com/example/App.java'. The issue is a 'Code Smell' with 'Major' severity, 'Open' status, and '10min effort'. A warning message at the bottom states: 'Embedded database should be used for evaluation purposes only. The embedded database will not scale, it will not support upgrading to newer versions of SonarQube, and there is no support for migrating your data out of it into a different database engine.'

This demonstrates how SonarQube helps enforce coding standards and maintainability.

Cleanup

Stop and remove the SonarQube container (all data will be lost because we used the embedded H2 database):

```
docker stop sonar
docker rm sonar
```

If you want to remove the container automatically when stopped, add `--rm` to the `docker run` command.

Key Takeaways

- **SonarQube** provides continuous code quality inspection.
 - Running it in **Docker** is quick and isolated.
 - The **Maven plugin** integrates seamlessly with Java projects.
 - The dashboard gives **actionable metrics** (bugs, vulnerabilities, code smells).
 - Even a simple “Hello World” project can reveal real-world issues (e.g., using `System.out`).
 - For production, use an external database (PostgreSQL) and persistent volumes.
-

This experiment shows how to incorporate static analysis into a development workflow using SonarQube and Docker.

Author: Arnav

Date: April 25, 2026

```
**Instructions for GitHub:**
```

1. Inside ``Experiment-10/``, create an ``Images/`` folder.
2. Copy all screenshot files (``1.png` ... `10.png``) into ``Images/``.
3. Also place the ``sample-java-app`` folder (with ``pom.xml``, ``src/``, etc.) in the same directory.
4. Push the and the project files.

Now your SonarQube experiment documentation is complete and matches the images you provided.

Experiment 11: Orchestration using Docker Swarm

Orchestration using Docker Swarm

📁 Table of Contents

- [Part A – Orchestration Concepts](#part-a--orchestration-concepts)
- [Part B – Practical Steps](#part-b--practical-steps)
 - [Task 1: Check Current State (No Swarm)](#task-1-check-current-state-no-swarm)
 - [Task 2: Initialize Docker Swarm](#task-2-initialize-docker-swarm)
 - [Task 3: Deploy as a Stack](#task-3-deploy-as-a-stack)
 - [Task 4: Verify the Deployment](#task-4-verify-the-deployment)
 - [Task 5: Access WordPress](#task-5-access-wordpress)
 - [Task 6: Scale the Application](#task-6-scale-the-application)
 - [Task 7: Test Self-Healing](#task-7-test-self-healing)
 - [Task 8: Remove the Stack](#task-8-remove-the-stack)
- [Part C – Compose vs Swarm Analysis](#part-c--compose-vs-swarm-analysis)
- [Part D – Important Observations](#part-d--important-observations)
- [Learning Outcomes Check](#learning-outcomes-check)
- [Quick Reference Card](#quick-reference-card)
- [Cleanup](#cleanup)

Part A – Orchestration Concepts

****Orchestration**** = automatic management of containers.

It adds:

- ****Scaling**** – increase/decrease number of containers
- ****Self-healing**** – restart failed containers automatically
- ****Load balancing**** – distribute traffic across containers
- ****Multi-host**** – run containers across multiple machines

****Progression path:****

`docker run` → Docker Compose → ****Docker Swarm**** → Kubernetes

This experiment focuses on moving from ****Compose**** → ****Swarm****.

Part B – Practical Steps

Prerequisites

- Docker installed (with Swarm mode enabled)

- The `docker-compose.yml` file from Experiment 6 (WordPress + MySQL)

> If you don't have the file, it is reproduced below.

Task 1: Check Current State (No Swarm)

Ensure no containers from Experiment 6 are running:

```
```bash
docker compose down -v
docker ps
```

Expected: Only unrelated containers (if any).

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
41dce334009f nginx:alpine "/docker-entrypoint.s..." 6 days ago Up 26 seconds 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp limited-app
0a5250c213a4 node:18-alpine "/docker-entrypoint.s..." 6 days ago Up 26 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp webapp
fa0816c715a0 wordpress:latest "/docker-entrypoint.s..." 6 days ago Up 26 seconds 0.0.0.0:8082->80/tcp, [::]:8082->80/tcp wordpress
680e669d4622 mysql:5.7 "/docker-entrypoint.s..." 6 days ago Up 26 seconds 3306/tcp, 33060/tcp mysql
```

## Task 2: Initialize Docker Swarm

```
docker swarm init
```

This enables Swarm mode and makes the current node a manager.

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker swarm init
Swarm initialized: current node (9r2ndr63eakkcwaqhb3uo9iby) is now a manager.

To add a worker to this swarm, run the following command:

 docker swarm join --token SWMTKN-1-252a0grtwv8y12cg3hmdm8eatgba6roore9u79la4i8eswqqi2-clbx0ru7pm7lhuz5oqzexwlge 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

Verify Swarm is active:

```
docker node ls
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker node ls
ID HOSTNAME STATUS AVAILABILITY MANAGER STATUS ENGINE VERSION
9r2ndr63eakkcwaqhb3uo9iby * docker-desktop Ready Active Leader 29.1.5
```

## Task 3: Deploy as a Stack

Use the same Compose file to deploy a **stack** (Swarm's name for a group of services).

```
docker stack deploy -c docker-compose.yml wpstack
```

```

arnav@LAPTOP-9T1CA4LD: /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_db
Creating service wpstack_wordpress

```

The Compose file (same as Experiment 6):

```

version: '3.9'
services:
 db:
 image: mysql:5.7
 container_name: wordpress_db # ignored by Swarm
 restart: always
 environment:
 MYSQL_ROOT_PASSWORD: rootpass
 MYSQL_DATABASE: wordpress
 MYSQL_USER: wpuser
 MYSQL_PASSWORD: wppass
 volumes:
 - db_data:/var/lib/mysql
 wordpress:
 image: wordpress:latest
 container_name: wordpress_app # ignored by Swarm
 depends_on: [db]
 ports:
 - "8080:80"
 restart: always
 environment:
 WORDPRESS_DB_HOST: db:3306
 WORDPRESS_DB_USER: wpuser
 WORDPRESS_DB_PASSWORD: wppass
 WORDPRESS_DB_NAME: wordpress
 volumes:
 - wp_data:/var/www/html
volumes:
 db_data:
 wp_data:

```

Swarm ignores `container_name` and `restart` (it handles restart policies differently) but accepts the rest.

## Task 4: Verify the Deployment

List all services in the stack:

```
docker service ls
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
uq15e137mupc	wpstack_db	replicated	1/1	mysql:5.7	
489wrvb7gvt7	wpstack_wordpress	replicated	1/1	wordpress:latest	*:8080->80/tcp

See detailed tasks (containers) for a service:

```
docker service ps wpstack_wordpress
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker service ps wpstack_wordpress
```

ID	NAME	IMAGE	MODE	DESIRED STATE	CURRENT STATE	ERROR	PORT
7zt87rgqw4y0	wpstack_wordpress.1	wordpress:latest	docker-desktop	Running	Running 6 minutes ago		
rrhln9qicvi5	wpstack_wordpress.2	wordpress:latest	docker-desktop	Running	Running 2 seconds ago		
y886j1j17pxk	\_ wpstack_wordpress.2	wordpress:latest	docker-desktop	Shutdown	Failed 7 seconds ago	"task: non-zero exit (137)"	
br4o1ymuhbd0	wpstack_wordpress.3	wordpress:latest	docker-desktop	Running	Running about a minute ago		

See all running containers:

```
docker ps | grep wordpress
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker ps | grep wordpress
```

ID	NAME	IMAGE	ENTRYPOINT	STATE	UP	PORTS	IP
d3ee2aabe25b	wordpress:latest	"docker-entrypoint.s_"	34 seconds ago	Up 29 seconds	80/tcp	wpstack_wordpress.2.rrhln9qicvi5ih	
71mrmlaasw	wordpress:latest	"docker-entrypoint.s_"	About a minute ago	Up About a minute	80/tcp	wpstack_wordpress.3.br4o1ymuhbd0eq	
7fca32a90cc0	wordpress:latest	"docker-entrypoint.s_"	6 minutes ago	Up 6 minutes	80/tcp	wpstack_wordpress.1.7zt87rgqw4y0pi	

Observation: Containers are now named by Swarm (e.g., `wpstack_wordpress.1.xxx`).

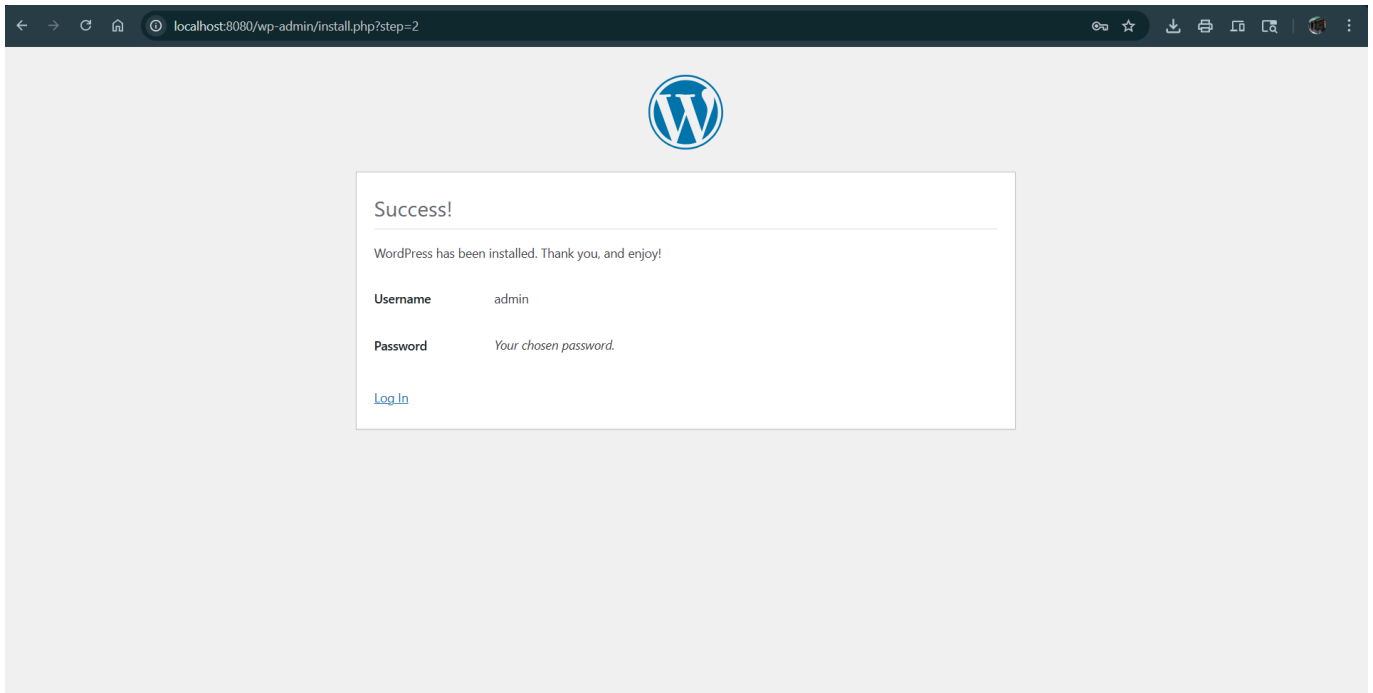
## Task 5: Access WordPress

Open your browser: **`http://localhost:8080`**

You will see the WordPress installation screen (same as Experiment 6).

After completing the installation, you get the success page.





## Task 6: Scale the Application

This is where Swarm shines: one command scales WordPress from 1 to 3 replicas.

```
docker service scale wpstack_wordpress=3
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
```

Verify the new replica count:

```
docker service ls
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
uq15e137mupc wpstack_db replicated 1/1 mysql:5.7
489wrvb7gvt7 wpstack_wordpress replicated 3/3 wordpress:latest *:8080->80/tcp
```

Check that 3 WordPress containers are running:

```
docker ps | grep wordpress
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker ps | grep wordpress
75e66ef7a529 wordpress:latest "docker-entrypoint.s..." 45 seconds ago Up 44 seconds 80/tcp wpstack_wordpress.2.y886j1j17pxkulfdvj1jtp
f1f wordpress:latest "docker-entrypoint.s..." 45 seconds ago Up 44 seconds 80/tcp wpstack_wordpress.3.br4olymuhbd0eqkw9q9dr1
qke wordpress:latest "docker-entrypoint.s..." 5 minutes ago Up 5 minutes 80/tcp wpstack_wordpress.1.7zt87rgqw4y0pi1dbgwbyu
gxt
```

## How do 3 containers share port 8080?

Swarm creates an internal **load balancer**. All traffic to **localhost:8080** is distributed automatically across the

three containers – no port conflicts.

## Task 7: Test Self-Healing

Swarm automatically replaces failed containers.

1. Find a WordPress container ID:

```
docker ps | grep wordpress
```

2. Kill it (simulate a crash):

```
docker kill <container-id>
```

3. Watch Swarm recreate it:

```
docker service ps wpstack_wordpress
```

You will see the killed container marked as **Shutdown Failed** and a new container created automatically.

```
arnav@LAPTOP-9T1CA4LD: /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker service ps wpstack_wordpress
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORT
7zt87rgqw4y0	wpstack_wordpress.1	wordpress:latest	docker-desktop	Running	Running 6 minutes ago		
rrhln9qicvi5	wpstack_wordpress.2	wordpress:latest	docker-desktop	Running	Running 2 seconds ago		
y886j1jl7pxk	_ wpstack_wordpress.2	wordpress:latest	docker-desktop	Shutdown	Failed 7 seconds ago	"task: non-zero exit (137)"	
br4oiymuhbd0	wpstack_wordpress.3	wordpress:latest	docker-desktop	Running	Running about a minute ago		

4. Verify still 3 containers running:

```
docker ps | grep wordpress
```

Swarm fixed it without any manual intervention!

## Task 8: Remove the Stack

Clean up everything:

```
docker stack rm wpstack
```

```
arnav@LAPTOP-9T1CA4LD: /mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker stack rm wpstack
Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
```

Verify removal:

```
docker service ls
docker ps
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrent-lab/Containerization-and-devops/Cloud Lab/Experiment-11$ |
```

Volumes persist unless you delete them manually with `docker volume prune`.

## Part C – Compose vs Swarm Analysis

Feature	Docker Compose	Docker Swarm
Scope	Single host only	Multi-node cluster
Scaling	<code>--scale</code> (basic, no balancing)	<code>docker service scale</code> (built-in)
Load Balancing	No (port conflicts when scaling)	Yes (internal ingress network)
Self-Healing	No (only <code>restart: always</code> on same host)	Yes (re-creates failed tasks)
Rolling Updates	No	Yes (zero downtime)
Service Discovery	Via container names	Via DNS + Virtual IP
Use Case	Development, testing	Simple production clusters
Complexity	Low	Medium

### When to Use What?

- **Docker Compose** → local development, CI pipelines, single-host demos.
- **Docker Swarm** → small- to medium-scale production, easy scaling, self-healing needs.
- **Kubernetes** → large-scale, complex microservices, advanced scheduling.

## Part D – Important Observations

### Observation 1: Compose File Reuse

The **same `docker-compose.yml`** works for both Compose and Swarm. Swarm extends Compose, it does not replace it.

### Observation 2: Containers vs Services

- **Container** – a single running instance.
- **Service** – a definition of how to run containers (image, replicas, ports, etc.).  
In Swarm, you manage **services**, not individual containers.

### Observation 3: The Port Mystery Solved

- In Compose, scaling WordPress to 3 would fail because all three would try to bind to host port **8080**.
  - In Swarm, the **load balancer** listens once on **8080** and distributes traffic internally – no conflicts.
- 

## Learning Outcomes Check

Answer these questions in your lab book:

1. Why is Compose not enough for production?
  2. What does **docker stack deploy** do differently than **docker compose up**?
  3. How does Swarm achieve self-healing?
  4. What happens if you run **docker kill** on a container managed by Swarm?
  5. Can you use the same Compose file for both development (Compose) and production (Swarm)? Why?
- 

## Quick Reference Card

```
Initialize Swarm
docker swarm init

Deploy a stack
docker stack deploy -c docker-compose.yml <stack-name>

List services
docker service ls

Scale a service
docker service scale <stack-name_service-name>=<replicas>

See tasks of a service
docker service ps <service-name>

Remove a stack
docker stack rm <stack-name>

Leave Swarm (if needed)
docker swarm leave --force
```

---

## Cleanup

To fully clean up after the experiment:

```
docker stack rm wpstack
docker swarm leave --force
docker volume prune -f
```

---

## Summary

You started with	You can now do
Single container ( <code>docker run</code> )	Multi-container (Compose)
Manual scaling	One-command scaling ( <code>scale</code> )
Manual recovery	Automatic self-healing
Single host	Multi-host cluster ready

**Final takeaway:** Compose *defines* the application. Swarm *runs it reliably*.

*This experiment shows how Docker Swarm adds production-grade orchestration (scaling, self-healing, load balancing) to the same Compose file used for local development.*

**Author:** Arnav

**Date:** May 1, 2026

**\*\*Instructions for GitHub:\*\***

1. Inside `Experiment-11/``, create an `Images/`` folder.
2. Copy all screenshot files (``1.png` ... `14.png``) into `Images/``.
3. Also place the `docker-compose.yml`` file (the same one from Experiment 6) in the directory.
4. Push the `README.md`` and the project files.

Your Experiment 11 documentation is now complete and ready for GitHub.

# Experiment 12: Study and Analyse Container Orchestration using Kubernetes

## Study and Analyse Container Orchestration using Kubernetes

### Objective

Learn why Kubernetes is used, its basic concepts, and how to deploy, scale, and fix apps using Kubernetes commands.

### Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

### Core Kubernetes Concepts

Docker Concept	Kubernetes Equivalent	What it means
Container	Pod	A pod is a group of one or more containers. Smallest unit in K8s.

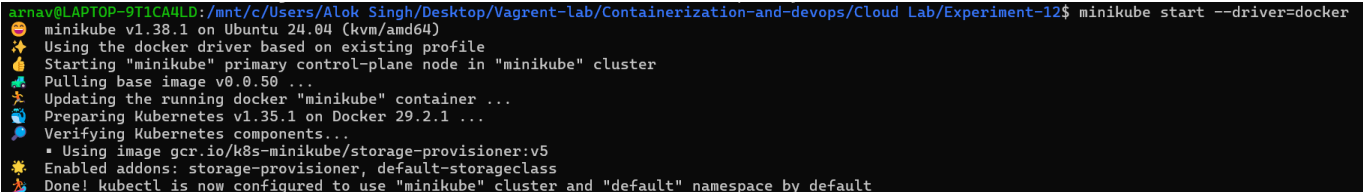
### Hands-On Lab (Using Minikube)

#### Prerequisites

- `kubectl` and a running Minikube cluster installed.

Start Minikube:

```
minikube start
```



Check cluster:

```
kubectl get nodes
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl get nodes
NAME STATUS ROLES AGE VERSION
minikube Ready control-plane 2m8s v1.35.1
```

## Task 1: Create a Deployment

Create `wordpress-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: wordpress
spec:
 replicas: 2
 selector:
 matchLabels:
 app: wordpress
 template:
 metadata:
 labels:
 app: wordpress
 spec:
 containers:
 - name: wordpress
 image: wordpress:latest
 ports:
 - containerPort: 80
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ nano wordpress-deployment.yaml
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ ls
Images wordpress-deployment.yaml
```

Apply:

```
kubectl apply -f wordpress-deployment.yaml
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
```

Check pods:

```
kubectl get pods
```

```
deployment.apps/wordpress created
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-12$ kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-6698dd7d66-mfkgc 0/1 ContainerCreating 0 10s
wordpress-6698dd7d66-s42jx 0/1 ContainerCreating 0 10s
```

## Task 2: Expose as a Service

Create `wordpress-service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
 name: wordpress-service
spec:
 type: NodePort
 selector:
 app: wordpress
 ports:
 - port: 80
 targetPort: 80
 nodePort: 30007
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containeri
zation-and-devops/Cloud Lab/Experiment-12$ ls
images wordpress-deployment.yaml wordpress-service.yaml
```

Apply:

```
kubectl apply -f wordpress-service.yaml
```

## Task 3: Verify

Check service:

```
kubectl get svc
```



```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
wordpress-service	NodePort	10.104.31.140	<none>	80:30007/TCP

Get access URL:

```
minikube service wordpress-service --url
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ minikube service wordpress-service --url
http://127.0.0.1:33533
! Because you are using a Docker driver on linux, the terminal needs to be open to run it.
```

Open in browser → WordPress installation screen.

#### Task 4: Scale the Deployment

Scale from 2 to 4 replicas:

```
kubectl scale deployment wordpress --replicas=4
```

```
^Carnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled
```

Verify:

```
kubectl get pods
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-mfkgc	1/1	Running	0	7m4s
wordpress-6698dd7d66-s42jx	1/1	Running	0	7m4s
wordpress-6698dd7d66-wdq7w	1/1	Running	0	25s

#### Task 5: Self-Healing Demonstration

Delete one pod manually:

```
kubectl delete pod <pod-name>
```

```
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl delete pod wordpress-6698dd7d66-mfkgc
pod "wordpress-6698dd7d66-mfkgc" deleted from default namespace
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ kubectl get pods
NAME READY STATUS RESTARTS AGE
wordpress-6698dd7d66-5ggdt 0/1 ContainerCreating 0 6s
wordpress-6698dd7d66-s42jx 1/1 Running 0 24m
wordpress-6698dd7d66-wdq7w 1/1 Running 0 17m
wordpress-6698dd7d66-zljfw 1/1 Running 0 17m
arnav@LAPTOP-9T1CA4LD:/mnt/c/Users/Alok Singh/Desktop/Vagrant-lab/Containerization-and-devops/Cloud Lab/Experiment-12$ |
```

Kubernetes automatically recreates the pod to maintain desired count.

## Swarm vs Kubernetes Comparison

Feature	Docker Swarm	Kubernetes
Setup	Very easy	More complex
Scaling	Basic	Advanced
Ecosystem	Small	Huge
Industry use	Rare	Standard

## Cheat Sheet

Command	Purpose
<code>kubectl apply -f &lt;file&gt;</code>	Create/update resources
<code>kubectl get pods</code>	List pods
<code>kubectl get svc</code>	List services
<code>kubectl scale deployment &lt;name&gt; --replicas=N</code>	Scale replicas
<code>kubectl delete pod &lt;pod-name&gt;</code>	Delete a pod
<code>minikube service &lt;service-name&gt; --url</code>	Get access URL

## Cleanup

```
kubectl delete -f wordpress-service.yaml
kubectl delete -f wordpress-deployment.yaml
minikube stop # optional
```

## End of Experiment

**\*\*Instructions:\*\***

- Place all images (`1.png` ... `11.png`) inside an `Images/` folder in `Experiment-12/`.
- Save this as `README.md`.
- Push to GitHub.